PENERBIT ANDI

robotika

teori + implementasi

- Pengantar robotika
- Mekanika robotika
- Pemrograman robotika
- Robotika vision

Widodo Budiharto

Robotika

TEORI DAN IMPLEMENTASI

Widodo Budiharto

Penerbit ANDI Yogyakarta

Robotika – Teori dan Implementasinya
Oleh: Widodo Budiharto

Hak Cipta © 2010 pada Penulis

Editor : Fl. Sigit Suyantoro
Setting : Sulis
Desain Cover : Bowo
Korektor : Marsi / Aktor Sadewa

Hak Cipta dilindungi undang-undang.
Dilarang memperbanyak atau memindahkan sebagian atau seluruh isi buku ini dalam bentuk apapun, baik secara elektronis maupun mekanis, termasuk memfotocopy, merekam atau dengan sistem penyimpanan lainnya, tanpa izin tertulis dari Penulis.

Penerbit: C.V ANDI OFFSET (Penerbit ANDI)
Jl. Beo 38-40, Telp. (0274) 561881 (Hunting), Fax. (0274) 588282 Yogyakarta 55281

Percetakan: ANDI OFFSET
Jl. Beo 38-40, Telp. (0274) 561881 (Hunting), Fax. (0274) 588282 Yogyakarta 55281

Perpustakaan Nasional: Katalog dalam Terbitan (KDT)

Budiharto, Widodo
Robotika – Teori dan Implementasinya/
Widodo Budiharto; – **Ed. I.** – Yogyakarta: ANDI,
19 18 17 16 15 14 13 12 11 10
x + 166 hlm .; 16 x 23 Cm.
10 9 8 7 6 5 4 3 2 1
ISBN: 978 – 979 – 29 – 1440 – 5
I. Judul
1. Robotics
DDC'21 : 629.892

KATA PENGANTAR

Memelajari robotika dengan teknologi terkini merupakan hal yang menarik bagi siapa saja. Untuk itulah penulis dengan senang hati meluncurkan buku *Robotika, Teori dan Implementasi* berdasarkan pengalaman bertahun-tahun di dalam memelajari dan mengembangkan robot di Indonesia. Buku ini tepat dibaca oleh pelajar, mahasiswa serta penghobi elektronika dan perekayasa. Isi buku ini membahas secara detail mengenai konsep robot standar, kinematika, navigasi, *motion planning*, aktuator dan konsep robot canggih yang dikontrol oleh mikrokontroler handal hingga penerapan *vision* pada robot. Buku ini merupakan buku robot terlengkap yang pernah ada di Indonesia. Untuk memajukan teknologi robotika di Indonesia, penulis juga mempopulerkan teknologi sensor dan kendali cerdas robot terkini pada buku ini.

Robot di masa yang akan datang merupakan suatu kebutuhan utama dalam hidup. Oleh karena itu, menguasai teknologi ini merupakan salah satu hal yang menantang dan menjanjikan. Untuk mempermudah Anda mencoba berbagai proyek robot di buku ini, Anda dapat memiliki kit yang dibahas melalui situs e-commerce penulis di **www.toko-elektronika.com**.

Penulis mengucapkan terima kasih sebesar-besarnya kepada Bpk. Prof. Dr. Achmad Jazidie, M.Eng dan Bpk. Dr. Ir. Djoko Purwanto, M.Eng yang telah membimbing dan mengizinkan penulis menggunakan sarana riset di Lab. ELKA ITS.

Jika Anda ingin mengikuti training secara inhouse atau private, atau ingin mengadakan seminar atau workshop, Anda dapat menghubungi penulis di:

CV Pusat e-Technology
Perumahan Mutiara Gading Timur I, Jalan Cendana 5, Blok D3/2 RT 6/24
Bekasi Timur – Jawa barat 17510
HP:081410043883, Yahoo Messg: **prof_widodo**
Email:Widodo@widodo.com

www.Toko-elektronika.com
www.robotika-vision.com
www.toko-robot.com
www.Widodo.com

Sebagai penutup, penulis mengharap saran dan kritik yang membangun guna penyempurnaan buku ini. Anda dapat berkomunikasi dengan penulis melalui email di Widodo@widodo.com.

Jakarta, Maret 2010

Widodo Budiharto

DAFTAR ISI

KATA PENGANTAR iii
DAFTAR ISI v
BAB I PENGANTAR ROBOTIKA 1
1.1 SEJARAH ROBOT 1
1.2 TIPE ROBOT 3
1.2.1 Desain Robot 3
1.2.2 Robot ASIMO 4
1.3 KONSEP ROBOT MODERN 5
1.3.1 Karakteristik Robot 5
1.3.2 Sistem Embedded Robot 6
BAB II MEKANIKA ROBOT 11
2.1 ROBOT MANIPULATOR 11
2.1.1 Konfigurasi Kartesian 12
2.1.2 Konfigurasi Sylindrical 14
2.1.3 Konfigurasi Spherical 16
2.1.4 Konfigurasi Artikulated 17
2.2 RODA GEAR 18
2.2.1 Torsi 18
2.2.2 Torsi vs Kecepatan 18
2.2.3 Rasio Roda Gigi 20
2.2.4 Efisiensi Roda Gigi 20
2.2.5 Arah Putaran Roda Gigi 21
2.2.6 Gear Chain 21
2.2.7 Compound Gear 23
2.2.8 Backlash 23
2.3 JENIS RODA GIGI 23
BAB III KINEMATIKA ROBOT 31
3.1 ANALISIS KINEMATIK ROBOT 31
3.1.1 Kinematika Maju dan Kinematika Inversi 31
3.1.2 Analisis Kinematik dengan Persamaan Trigonometri 33
3.2 MATRIKS ROTASI DAN TRANSLASI 36

3.2.1 Matriks Transformasi Homogen 37
3.2.2 Metode Denavit–Hartenberg (D - H) 40

BAB IV AKTUATOR 45
4.1 MENGENAL AKTUATOR 45
4.1.1 Fungsi Aktuator 45
4.1.2 Motor DC Magnet Permanen 46
4.1.3 Prinsip Kerja Motor DC 46
4.1.4 Pengontrolan Motor DC Magnet Permanen 47
4.2 MOTOR DC BRUSHLESS 55
4.2.1 Konstruksi dan Cara Kerja Motor DC Brushless 55
4.2.2 Pengontrolan Motor DC Brushless 58
4.3 MOTOR STEPPER 63
4.3.1 Motor Stepper Magnet Permanen 64
4.3.1.1 Motor Stepper Unipolar 64
4.3.1.2 Motor Stepper Bipolar 67
4.3.2 Kontroler dan Driver Motor Stepper 69
4.4 MOTOR SERVO 73

BAB V PEMROGRAMAN DASAR ROBOT 77
5.1 MENGENAL MIKROKONTROLER 77
5.1.1 Prinsip Kerja Mikrokontroler 77
5.1.2 Arsitektur Mikrokontroler 79
5.2 MEMROGRAM ROBOT LINE FOLLOWER 83
5.3 MEMROGRAM ROBOT AVOIDER 87
5.4 MEMROGRAM ROBOT TANK 91

BAB VI NAVIGASI ROBOT 97
6.1 PENDAHULUAN 97
6.2 NAVIGASI ROBOT INDOOR 97
6.3 NAVIGASI OUTDOOR 99
6.4 MOTION PLANNING 102

BAB VII SOFT COMPUTING ROBOT 109
7.1 PENDAHULUAN 109
7.1.1 Soft Computing 109
7.1.2 Artificial Neural Network 111
7.2 NEURAL NETWORK PADA ROBOTIKA 111
7.3 FEED-FORWARD NETWORK 112
7.4 ANFIS 115

BAB VIII COMPUTER VISION DENGAN OpenCV 129
8.1 PENDAHULUAN 129
8.2 PROGRAM OpenCV 130
8.3 KAMERA DAN VIDEO 137

BAB IX ROBOTICS VISION 143
9.1 PENDAHULUAN 143
9.2 HUMAN ROBOT INTERACTION 143
9.3 STEREO VISION 145
9.4 DETEKSI DAN PENGENALAN WAJAH 146
9.5 KAMERA DAN VIDEO 152

LAMPIRAN 161

BAB I

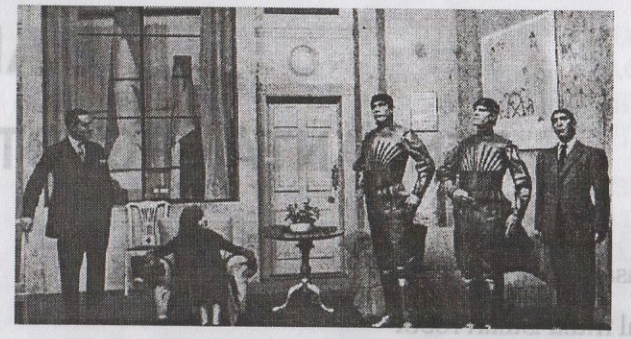
PENGANTAR ROBOTIKA

Pokok bahasan pada bab ini:

1. Asal mula istilah robot
2. Beberapa tipe robot
3. Konsep robot modern

1.1 SEJARAH ROBOT

Istilah robot sudah tidak asing lagi bagi pendengaran kita. Istilah robot berasal dari Czech, *robota,* yang berarti bekerja. Istilah itu diperkenalkan ke publik oleh Karel Čapek saat mementaskan RUR (*Rossum's Universal Robots*) pada tahun 1921 [1, 2]. Meskipun begitu, awal kemunculan robot sebetulnya dapat dirunut dari bangsa Yunani kuno yang membuat patung yang dapat dipindah-pindahkan. Sekitar 270 BC, Ctesibus, seorang insinyur Yunani, membuat organ dan jam air dengan komponen yang dapat dipindahkan. Pada zaman Nabi Muhammad SAW telah dibuat mesin perang dengan menggunakan roda dan dapat melontarkan bom. Bahkan Al-Jajari (1136-1206), seorang ilmuwan Islam pada dinasti Artuqid, dianggap sebagai tokoh yang pertama kali menciptakan robot humanoid yang berfungsi sebagai 4 musisi. Hebat, kan?



Gambar 1.1 *Pementasan RUR*

Gambar 1.2 *Robot Humanoid Dapat Diprogram oleh Al-Jajari*

Pada tahun 1770, Pierre Jacquet Droz, seorang pembuat jam berkebangsaan Swiss, membuat 3 boneka mekanis. Uniknya, boneka-boneka itu dapat melakukan fungsi spesifik. Sebuah boneka dapat menulis, yang kedua dapat memainkan musik dan organ, dan yang ketiga dapat menggambar. Pada tahun 1898 Nikola Tesla membuat sebuah boat yang dikontrol melalui radio *remote control*, dan didemokan di Madison Square Garden. Namun usaha untuk membuat *autonomous boat* tersebut gagal karena masalah dana.

Jepang, pada tahun 1967, yang pada saat itu merupakan negara yang baru bangkit, mengimpor robot Versatran dari AMF. Kejayaan robot dimulai pada tahun 1970 ketika Profesor Victor Scheinman dari Universitas Stanford mendesain lengan standar. Saat ini, konfigurasi kinematiknya dikenal sebagai standar lengan robot. Terakhir, pada tahun 2000, Honda memamerkan robot yang dibangun bertahun-tahun lamanya, yang diberi nama ASIMO, yang kemudian disusul oleh Sony dengan robot anjing AIBO-nya.

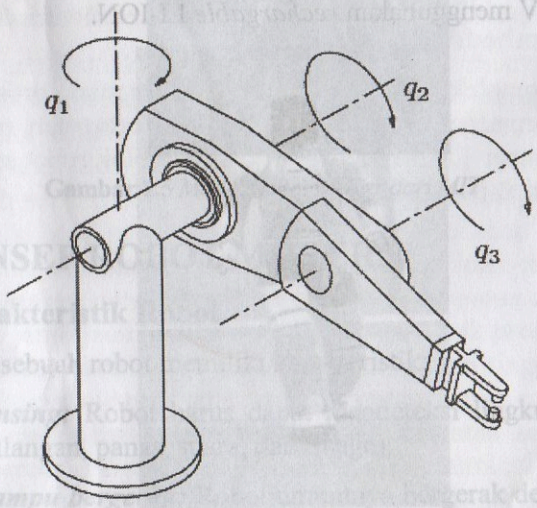
1.2 TIPE ROBOT

1.2.1 Desain Robot

Robot didesain dan dibuat sesuai kebutuhan pengguna. Robot, hingga saat ini, secara umum dibagi menjadi:

1. Robot manipulator
2. Robot mobil (*mobile robot*)
   a. Robot daratan (*ground robot*)
      1). Robot beroda
      2). Robot berkaki
   b. Robot air (*submarine robot*)
   c. Robot terbang (*aerial robot*)

Robot manipulator biasanya dicirikan dengan memiliki lengan *(arm robot)*, sedangkan robot mobil mengarah ke robot yang bergerak, meskipun nantinya pada bagian robot tersebut juga dipasang manipulator. Robot manipulator umumnya memiliki 6 DOF (*Degrees of Freedom*), di mana 3 bagian untuk menentukan posisi ujung *link* terakhir pada ruang kartesian dan 3 sisanya menentukan orientasi. Sebagai contoh sederhana, pada gambar di bawah, variabel q_1, q_2 dan q_3 menghubungkan sendi dari robot manipulator.



Gambar 1.3 *Robot Manipulator [4]*

Seiring berkembangnya teknologi robot, muncul robot humanoid yang dilengkapi roda atau kaki. Robot humanoid adalah robot dengan tampilan keseluruhan mirip tubuh manusia, yang membuatnya mampu berinteraksi secara sosial. Pembuatan robot humanoid memiliki tingkat kesulitan yang lebih tinggi dibanding pembuatan robot lainnya. Robot humanoid yang cukup menarik adalah ASIMO dari Honda.

1.2.2 Robot ASIMO

ASIMO adalah robot humanoid yang dibuat oleh Honda. Dengan tinggi 130 sentimeter dan berat 54 kilogram, penampilan robot ini menyerupai seorang astronot dengan baju astronot dan membawa ransel. ASIMO dapat berjalan dengan dua kaki dengan gaya berjalan yang menyerupai manusia dengan kecepatan hingga 6 km/jam. ASIMO diciptakan pada pusat penelitian dan pengembangan Honda di Pusat Penelitian Teknik Fundamental Wako di Jepang. Model yang sekarang merupakan versi ke sebelas, semenjak dimulainya proyek ASIMO pada 1986. Menurut Honda, ASIMO merupakan akronim dari *Advanced Step in Innovative Mobility* (langkah besar dalam pergerakan inovatif). Pernyataan resmi Honda tersebut bertentangan dengan persepsi umum yang mengira bahwa kata ASIMO berasal dari nama penemu Tiga Hukum Robotika, Isaac Asimov Robot ini memiliki tinggi 130 cm dengan total 34 DOF dengan total catuan 51,8V menggunakan *rechargable* LI-ION

Gambar 1.4 *Robot ASIMO [3]*

Robot banyak dibuat oleh institusi riset, universitas, departemen pertahanan, serta institusi besar lainnya seperti NASA dan Sony.

Saat ini hampir semua industri manufaktur menggunakan robot. Hal itu karena biaya operasional per jam untuk robot jauh lebih murah dibandingkan menggunakan tenaga manusia. Robot pada awalnya hanya digunakan untuk melakukan fungsi spesifik, misalnya pengecoran, penyolderan, atau yang lain, tetapi saat ini sudah banyak robot yang dapat melakukan banyak fungsi. Tren saat ini ialah robot yang mampu membantu kerja manusia, seperti model robot dari MIT, di mana beban berat seorang tentara dibantu dengan pergelangan kaki yang kuat dari robot.

Gambar 1.5 *Model Robot Militer dari MIT*

1.3 KONSEP ROBOT MODERN

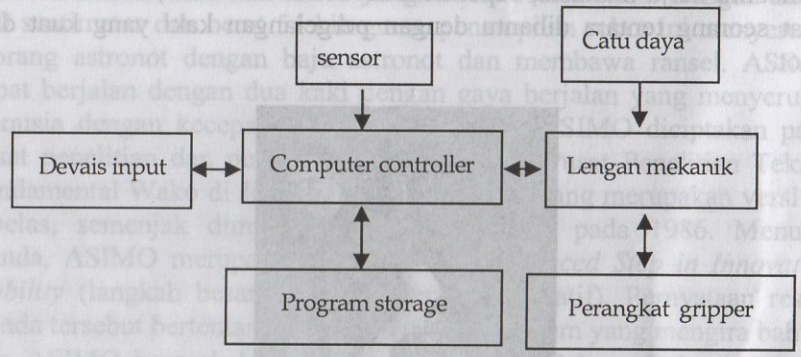
1.3.1 Karakteristik Robot

Umumnya sebuah robot memiliki karakteristik:

1. ***Sensing:*** Robot harus dapat mendeteksi lingkungan sekitarnya (halangan, panas, suara, dan image).
2. ***Mampu bergerak:*** Robot umumnya bergerak dengan menggunakan kaki atau roda, dan pada beberapa kasus robot diharapkan dapat terbang atau berenang.

3. ***Cerdas:*** Robot memiliki kecerdasan buatan agar dapat memutuskan aksi yang tepat dan akurat.
4. ***Membutuhkan energi yang memadai:*** Robot membutuhkan catu daya yang memadai agar unit pengontrol dan aktuator dapat menjalankan fungsinya dengan baik.

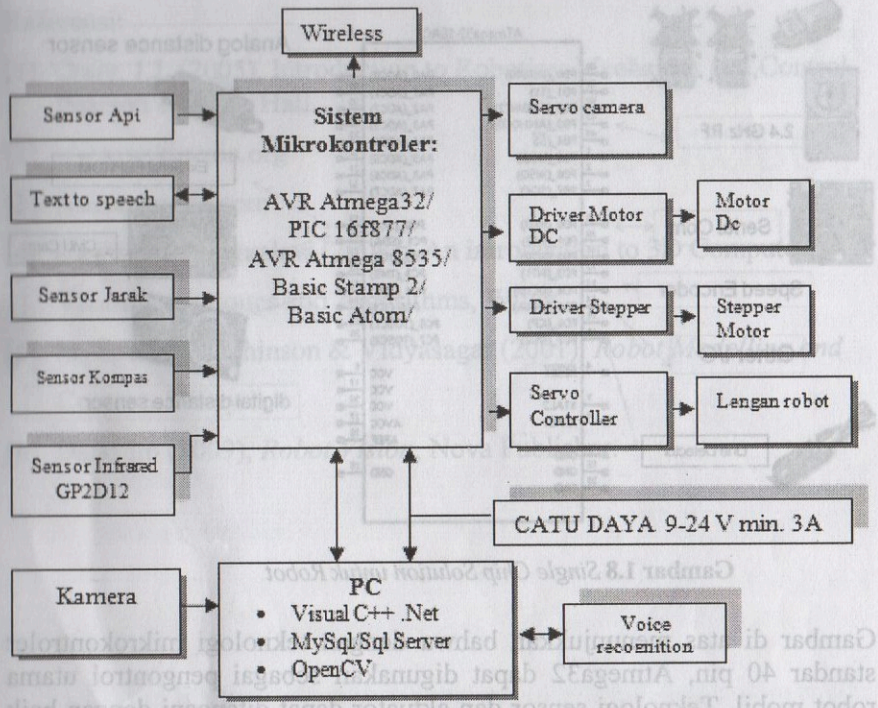
Komponen sistem robot manipulator yang umum ditampilkan pada Gambar 1.6. Sistem robot manipulator terdiri dari input, sensor catu daya, lengan mekanis, perangkat *gripper*, prosesor, dan memori program.



Gambar 1.6 *Komponen Sistem Robot Manipulator [5]*

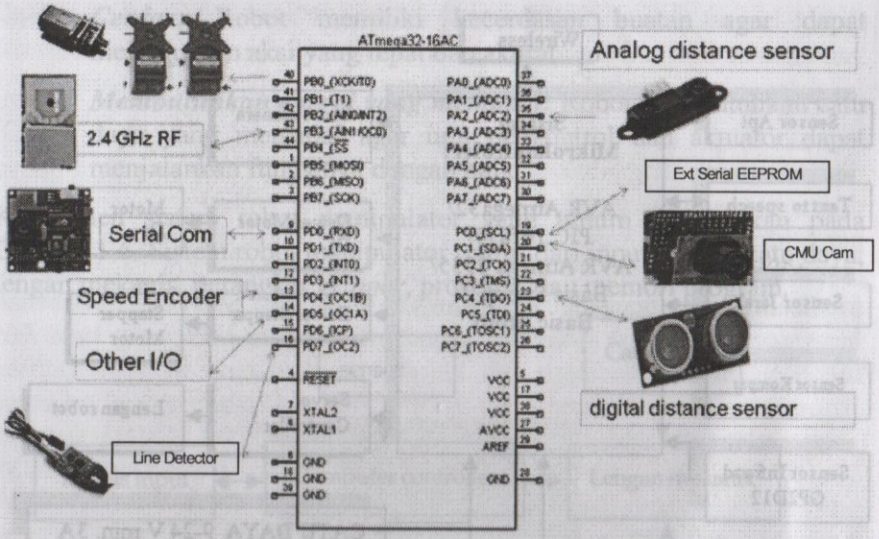
1.3.2 Sistem Embedded Robot

Robot yang kompleks membutuhkan kemampuan prosesor yang memadai. Kemampuan prosesor antara lain ialah kecepatan, memori, dan fasilitas I/O. Gambar 1.7 menunjukkan diagram blok robot standar yang dapat dibangun oleh para pemula.



Gambar 1.7 *Rancangan robot cerdas yang umum (hak cipta penulis)*

Anda dapat menggunakan berbagai mikroprosesor/mikrokontroler untuk membuat robot tersebut secerdas mungkin. Anda dapat menggunakan unit pengontrol standar yang umum beredar, seperti AVR, Basic Stamp, dan Basic Atom dengan kemampuan luar biasa. Semua input yang diterima oleh sensor akan diolah oleh mikrokontroler. Melalui program yang telah kita buat, mikroprosesor/mikrokontroler akan melakukan aksi ke aktuator, seperti lengan robot atau roda dan kaki robot. Teknologi wireless digunakan untuk keperluan transmisi data atau untuk menerima perintah dari jarak jauh. PC/Laptop digunakan untuk program utama dan melakukan proses komputasi data/image kecepatan tinggi, karena tidak mampu dilakukan oleh mikrokontroler standar. Untuk memberikan catu daya pada robot dapat digunakan baterai, aki, atau solar cell. Untuk keperluan percobaan dapat digunakan mikrokontroler standar sebagai pengendali utama, yaitu menggunakan Atmega32, seperti tampak pada Gambar 1.8.



Gambar 1.8 *Single Chip Solution untuk Robot*

Gambar di atas menunjukkan bahwa dengan teknologi mikrokontroler standar 40 pin, Atmega32 dapat digunakan sebagai pengontrol utama robot mobil. Teknologi sensor dan aktuator dapat ditangani dengan baik menggunakan mikrokontroler dengan kemampuan I^2C untuk komunikasi data antara mikrokontroler dengan perangkat serial lainnya. Beberapa pertimbangan di dalam memilih mikrokontroler yang tepat untuk robot ialah jumlah I/O, kemampuan ADC, fitur pengolahan sinyal, dan besar memori RAM dan flash memori program.

Pada robot yang kompleks yang membutuhkan berbagai sensor dan input dengan jumlah besar sering dibutuhkan lebih dari 1 pengontrol yang menggunakan prinsip *master* dan *slave*. Pada model ini terdapat 1 buah pengontrol utama yang berfungsi untuk mengkoordinasi mikrokontroler slave.

Latihan:

Sebutkan beberapa tipe robot yang umum dikembangkan saat ini.

Telusurilah perkembangan teknologi robot sejak awal ditemukan hingga teknologi terkini.

Referensi:

[1]. Craig, J.J. (2005), Introduction to Robotics: Mechanics and Control, Pearson Prentice Hall.

[2]. www.wikipedia.org

[3]. Asimo.honda.com

[4]. Cyganek & Boguslaw (2008), An introduction to 3D Computer Vision Techniques and Algorithms, Wiley.

[5]. M. Spong, Hutchinson & Vidyasagar (2001). *Robot Modelling and* Control, Wiley.

[6] Daiki Ito (2009), *Robot Vision*, Nova Publisher.

BAB II
MEKANIKA ROBOT

Pokok bahasan pada bab ini:

1. Robot Manipulator
2. Roda Gear
3. Jenis Roda Gigi

2.1 ROBOT MANIPULATOR

Robot manipulator adalah nama yang populer untuk lengan mekanik. Robot manipulator merupakan gabungan dari beberapa segmen dan sendi yang secara umum dibagi menjadi 3 bagian, yaitu: *arm, wrist* dan *gripper*.

Konfigurasi robot digunakan untuk mengklasifikasikan robot-robot industri. Konfigurasi robot mengarah pada bentuk geometri dari manipulator robot, yaitu informasi hubungan dari setiap sendi pada manipulator. *Robotic Industries Association* (RIA) mendefinisikan robot sebagai manipulator yang didesain untuk memindahkan material, benda, alat atau peralatan tertentu lewat pergerakan yang terprogram untuk melakukan berbagai macam tugas.

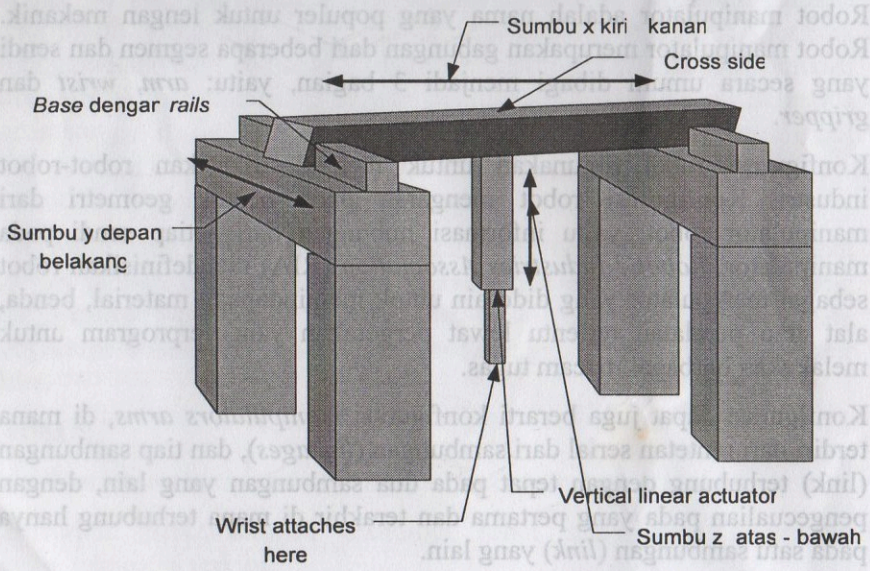
Konfigurasi dapat juga berarti konfigurasi *manipulators arms*, di mana terdiri dari rentetan serial dari sambungan (*linkages*), dan tiap sambungan (link) terhubung dengan tepat pada dua sambungan yang lain, dengan pengecualian pada yang pertama dan terakhir di mana terhubung hanya pada satu sambungan (*link*) yang lain.

Konfigurasi merujuk pada bagaimana cara tiap sambungan (*link*) manipulator terhubung antara satu dengan yang lain pada setiap sendi. Tiap sambungan *(link)* akan terhubung pada link berikutnya, baik berupa hubungan *linear joint* (sliding atau prismatik), yang dapat disingkat dengan P, atau berupa hubungan *revolute joint*, yang dapat disingkat dengan R. Dengan menggunakan notasi ini, sebuah robot dengan tiga revolute joint, disingkat dengan RRR, atau sebuah revolute joint yang diikuti dengan 2 linear joint disingkat dengan RPP. Secara umum terdapat lima konfigurasi robot yang digunakan di industri, yaitu: Cartesian Robot, Cylindrical Robot, Spherical Robot, Articulated Robot, dan SCARA *(Selectively Compliant Assembly Robot Arm)*.

DOF adalah singkatan dari *degree of freedom*. Setiap sendi minimal terdiri atas 1 DOF. Robot lengan mempunyai 3 DOF sehingga dia dapat bergerak ke atas–bawah, ke kiri–kanan, ke depan–belakang.

2.1.1 Konfigurasi Kartesian

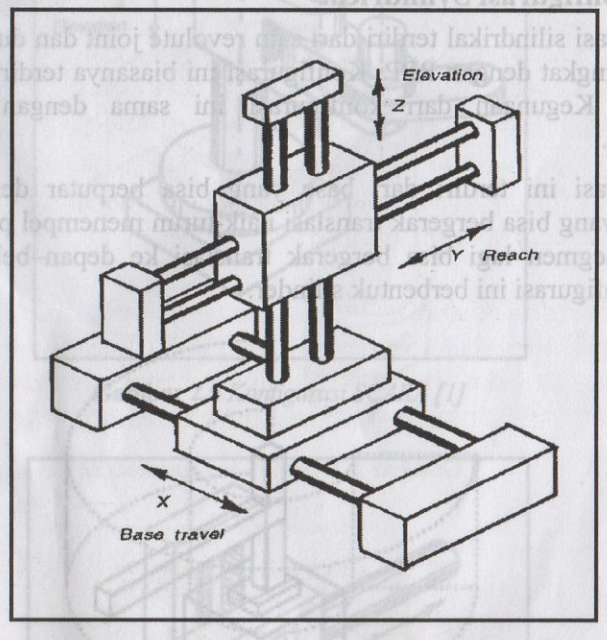
Konfigurasi kartesian terdiri dari tiga sumbu linear yang saling tegak lurus, yang disingkat dengan PPP, seperti tampak pada gambar di bawah. Area kerja robot berada pada sistem koordinat xyz kartesian.



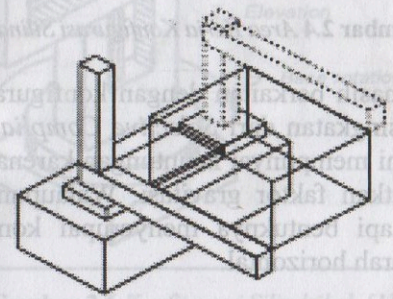
Gambar 2.1 *Konfigurasi Kartesian Jenis Gantry [1]*

Keuntungan konfigurasi ini adalah perhitungan persamaan gerak robot yang mudah, karena pergerakan setiap sumbu kartesian dipengaruhi oleh satu aktuator sehingga memudahkan pemograman pergerakannya.

Ada dua jenis robot kartesian yang umum, yaitu *cantilevered* dan *gantry*. Jika konfigurasi manipulator hendak diaplikasikan pada robot mobil maka konfigurasi cantilevered yang memungkinkan untuk dipasang.



Gambar 2.2 *Konfigurasi Kartesian Jenis Cantilevered [1]*



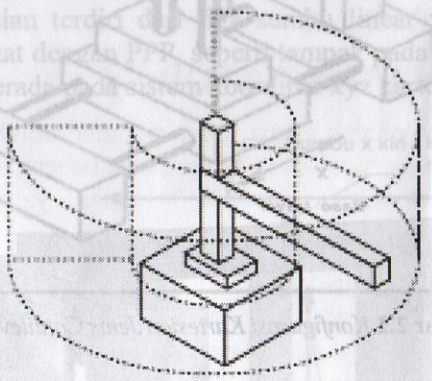
Gambar 2.3 *Area Kerja Konfigurasi Kartesian [2]*

Robot Kartesian digunakan pada beberapa aplikasi, seperti operasi perakitan di mana robot ini sangat mudah dalam melakukan pergerakan vertikal secara lurus. Karena kemudahan dalam pergerakan lurus, robot ini juga cocok untuk digunakan pada mesin pemuatan (*loading*) dan pembongkaran (*unloading*).

2.1.2 Konfigurasi Sylindrical

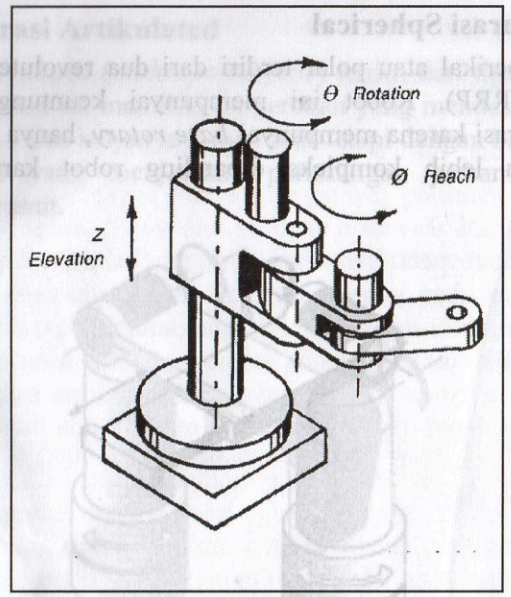
Konfigurasi silindrikal terdiri dari satu revolute joint dan dua linear joint yang disingkat dengan RPP. Konfigurasi ini biasanya terdiri dari 3 DOF segmen. Kegunaan dari konfigurasi ini sama dengan konfigurasi kartesian.

Konfigurasi ini terdiri dari base yang bisa berputar dengan sebuah segmen yang bisa bergerak translasi naik-turun menempel pada base dan sebuah segmen lagi bisa bergerak translasi ke depan–belakang. Area kerja konfigurasi ini berbentuk silinder.

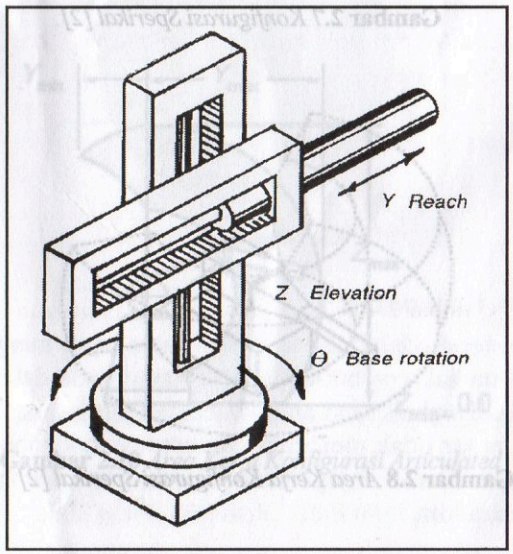


Gambar 2.4 *Area Kerja Konfigurasi Silindrikal*

Bentuk lain yang masih berkaitan dengan konfigurasi silindrikal adalah SCARA. SCARA singkatan dari *Selective Compliance Assembly Robot Arm.* Konfigurasi ini mempunyai keuntungan karena dalam perhitungannya tidak mengikutkan faktor gravitasi. Walaupun area kerjanya berbentuk silinder tetapi bentuknya menyerupai konfigurasi polar yang diposisikan dalam arah horizontal.



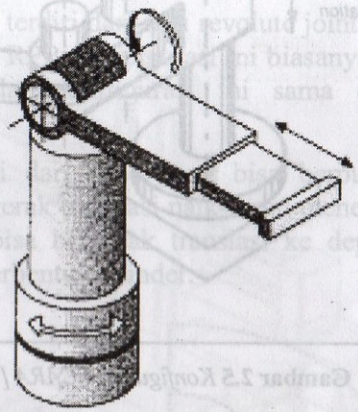
Gambar 2.5 *Konfigurasi SCARA [1]*



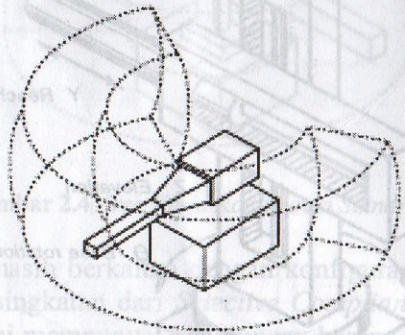
Gambar 2.6 *Konfigurasi Silindrikal [1]*

2.1.3 Konfigurasi Spherical

Konfigurasi sperikal atau polar terdiri dari dua revolute joint dan satu linear joint (RRP). Robot ini mempunyai keuntungan dalam hal kecepatan operasi karena mempunyai *base rotary*, hanya saja persamaan geraknya jauh lebih kompleks dibanding robot kartesian maupun silindrikal.



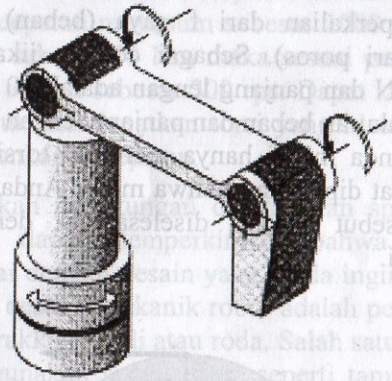
Gambar 2.7 *Konfigurasi Sperikal [2]*



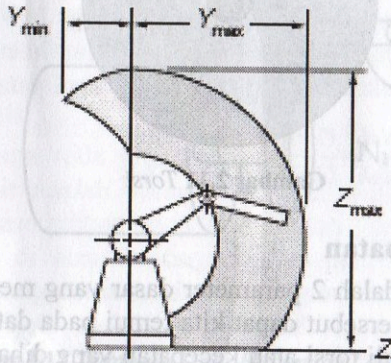
Gambar 2.8 *Area Kerja Konfigurasi Sperikal [2]*

2.1.4 Konfigurasi Artikulated

Konfigurasi *articulated* (*anthropomorphic*) terdiri dari tiga revolute joint (RRR). Konfigurasi ini memiliki pergerakan yang mendekati pergerakan tangan manusia. Area kerjanya sangat luas tetapi dengan bentuk geometri yang kompleks yang menjadikan perhitungan persamaan geraknya menjadi cukup rumit.



Gambar 2.9 *Konfigurasi Articulated*



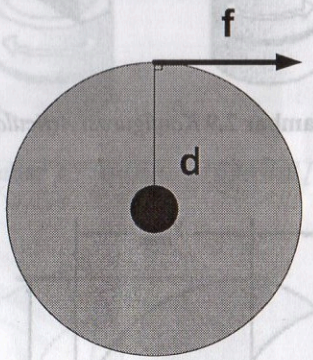
Gambar 2.10 *Area Kerja Konfigurasi Articulated*

2.2 RODA GEAR

Roda gigi merupakan komponen penting ketika kita akan membuat robot. Penggunaan roda gigi dalam robot sangat berpengaruh terhadap 2 parameter penting, yaitu torsi dan kecepatan.

2.2.1 Torsi

Torsi merupakan perkalian dari f gaya (beban) dengan d jari–jari (panjang lengan dari poros). Sebagai contoh, jika beban yang harus diputar sebesar 10 N dan panjang lengan adalah 10 cm maka besar torsi adalah 1 Nm. Jika ukuran beban dan panjang lengan sesuai contoh di atas dan motor yang Anda miliki hanya memiliki torsi maksimum sebesar 0,75 Nm maka dapat dipastikan bahwa motor Anda tidak akan berputar. Permasalahan tersebut dapat diselesaikan dengan menggunakan kombinasi roda gigi.



Gambar 2.11 *Torsi*

2.2.2 Torsi vs Kecepatan

Kecepatan dan torsi adalah 2 parameter dasar yang menjadi ukuran bagi suatu motor. Dua hal tersebut dapat kita temui pada datasheet dari motor tersebut tetapi seringkali torsi atau kecepatan yang dihasilkan oleh motor yang dijual di pasar tidak memenuhi aktuator untuk robot kita. Di sinilah peran roda gigi untuk mengonversi torsi dan kecepatan agar sesuai dengan keinginan kita.

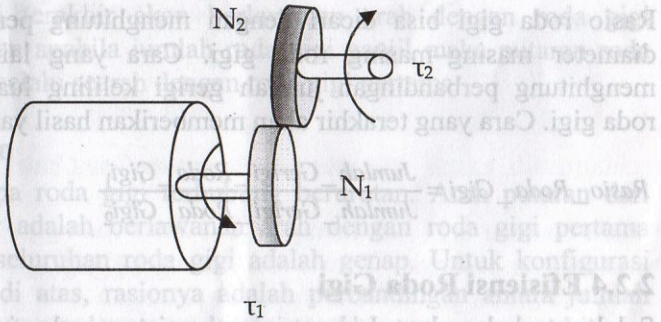
Salah satu keuntungan dari pemakaian roda gigi adalah kita dapat mengubah kecepatan tinggi menjadi torsi yang tinggi.

$\text{Torsi}_1 \times \text{kecepatan}_1 = \text{torsi}_2 \times \text{kecepatan}_2$

Torsi_1 dan kecepatan_1 merupakan parameter output dari motor sedangkan torsi_2 dan kecepatan_2 merupakan parameter output dari roda gigi pada poros output (biasanya yang terhubung ke roda).

Sebagai contoh, jika motor Anda mempunyai parameter output sesuai datasheet, torsi sebesar 3 Nm dan kecepatan sebesar 2000 rps (*round per second*) dengan kecepatan minimum sebesar 1000 rps sedangkan Anda menginginkan torsi sebesar 20 Nm maka sesuai perhitungan, kecepatan yang kita dapatkan adalah sebesar 300 rps. Dapat kita amati dari contoh tersebut bahwa untuk mendapatkan torsi yang ideal (lebih besar) maka kita harus mengorbankan kecepatan.

Dengan menggunakan perhitungan di atas dan spesifikasi dari motor Anda maka Anda dapat memperkirakan bahwa motor yang Anda gunakan tidak sesuai dengan desain yang Anda inginkan. Hal yang perlu diperhatikan dalam desain mekanik robot adalah perhitungan kebutuhan torsi untuk menggerakkan sendi atau roda. Salah satu metode yang paling umum ialah menggunakan sistem *gear* seperti tampak pada gambar di bawah. Di sini digunakan dua buah gear.



Gambar 2.12 *Transmisi Gear Hubungan Langsung Motor DC*

Pada gambar di atas, N_1 adalah jumlah gigi pada gear poros motor, N_2 ialah jumlah gigi pada poros output, τ_1 ialah torsi pada poros motor dan τ_2 ialah torsi pada poros output.

Torsi output dapat dihitung dengan rumus:

$$\tau_2 = N_2/N_1 * \tau_1$$

Sedangkan putaran output dapat dihitung:

$$\text{Putaran}_{out} = \frac{N_1}{N_2} \text{ x Putaran Motor}$$

2.2.3 Rasio Roda Gigi

Salah satu cara mudah untuk mengimplementasikan perhitungan di atas adalah dengan menggunakan sepasang roda gigi dengan diameter yang berbeda. Bila kita menggunakan roda gigi dengan diameter yang besar maka kita akan mendapatkan gerakan yang lebih lambat tetapi torsinya lebih besar. Semakin besar perbedaan diameter di antara roda gigi yang berpasangan maka semakin besar perbedaan antara kecepatan dan torsi yang dihasilkan masing–masing roda gigi.

$$Rasio_Roda_Gigi = \frac{Diameter_Roda_Gigi_1}{Diameter_Roda_Gigi_2} = \frac{Torsi_1}{Torsi_2}$$

Rasio roda gigi bisa dicari dengan menghitung perbandingan antara diameter masing–masing roda gigi. Cara yang lain adalah dengan menghitung perbandingan jumlah gerigi keliling luar masing–masing roda gigi. Cara yang terakhir akan memberikan hasil yang lebih akurat.

$$Rasio_Roda_Gigi = \frac{Jumlah_Gerigi_Roda_Gigi_1}{Jumlah_Gerigi_Roda_Gigi_2}$$

2.2.4 Efisiensi Roda Gigi

Salah satu kelemahan dari penggunaan sistem roda gigi adalah turunnya efisiensi (Pout/Pin). Hal ini disebabkan oleh banyak faktor, antara lain *friction*, *backlash*, masalah pelumasan, momentum sudut putar, dsb.

Efisiensi tiap sistem roda gigi tidaklah sama. Hal ini dikarenakan perbedaan konstruksi, perbedaan jenis roda gigi yang dipakai, perbedaan bahan pembuatan roda gigi, dsb. Sebagai contoh, jika Anda memiliki kombinasi roda gigi spur, efisiensi yang didapatkan adalah sebesar ~90% dan untuk motor yang Anda gunakan seperti pada contoh sebelumnya

dapat Anda hitung torsi dan kecepatannya yang sebenarnya setelah dikalikan efisiensi.

Contoh:

$Torsi_1 = 3$ Nm, dan $kecepatan_1 = 2000$ rps

Rasio roda gigi = 3/8, maka

$Torsi_2 = 8/3 * 3 = 8$ Nm dan $kecepatan_2 = 3/8 * 750$ rps, kemudian setelah diperhitungkan dengan efisiensinya:

Torsi = 8 * 0,9 = 7,2 Nm, dan kecepatan = 750 * 0,9 = 675 rps.

2.2.5 Arah Putaran Roda Gigi

Di dalam membuat sistem roda gigi kita harus dapat memperkirakan bagaimana arah putaran dari roda gigi tersebut. Sebagai contoh, apabila kita mempunyai 2 buah roda gigi yang terhubung secara langsung, bila salah satu berputar searah jarum jam maka yang lain pasti berputar berlawanan arah dengan jarum jam. Kemudian bagaimana arah dari roda gigi terakhir apabila sistem roda gigi tersebut mempunyai 20 buah roda gigi yang saling terhubung secara langsung. Rumus yang digunakan untuk permasalahan tersebut adalah bila jumlah roda gigi genap maka putaran roda gigi terakhir akan berlawanan arah dengan roda gigi pertama. Sedangkan apabila jumlah roda gigi ganjil maka putaran roda gigi terakhir akan selalu searah dengan roda gigi pertama.

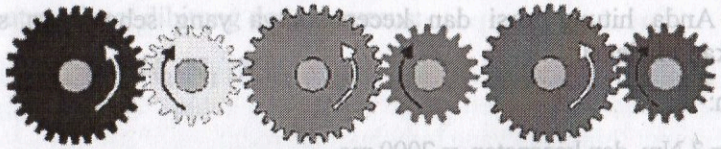
2.2.6 Gear Chain

Gabungan beberapa roda gigi terhubung berurutan. Arah putaran dari roda gigi terakhir adalah berlawanan arah dengan roda gigi pertama karena jumlah keseluruhan roda gigi adalah genap. Untuk konfigurasi roda gigi seperti di atas, rasionya adalah perbandingan antara jumlah gerigi roda gigi pertama dengan roda gigi terakhir tanpa memperhitungkan roda gigi yang berada di antaranya. Efisiensinya dihitung dengan cara sebagai berikut:

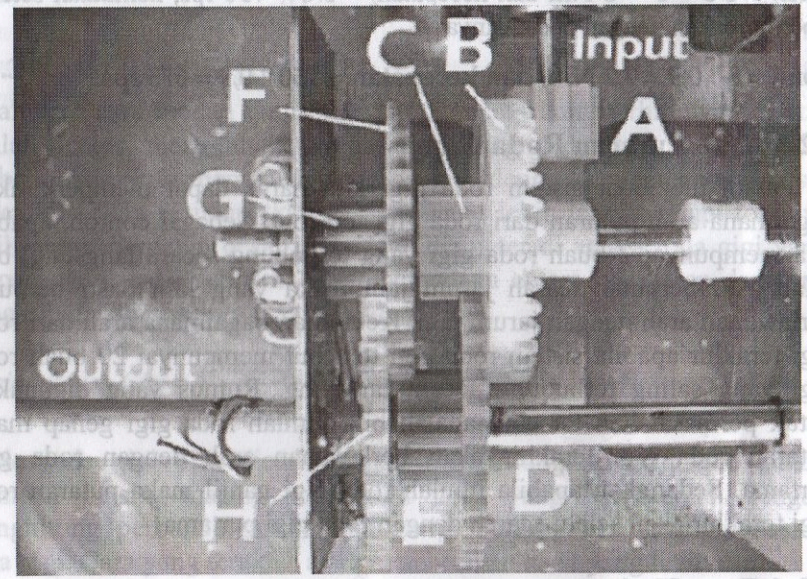
Efisiensi total = efisiensi roda gigi ^ (jumlah roda gigi - 1)

Sebagai contoh, jika Anda mempunyai 5 buah roda gigi yang disusun secara spur maka:

Efisiensi total = 0,9^(5-1)= 65,6%



Gambar 2.13 *Gear Chain* [3]

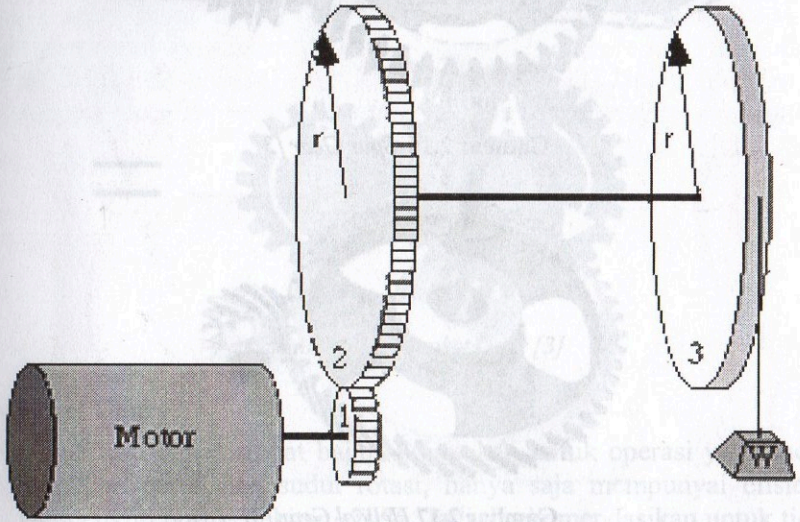


Gambar 2.14 *Penerapan Gear Chain*

Pada Gambar 2.14 kita mempunyai 8 buah roda gigi. Roda gigi A dan B merupakan pasangan Crown and Pinion, sedangkan roda gigi C-D, roda gigi E-F, roda gigi G-H merupakan pasangan *spur gear*. Roda gigi B-C, roda gigi D-E, roda gigi F-G berada pada satu poros. Roda gigi A berada pada poros input dan roda gigi H berada pada poros output. Jumlah gigi roda gigi A = 10, roda gigi B = 36, roda gigi C = 14, roda gigi D = 36, roda gigi E = 14, roda gigi F = 36, roda gigi G = 14, roda gigi H = 36. Rasio A-B adalah 1:3,6 sedangkan 3 rasio lainnya C-D, E-F,G-H adalah 36:14 atau 2,57:1 sehingga rasio roda gigi keseluruhan adalah 1:(3,6x2,57x2,57x2,57) = 1:61.

2.2.7 Compound Gear

Compound gear adalah 2 buah roda gigi yang terhubung pada poros yang sama. Kita dengan mudah dapat mengetahui bahwa 2 roda gigi tersebut akan berputar dengan kecepatan yang sama. Pada Gambar 2.15, roda gigi 2 dan 3 merupakan compound gear.



Gambar 2.15 *Compound Gear [3]*

2.2.8 Backlash

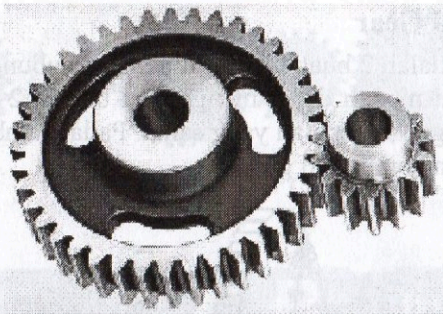
Backlash adalah jeda gerakan antara 2 roda gigi ketika dikemudikan dalam arah yang berlawanan.

2.3 JENIS RODA GIGI

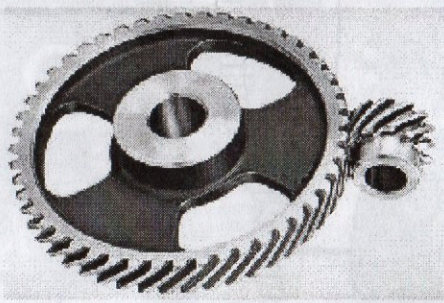
Ada berbagai macam jenis roda gigi, yaitu:

1. **Spur Gears**

   Kombinasi roda gigi ini banyak dipakai karena pemasangannya yang mudah dan efisiensinya cukup tinggi. Salah satu bentuk penggunaan yang harus dihindari adalah pada beban berat karena dapat merusak geriginya. Efisiensi yang diberikan oleh kombi-nasi roda gigi ini berkisar ~90% tergantung datasheet komponen.



Gambar 2.16 *Spur Gear [3]*



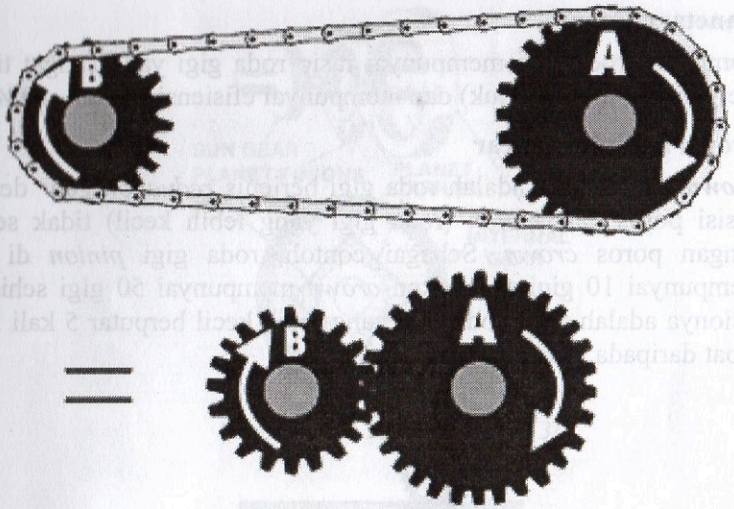
Gambar 2.17 *Helical Gear [3]*

2. **Helical Gears**

   Kombinasi roda gigi ini beroperasi seperti spur gear tetapi dengan pergerakan yang lebih lembut. Efisiensi roda gigi ini sebesar ~90%, tergantung datasheet komponen. Harga komponen ini cukup mahal karena bentuk geriginya yang kompleks.

3. **Sproket Gears dengan Rantai**

   Kombinasi antara dua buah roda gigi dengan menggunakan rantai atau menggunakan belt (*timing belt*) dapat dianggap sebagai kombinasi antara tiga buah roda gigi. Arah putaran dari salah satu roda gigi selalu sama seperti roda gigi lainnya karena jumlah roda gigi setara dengan 3. Kombinasi ini bergerak seperti spur gear tetapi mempunyai efisiensi yang sangat rendah karena besarnya area kontak sehingga friksi yang terjadi meningkat. Efisiensi sebesar ~80%.



Gambar 2.18 *Sproket Gear [3]*

4. **Bevel Gears**

   Kombinasi *bevel* sangat bagus digunakan untuk operasi yang membutuhkan perubahan sudut rotasi, hanya saja mempunyai efisiensi yang cukup buruk, sekitar **~70%.** Jadi direkomendasikan untuk tidak memakai kombinasi roda gigi ini.

5. **Rack and Pinion**

   Kombinasi roda gigi ini banyak ditemukan dalam sistem pengemudian. Kombinasi roda gigi ini sangat bagus untuk meng-ubah gerak rotasi menjadi gerak translasi. Efisiensi sebesar ~90%.

6. **Worm Gears**

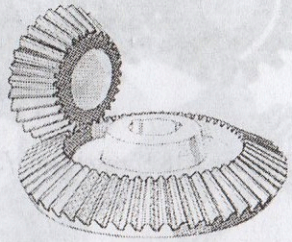
   Efisiensi kombinasi roda gigi ini cukup rendah, sekitar ~70%. Kombinasi ini mempunyai rasio yang cukup tinggi. Keuntungan lainnya adalah tidak bisa *back-driveable* sehingga yang bisa memutar roda gigi (*worm gear*) adalah worm yang terpasang pada motor sehingga hal seperti gravitasi atau gaya lainnya tidak bisa memutar roda gigi. Keuntungan ini hampir mirip motor servo yang biasa digunakan untuk menahan beban pada robot tangan.

7. **Planetary Gears**

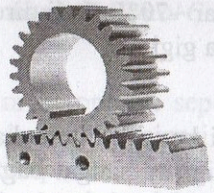
   Kombinasi roda gigi mempunyai rasio roda gigi yang sangat tinggi (bergantung jenis produk) dan mempunyai efisiensi sekitar **~80%.**

8. **Crown and pinion gear**

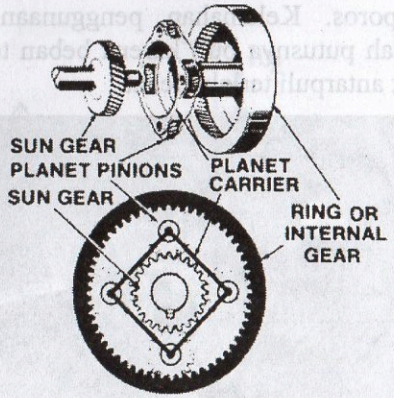
   Crown and Pinion adalah roda gigi berjenis *reduction gear* dengan posisi poros dari *pinion* (roda gigi yang lebih kecil) tidak searah dengan poros *crown.* Sebagai contoh, roda gigi *pinion* di atas mempunyai 10 gigi sedangkan *crown* mempunyai 50 gigi sehingga rasionya adalah 5:1. Roda gigi yang lebih kecil berputar 5 kali lebih cepat daripada roda gigi yang lebih besar.



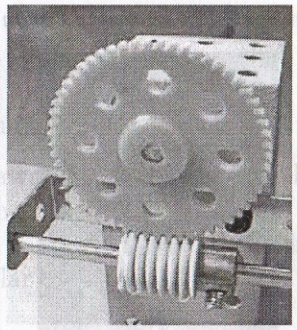
Gambar 2.19 *Bevel Gear*



Gambar 2.20 *Rack and Pinion*



Gambar 2.21 *Planetary Gear [3]*



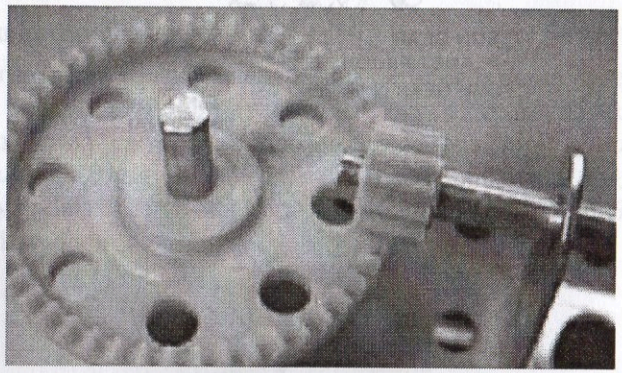
Gambar 2.22 *Worm Gears [4]*

9. **Roda Pulley**

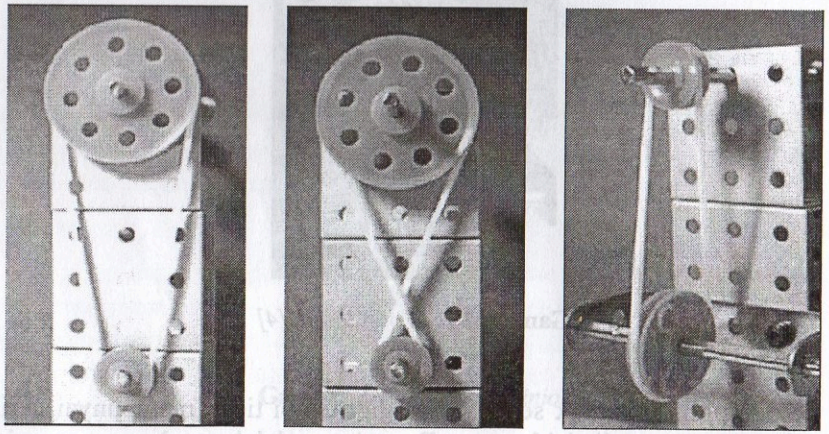
   Roda puli berbentuk seperti roda gigi tetapi tidak mempunyai gerigi dengan rongga di sisi luarnya. Fungsinya adalah untuk mentransmisikan gaya pada jarak jauh dan jika diameter antara 2 buah puli berbeda maka fungsinya sama dengan rantai roda gigi. Dua buah puli terhubung satu sama lain dengan menggunakan *belt drive* yang elastis. Hal ini juga didukung dengan keelastisan dari belt drive yang membuat puli dapat dihubungkan pada jarak berapapun asalkan belt drive tidak slip atau putus.

   Belt–drive dapat terpasang terbalik dan dapat digunakan untuk menghubungkan puli yang mempunyai sudut rotasi yang berbeda

sesuai posisi poros. Kelemahan penggunaan puli yang harus diwaspadai adalah putusnya puli karena beban terlalu berat ataukah slip karena jarak antarpuli terlalu dekat.



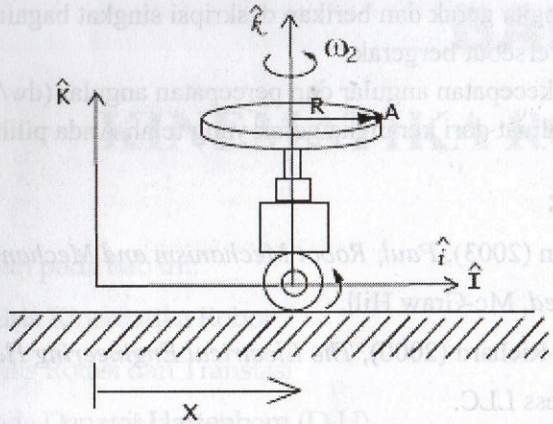
Gambar 2.23 *Crown and Pinion [4]*



Gambar 2.24 *Konfigurasi Pulley Wheel [4]*

Latihan:

1. Buatlah sistem motor DC yang menggunakan gear box yang dihubungkan dengan roda gigi membentuk sproket gear.
2. Sebuah plat bulat dengan radius R bergerak di antara 2 titik pada sumbu berputar pada kecepatan ω_2. Andaikan X diberikan sebagai $X = \text{asin}(\omega_1 t)$

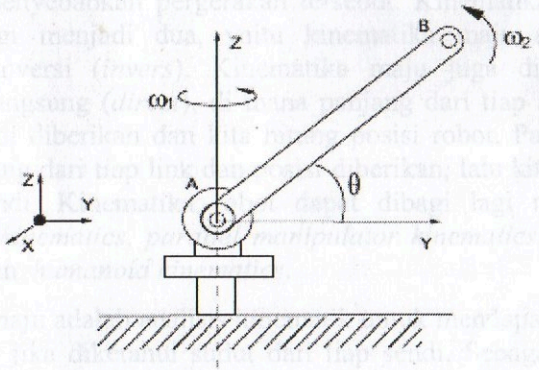


Gambar 2.25 *Plat Bulat*

Tentukan:

a. Kecepatan pada titik A pada waktu t.
b. Percepatan pada titik A pada waktu t.

3. Robot lengan di bawah memiliki 2 sendi berputar pada kecepatan konstanta ω_1 dan ω_2. Anggap panjang AB sama dengan r, θ ialah sudut di antara AB dan bidang XY, dan seluruh sudut dalam radian dan kecepatan angular dalam rad/s.



Gambar 2.26 *Robot Lengan*

a. Pilih kerangka gerak dan berikan deskripsi singkat bagaimana kerangka tersebut bergerak.
b. Tentukan kecepatan angular dan percepatan angular (dw/dt) dari AB pada koordinat dari kerangka gerak yang telah Anda pilih pada (a).

Referensi:

[1]. E. Sandin (2003), *Paul, Robot Mechanism and Mechanical Devices Illustrated*, Mc-Graw Hill.

[2]. C. Dorf, Richard (2000), *The Electrical Engineering Handbook*, CRC Press LLC.

[3]. http://www.societyofrobots.com/mechanics_gears.shtml

[4]. B., Owen (2007), *Robot Builder's Cookbook*, Elsevier Ltd.

[5]. http://www.fi.edu/time/Journey/Time/Escapements/geartypes.htm

[6]. http://www.statetoolgear.com/images/oldsite/rack_pinion.jpg

[7]. Pitowarno, Endro (2007), *Robotika, Desain dan Kontrol*, Andi Offset Yogyakarta

BAB III
KINEMATIKA ROBOT

Pokok bahasan pada bab ini:

1. Analisis Kinematika Robot
2. Matriks Rotasi dan Translasi
3. Metode Denavit Hartenberg (D-H)

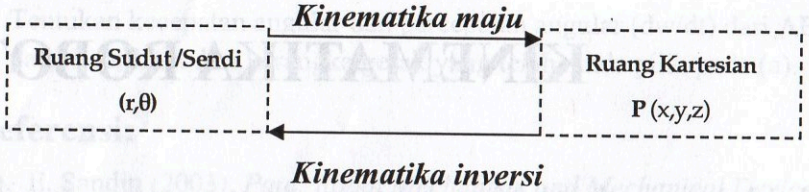
3.1 ANALISIS KINEMATIK ROBOT

3.1.1 Kinematika Maju dan Kinematika Inversi

Kinematika dapat didefinisikan sebagai studi pergerakan robot *(motion)* tanpa memperhatikan gaya (*force*) ataupun faktor lain yang memengaruhi pergerakan robot tersebut. Pada sebuah analisis kinematik, posisi, kecepatan dan akselerasi dari seluruh link dihitung tanpa memperhatikan gaya yang menyebabkan pergerakan tersebut. Kinematika robot secara umum terbagi menjadi dua, yaitu kinematika maju (*forward*) dan kinematika inversi (*invers*). Kinematika maju juga dikenal sebagai kinematika langsung (*direct*), di mana panjang dari tiap link dan sudut dari tiap sendi diberikan dan kita hitung posisi robot. Pada kinematika inversi, panjang dari tiap link dan posisi diberikan, lalu kita hitung sudut dari tiap sendi. Kinematika robot dapat dibagi lagi menjadi *serial manipulator kinematics*, *parallel manipulator kinematics*, *mobile robot kinematics* dan *humanoid kinematics*.

Kinematika maju adalah analisis kinematik untuk mendapatkan koordinat posisi (x,y,z) jika diketahui sudut dari tiap sendi. Sebagai contoh, jika mempunyai robot n-DOF dan diketahui sudut dari tiap sendi maka kita dapat menggunakan analisis kinematik maju untuk mendapatkan koordinat posisi dari robot. Kinematika inversi adalah analisis kinematik

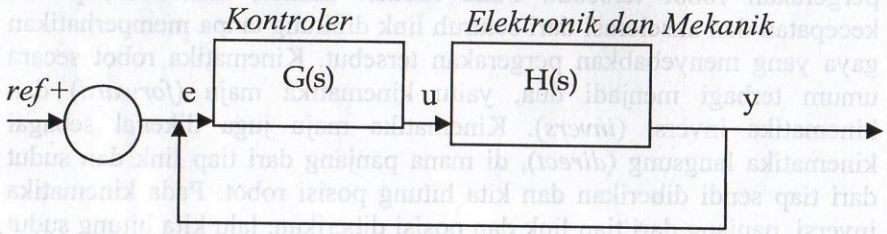
untuk mendapatkan besar sudut dari masing–masing sendi jika kita mempunyai data koordinat posisi (x,y,z).



Gambar 3.1 *Relasi Kinematika Maju dan Inversi*

Model matematik dari pergerakan robot secara umum dibedakan menjadi 2, yaitu gerak holonomik dan gerak non-holonomik. Dalam robotika, gerak holonomik, misalnya robot manipulator dua DOF dengan konfigurasi planar. Ujung robot tangan ini dapat menjangkau daerah kerja dalam koordinat kartesian dari satu titik koordinat ke titik koordinat yang lain secara langsung tanpa mengubah arah hadap dari badan robot itu sendiri. Sedangkan gerak non-holonomik adalah seperti gerak robot mobil di mana di dalam geraknya ke segala arah, robot itu harus selalu mengubah arah hadapnya.

Sistem robotika umumnya terdiri dari sistem kontroler, elektronik, dan mekanik yang dalam bentuk diagram dapat dinyatakan sebagai berikut:

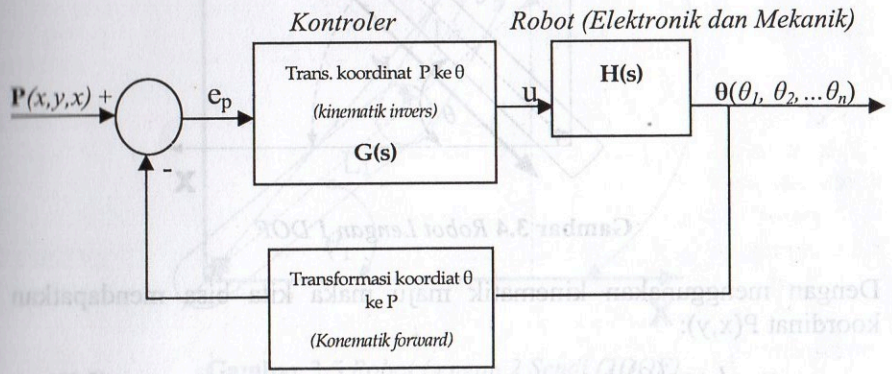


Gambar 3.2 *Diagram Sistem Robotika*

Pada Gambar 3.2, simbol ref menyatakan referensi input (bisa berupa posisi, kecepatan, dan akselerasi), simbol **G**(s) ialah persamaaan matematik kontroler, sedangkan **H**(s) adalah persamaan sistem robot secara fisik yang terdiri dari sistem elektronika dan aktuator. Komponen e ialah *error* dan komponen u ialah *output* dari kontroler tersebut, dengan output y sebagai fungsi gerak robot yang diharapkan selalu sama dengan

referensi gerak yang didefinisikan pada input ref. Pada praktiknya, input merupakan fungsi dari suatu koordinat vektor posisi dan orientasi **P**(x,y,z) dan output berupa $\boldsymbol{\theta}(\theta_1, \theta_2, \ldots\theta_n)$ dengan n merepresentasikan jumlah sendi atau DOF.

Penjabaran lebih detail dari sistem robotika di atas ditampilkan pada Gambar 3.3.

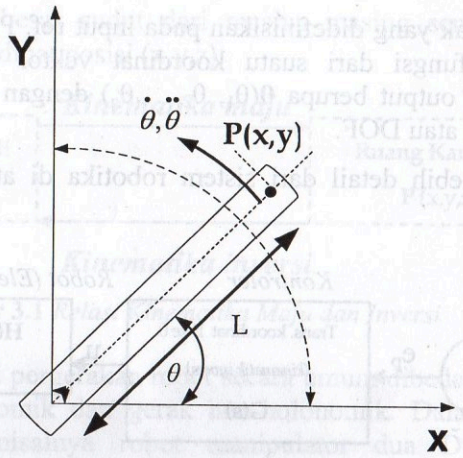


Gambar 3.3 *Diagram Sistem Kontrol Robotika*

Pada Gambar 3.3 terlihat bahwa output yang diukur dari gerakan robot berupa domain sudut dari sendi, atau sudut dari perputaran roda jika robot berupa robot mobil. Kombinasi transformasi koordinat **P** ke θ dengan kontroler **G(s)** disebut sebagai kontroler kinematik. Dengan input berupa sinyal error P, e_p dan output sinyal kemudi u untuk aktuator (bisa berupa sinyal analog dari DAC untuk aktuator). Untuk mengontrol sistem dapat digunakan Fuzzy atau Adaptive Neuro Fuzzy yang digunakan secara luas saat ini [8].

3.1.2 Analisis Kinematik dengan Persamaan Trigonometri

Salah satu cara paling mudah menyelesaikan analisis kinematik robot adalah dengan menggunakan persamaan trigonometri, misalnya analisis kinematik robot lengan 1DOF.



Gambar 3.4 *Robot Lengan 1 DOF*

Dengan menggunakan kinematik maju maka kita bisa mendapatkan koordinat P(x,y):

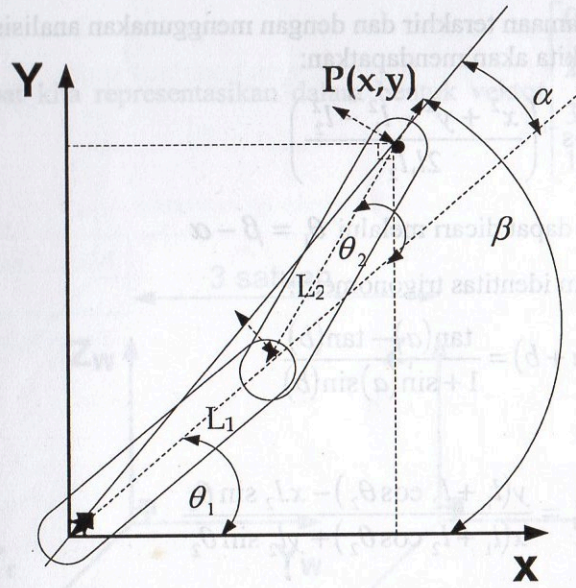
$$x = l.\cos(\theta) \text{ dan} \tag{3.1}$$

$$y = l.\sin(\theta) \tag{3.2}$$

Sedangkan dengan menggunakan kinematik inversi kita bisa mendapatkan θ :

$$\theta = \tan^{-1}\left(\frac{y}{x}\right) \tag{3.3}$$

Sedangkan analisis kinematik robot lengan 2 DOF:



Gambar 3.5 *Robot Lengan 2 Sendi (2DOF)*

Kedudukan ujung lengan dinyatakan sebagai P(x,y) di mana:

$$x = l_1 \cos\theta_1 + l_2 \cos(\theta_1 + \theta_2) \tag{3.4}$$

$$y = l_1 \sin\theta_1 + l_2 \sin(\theta_1 + \theta_2) \tag{3.5}$$

Persamaan di atas dapat diperoleh dengan menggunakan analisis kinematik maju. Dengan hukum identitas trigonometri:

$$\cos(a+b) = \cos(a)\cos(b) - \sin(a)\sin(b) \tag{3.6}$$

$$\sin(a+b) = \sin(a)\cos(b) + \sin(b)\cos(a) \tag{3.7}$$

maka kita dapatkan:

$$x = l_1 \cos\theta_1 + l_2 \cos\theta_1 \cos\theta_2 - l_2 \sin\theta_1 \sin\theta_2 \tag{3.8}$$

$$y = l_1 \sin\theta_1 + l_2 \sin\theta_2 \cos\theta_2 + l_2 \cos\theta_1 \sin\theta_2 \tag{3.9}$$

Dari dua persamaan terakhir dan dengan menggunakan analisis kinematik inversi maka kita akan mendapatkan:

$$\theta_2 = \cos^{-1}\left(\frac{x^2 + y^2 - l_1^2 - l_2^2}{2l_1 l_2}\right) \tag{3.10}$$

Dan sudut θ_1 dapat dicari melalui $\theta_1 = \beta - \alpha$

Dengan hukum identitas trigonometri:

$$\tan(a+b) = \frac{\tan(a) - \tan(b)}{1 + \sin(a)\sin(b)} \tag{3.11}$$

Didapat:

$$\tan\theta_1 = \frac{y(l_1 + l_2\cos\theta_2) - x.l_2\sin\theta_2}{x(l_1 + l_2\cos\theta_2) + yl_2\sin\theta_2} \tag{3.12}$$

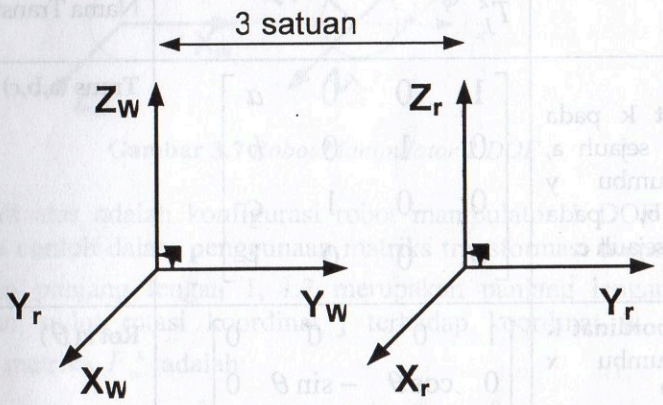
sehingga θ_1:

$$\theta_1 = \tan^{-1}\frac{y(l_1 + l_2\cos\theta_2) - x.l_2\sin\theta_2}{x(l_1 + l_2\cos\theta_2) + yl_2\sin\theta_2} \tag{3.13}$$

3.2 MATRIKS ROTASI DAN TRANSLASI

Untuk menganalisis persamaan 3D dalam bentuk yang umum seperti koordinat (x,y,z) maka kita dapat merepresentasikannya dalam matriks 3x1 $\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$, di mana angka 1 merupakan angka pembagi. Jadi koordinat

(0,2,3) dapat kita representasikan dalam bentuk vektor $\begin{bmatrix} 0 \\ 2 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 8 \\ 12 \\ 4 \end{bmatrix}$ atau $(x,y,z,1)^T$.



Gambar 3.6 *Koordinat Kartesian, World (w) dan Robot (r)*

Misalkan kita mempunyai 2 koordinat kartesian yang berbeda dan satu sama lain dipisahkan sejauh 3 satuan pada sumbu Y. Jika pada gambar di atas koordinat r mempunyai titik asal $(0,3,0,1)^T$, jika dipandang dari koordinat w. Titik asal koordinat w jika kita pandang dari koordinat r, secara mudah kita dapatkan dengan mengurangkan (-3) pada sumbu Y sehingga titik asal koordinat w menjadi $(0,-3,0)^T$ pada koordinat r.

3.2.1 Matriks Transformasi Homogen

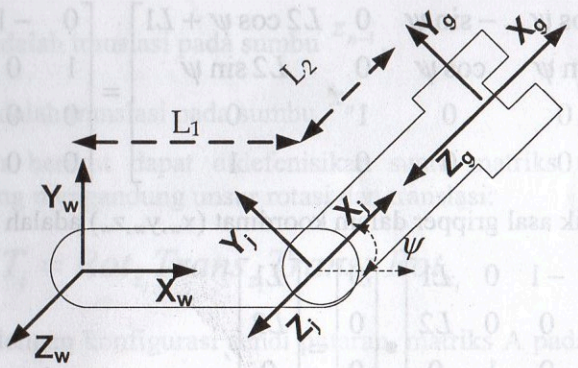
Matriks transformasi homogen merupakan matriks yang digunakan untuk mengonversikan lokasi suatu titik pada koordinat sumbu j ke koordinat sumbu k. Simbol yang biasa dipakai adalah T_j^k untuk merepresentasikan transformasi suatu titik pada koordinat sumbu j menjadi suatu titik yang sama pada koordinat sumbu k.

Simbol F_j^k juga digunakan untuk merepresentasikan transformasi koordinat sumbu j menjadi searah dengan koordinat sumbu k.

$$F_k^j = T_j^k \tag{3.14}$$

Tabel 3.1 *Matriks Transformasi*

F_k^j	T_j^k	Nama Transformasi
Translasi koordinat k pada sumbu x sejauh **a,** pada sumbu y sejauh **b,** pada sumbu z sejauh **c.**	$\begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}$	**Trans (a,b,c)**
Rotasi koordinat k pada sumbu x sebesar θ	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	**Rot x(θ)**
Rotasi koordinat k pada sumbu y sebesar θ	$\begin{bmatrix} \cos\theta & 0 & -\sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	**Rot y(θ)**
Rotasi koordinat k pada sumbu z sebesar θ	$\begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	**Rot z(θ)**



Gambar 3.7 *Robot Manipulator 2 DOF*

Gambar di atas adalah konfigurasi robot manipulator 1 DOF sebagai salah satu contoh dalam penggunaan matriks transformasi homogen. L1 merupakan panjang lengan 1, L2 merupakan panjang lengan 2, ψ merupakan sudut rotasi koordinat j terhadap koordinat w. Dengan demikian matriks F_w^g adalah:

$$F_w^g = \begin{bmatrix} 1 & 0 & 0 & L1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} \cos\psi & -\sin\psi & 0 & 0 \\ \sin\psi & \cos\psi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 & 0 & L2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\psi & -\sin\psi & 0 & L2\cos\psi + L1 \\ \sin\psi & \cos\psi & 0 & L2\sin\psi \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sebagai contoh, berdasarkan gambar di atas, jika ψ sebesar 90° dan titik asal koordinat gripper (x_g,y_g,z_g) adalah (0,0,0) maka titik asal tersebut bila direpresentasikan dalam koordinat (x_w,y_w,z_w) adalah:

$$F_w^g = \begin{bmatrix} \cos\psi & -\sin\psi & 0 & L2\cos\psi + L1 \\ \sin\psi & \cos\psi & 0 & L2\sin\psi \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 & L1 \\ 1 & 0 & 0 & L2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sehingga titik asal gripper dalam koordinat (x_w,y_w,z_w) adalah (L1,L2,0)

$$\begin{bmatrix} 0 & -1 & 0 & L1 \\ 1 & 0 & 0 & L2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} L1 \\ L2 \\ 0 \\ 1 \end{bmatrix},$$

3.2.2 Metode Denavit–Hartenberg (D - H)

Denavit & Hartenberg merepresentasikan analisis hubungan gerak rotasi dan translasi antara lengan–lengan yang terhubung dalam suatu manipulator. Analisis D-H digunakan untuk menentukan koordinat suatu titik seperti pada kinematik maju dan untuk menentukan hubungan dari setiap sendi pada robot manipulator.

Prinsip dasar representasi D – H adalah suatu matriks (4x4) yang berupa sistem koordinat dari suatu link dengan link yang terhubung pada pangkalnya (link sebelumnya). Dalam konfigurasi serial, koordinat (ujung) link-1 dihitung berdasarkan sendi-0 atau sendi pada tubuh robot. Sistem koordinat pada link-2 dihitung berdasarkan posisi sendi-1 yang berada di ujung link-1 dengan mengasumsikan link-1 adalah basis gerakan link-2. Demikian seterusnya, link-3 dihitung berdasarkan link-2, hingga link-ke(n) dihitung berdasarkan link-ke(n-1).

Metode D-H menggunakan 4 buah parameter, yaitu θ, α, d, a. Untuk robot dengan n-DOF maka keempat parameter itu ditentukan hingga yang ke-n. Penjelasannya:

1. θ_n adalah sudut putaran pada sumbu z_{n-1}.
2. α_n adalah sudut putaran pada sumbu x_n.

3. d_n adalah translasi pada sumbu z_{n-1}.
4. a_n adalah translasi pada sumbu x_n.

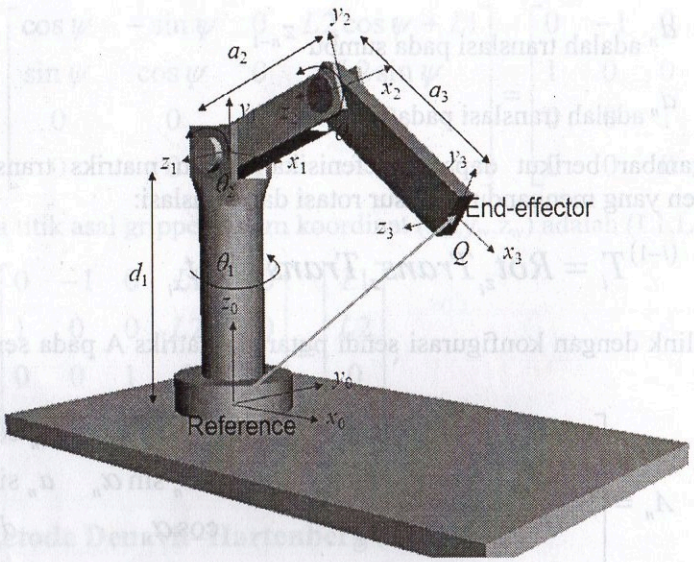
Dari gambar berikut dapat didefenisikan suatu matriks transformasi homogen yang mengandung unsur rotasi dan translasi:

$$^{(i-1)}T_i = Rot_{z_i} Trans_{z_i} Trans_{x_i} Rot_{x_i}$$

Untuk link dengan konfigurasi sendi putaran, matriks A pada sendi ke-n adalah:

$$A_n = \begin{bmatrix} \cos\theta_n & -\sin\theta_n\cos\alpha_n & \sin\theta_n\sin\alpha_n & a_n\cos\theta_n \\ \sin\theta_n & \cos\theta_n\cos\alpha_n & -\cos\theta_n\sin\alpha_n & a_n\sin\theta_n \\ 0 & \sin\alpha_n & \cos\alpha_n & d_n \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Untuk mencari kinematika maju dapat menggunakan metode ini. Metode ini berguna untuk menganalisis gerak rotasi translasi antara lengan-lengan yang terhubung dalam suatu manipulator yang berguna pada pemrograman robot manipulator di industri (misalnya robot SCARA). Prinsipnya ialah transformasi koordinat antar dua link yang berdekatan, dengan hasil suatu matriks 4x4 yang menyatakaan sistem koordinat dari suatu link dengan link yang terhubung pada link sebelumnya.



Gambar 3.8 *Menentukan D-H Parameter*

Pada Gambar 3.8, parameter D-H adalah sebagai berikut:

D-H Parameters

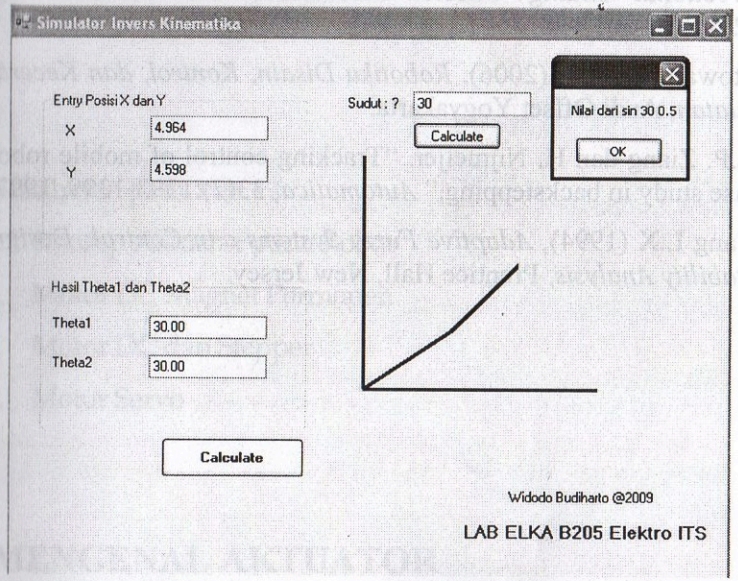
Joint, i	α_i	a_i	d_i	θ_i
1	$\pi/2$	0	d_1	θ_1
2	0	a_2	0	θ_2
3	0	a_3	0	θ_3

$$ {}^0A_1 = \begin{bmatrix} c\theta_1 & 0 & s\theta_1 & 0 \\ s\theta_1 & 0 & -c\theta_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^1A_2 = \begin{bmatrix} c\theta_2 & -s\theta_2 & 0 & a_2c\theta_2 \\ s\theta_2 & c\theta_2 & 0 & a_2s\theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^2A_3 = \begin{bmatrix} c\theta_3 & -s\theta_3 & 0 & a_3c\theta_3 \\ s\theta_3 & c\theta_3 & 0 & a_3s\theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

Gambar 3.9 *Hasil Parameter D-H*

Latihan:

1. Sebutkan definisi kinematika maju dan inversi.
2. Buatlah program kinematika inversi dengan menggunakan C# sebagai berikut:



Gambar 3.10 *Form yang Harus Anda Buat*

Referensi:

[1] E. Sandin, Paul (2003), *Robot Mechanism and Mechanical Devices Illustrated*, Mc-Graw Hill.

[2] C. Dorf, Richard (2000), *The Electrical Engineering Handbook, CRC Press LLC.*

[3] D'souza, A. Vijayakuma and Schall, S (2001). *Learning Inverse Kinematics*. Proc IEEE/RSJ International Conference Intelligent Robots and System, 298-302.

[4] Stefan Florczyk (2005), *Robot Vision, Video-Bsed Indoor Exploration with Autonomous and Mobile Robots*. Wiley VCH Verlag GmbH, Weinheim.

[5] J. Agulló, S. Cardona, dan J. Vivancos, Kinematics *vehicles with directional sliding wheels*. Mechanism and Machine Theory, 22(4):295-301,1987.

[6] Pitowarno, Endra (2006). *Robotika Disain, Kontrol, dan Kecerdasan Buatan*, Andi Offset Yogyakarta.

[7] Z.P. Ziang dan H. Nijmeijer. "Tracking control of mobile robots: A case study in backstepping." *Automatica*, 33(7):1393-1399, 1997.

[8] Wang L.X (1994), *Adaptive Fuzzy Systems adn Control: Design and Stability Analysis,* Prentice Hall, New Jersey.

BAB IV
AKTUATOR

Pokok bahasan pada bab ini:

1. Konsep Aktuator pada Robot
2. Motor DC Magnet Permanen
3. Motor DC dan Stepper
4. Motor Servo

4.1 MENGENAL AKTUATOR

4.1.1 Fungsi Aktuator

Aktuator adalah bagian yang berfungsi sebagai penggerak dari perintah yang diberikan oleh input. Aktuator biasanya merupakan peranti elektro-mekanik yang menghasilkan daya gerakan. Aktuator terdiri dari 2 jenis, yaitu:

1. Aktuator Elektrik
2. Aktuator Pneumatik dan Hidrolik

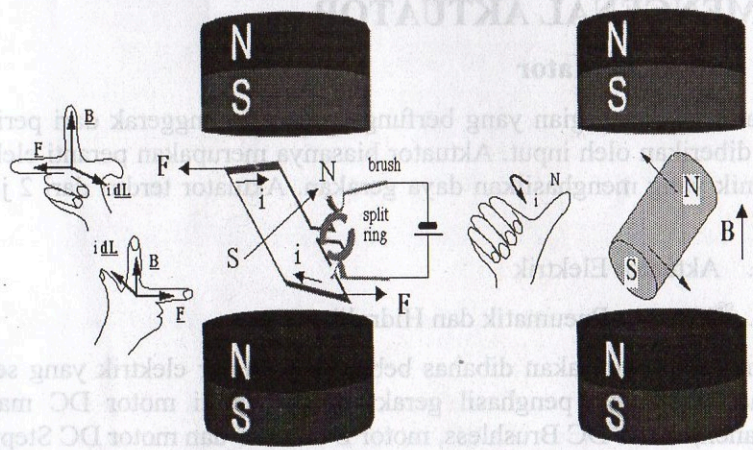
Dalam sub-bab ini akan dibahas beberapa aktuator elektrik yang sering digunakan sebagai penghasil gerak rotasi, seperti motor DC magnet permanen, motor DC Brushless, motor DC Servo dan motor DC Stepper.

4.1.2 Motor DC Magnet Permanen

Motor arus searah (DC) adalah suatu mesin yang berfungsi untuk mengubah tenaga listrik arus searah menjadi gerak atau energi mekanik. Konstruksi dasar motor DC terdiri dari 2 bagian utama, yaitu rotor dan stator. Rotor adalah bagian yang berputar atau *armature,* berupa koil di mana arus listrik dapat mengalir. Stator adalah bagian yang tetap dan menghasilkan medan magnet dari koilnya.

4.1.3 Prinsip Kerja Motor DC

Prinsip kerja motor DC adalah jika ada kumparan dilalui arus maka pada kedua sisi kumparan akan bekerja gaya Lorentz [1]. Aturan tangan kiri dapat digunakan untuk menentukan arah gaya Lorentz, di mana gaya jatuh pada telapak tangan, jari-jari yang direntangkan menunjukkan arah arus, ibu jari yang direntangkan menunjukkan arah gaya. Kedua gaya yang timbul merupakan sebuah kopel. Kopel yang dibangkitkan pada kumparan sangat tidak teratur karena kopel itu berayun antara nilai maksimum dan nol. Kumparan-kumparan tersebut dihubungkan dengan lamel tersendiri pada komutator sehingga motor arus searah tidak berbeda dengan generator arus searah. Di bawah ini ilustrasi dari prinsip kerja motor DC.



Gambar 4.1 *Prinsip Kerja Motor DC*

Dengan mengacu pada hukum kekekalan energi, energi listrik dapat diperoleh dari hasil penjumlahan energi mekanik, energi panas, dan energi di dalam medan magnet. Di dalam medan magnet akan dihasilkan kumparan medan dengan kerapatan fluks sebesar B dengan arus I serta panjang konduktor sama dengan L sehingga diperoleh gaya sebesar F, dengan persamaan sebagai berikut:

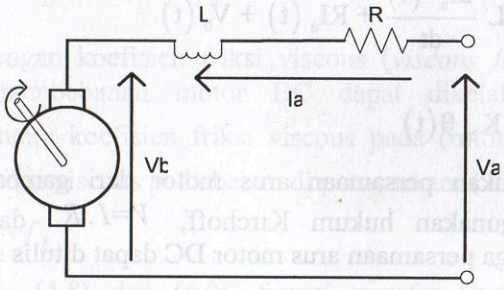
$$F=B.I.L \tag{4.1}$$

Untuk motor DC yang mempunyai jari-jari sepanjang r maka hubungan persamaan diperoleh sebagai berikut:

$$F=B.I.L.r \tag{4.2}$$

4.1.4 Pengontrolan Motor DC Magnet Permanen

Motor magnet permanen adalah motor di mana medan magnet di dalam stator dihasilkan oleh medan magnet permanen. Kekuatan medan magnet itu terbatas sehingga membatasi besar torsi yang dihasilkannya. Rangkaian ekivalen motor DC magnet permanen ditunjukkan pada Gambar 4.2.



Gambar 4.2 *Rangkaian Ekivalen Motor DC [2]*

Keterangan gambar:

V_a : Tegangan armature

I_a : Arus motor

R : Resistansi armature

L : Induktansi lilitan armature

V_b : Tegangan induksi balik (emf)

τ : Torsi motor

ω : Kecepatan putar motor

θ : Sudut putaran poros motor

Sehingga persamaan tegangan Va diperoleh sebagai berikut:

$$V_a = L\frac{dI_a}{dt} + RI_a + K_b\omega \qquad (4.3)$$

di mana Kb adalah konstanta yang diukur dari tegangan yang dihasilkan oleh motor ketika berputar setiap satuan kecepatan (Volt.det/rad). Magnitude dan polaritas Kb adalah fungsi dari kecepatan angular, ω dan arah putar arus motor. Persaman (3.3) dikenal sebagai persamaan motor DC secara umum. Persamaan domain waktu dari persamaan (3.3) dapat dituliskan sebagai berikut:

$$V_a(t) = L\frac{dI_a(t)}{dt} + RI_a(t) + V_b(t) \qquad (4.4)$$

Di mana: $V_b = K_b\,\theta(t)$

Untuk menentukan persamaan arus motor dari gambar dapat dicari dengan menggunakan hukum Kirchoff, $V=I.R$ dan transformasi Laplace sehingga persamaan arus motor DC dapat ditulis sebagai berikut:

$$I_a = \frac{V_a(s) - sK_b\,\theta(s)}{R + sL} \qquad (4.5)$$

Secara umum persamaan torsi output motor dapat ditulis sebagai berikut:

$$\tau(s) = K_{tn}\,I_a(s) \qquad (4.6)$$

Di mana: K_{tn} adalah konstanta proporsional torsi motor.

Dengan mensubtitusikan persamaan (4.5) ke dalam persamaan (4.6) maka didapat persamaan umum torsi motor dalam transformasi Laplace sebagai berikut:

$$\tau(s) = K_{tn}\left[\frac{V_a(s) - sK_b\,\theta(s)}{R + SL}\right] \qquad (4.7)$$

Persamaan torsi motor jika ditinjau dari proses pembebanan dapat dituliskan seperti berikut:

$$\tau(t) = J_{eff}\,\ddot{\theta}(t) + f_{eff}\,\ddot{\theta}(t) \qquad (4.8)$$

Momen inersia pada proses pembebanan motor DC merupakan hasil penjumlahan dari momen inersia poros (rotor) motor (J_m) dan momen inersia beban pada poros motor (J_L) sehingga persamaan momen inersia (J_{eff}) dapat ditulis sebagai berikut:

$$J_{eff} = J_m + J_L \qquad (4.9)$$

Begitu juga dengan koefisien friksi viscous (*viscous friction*) (f_{eff}) pada proses pembebanan motor DC dapat diketahui dari hasil penjumlahan antara koefisien friksi viscous pada (rotor) motor (f_m) dan koefisien friksi viscous pada beban di poros motor (f_L).

$$f_{eff} = f_m + f_L \qquad (4.10)$$

Dari persamaan (4.8) dan (4.9), fungsi transfer (*transfer function*) tegangan armature Va terhadap pergerakan sudut poros motor θ dapat ditulis sebagai berikut:

$$\frac{\theta(s)}{V_a(s)} = \frac{K_{tn}}{s\left[s^2\,J_{eff}\,L + \left(Lf_{eff} + RJ_{eff} + K_{tn}\,K_b\right)\right]} \qquad (4.11)$$

Persamaan di atas dapat disederhanakan sehingga secara singkat dapat ditulis sebagai berikut:

$$\frac{\theta(s)}{V_a(s)} = \frac{K_{tn}}{s\left[sRJ_{eff} + Rf_{eff} + K_{tn}K_b\right]} = \frac{K}{s(T_m s + 1)} \quad (4.12)$$

Dari persamaan di atas, konstanta penguatan motor (K) dan konstanta waktu motor (T_m) dinyatakan sebagai berikut:

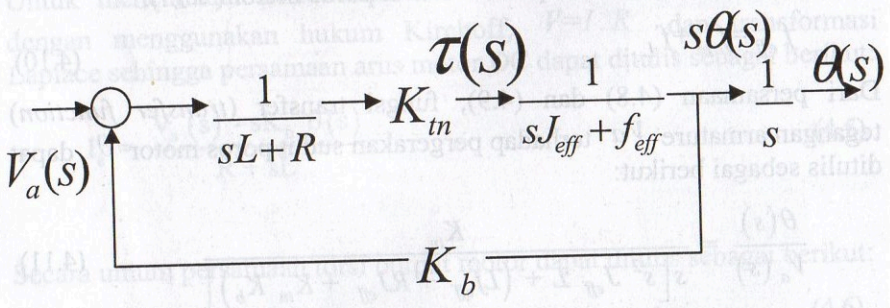
$$K = \frac{K_{tn}}{Rf_{eff} + K_{tn}K_b}$$

$$T_m = \frac{RJ_{eff}}{Rf_{eff} + K_{tn}K_b} \quad (4.13)$$

Jika motor DC tersebut menggunakan gearbox dengan rasio $n = \frac{N_1}{N_2}$ maka persamaan (4.12) dapat dituliskan kembali menjadi sebagai berikut:

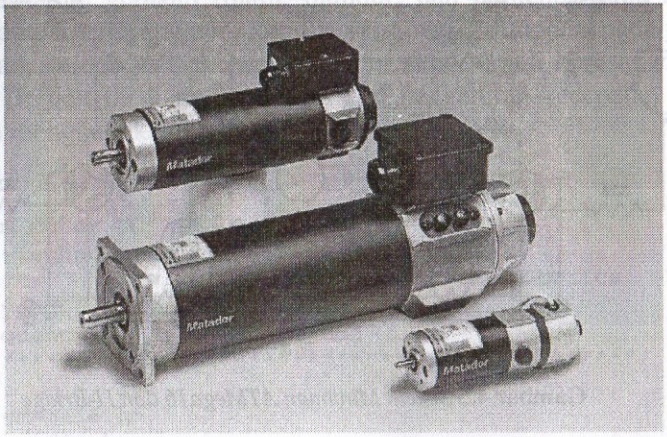
$$\frac{\theta_L(s)}{V_a(s)} = \frac{nK_{tn}}{s\left[sRJ_{eff} + Rf_{eff} + K_{tn}K_b\right]} \quad (4.14)$$

Dengan θ_L merupakan sudut poros output gearbox. Fungsi transfer *closed loop* motor DC magnet permanen dari persamaan (4.12) dapat dinyatakan dalam blok diagram sebagai berikut:



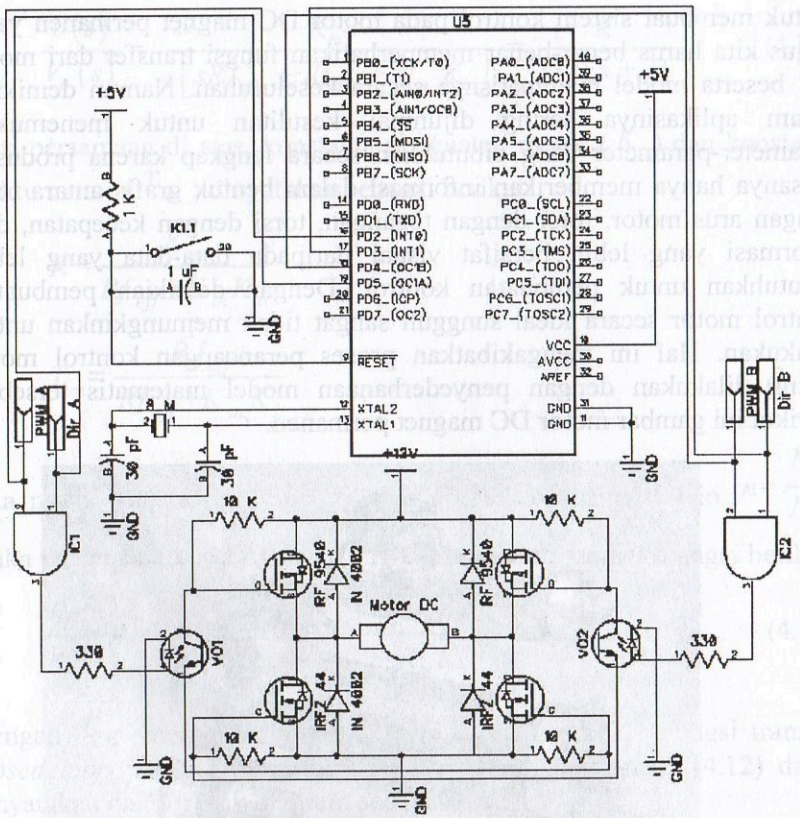
Gambar 4.3 *Fungsi Transfer Motor DC Magnet Permanen [3]*

Untuk membuat sistem kontrol pada motor DC magnet permanen yang bagus kita harus benar-benar memperhatikan fungsi transfer dari motor DC beserta model matematisnya secara keseluruhan. Namun demikian dalam aplikasinya sering dijumpai kesulitan untuk menemukan parameter-parameter yang dibutuhkan secara lengkap karena produsen biasanya hanya memberikan informasi dalam bentuk grafis antara torsi dengan arus motor, torsi dengan tegangan, torsi dengan kecepatan, dan informasi yang lebih bersifat visual daripada data-data yang lebih dibutuhkan untuk pembuatan kontrol. Dengan demikian pembuatan kontrol motor secara ideal sungguh sangat tidak memungkinkan untuk dilakukan. Hal ini mengakibatkan proses perancangan kontrol motor sering dilakukan dengan penyederhanaan model matematis tersebut. Berikut ini gambar motor DC magnet permanen.



Gambar 4.4 *Motor DC Magnet Permanen*

Berikut ini cara pengontrolan motor DC Brushless menggunakan metode PWM (*Pulse Wide Modulation*) di mana untuk generator PWM kita gunakan mikrokontroler ATMega16. Transistor yang digunakan untuk rangkaian H-bridge adalah tipe MOSFET, yaitu IRFZ 44 dan IRF 9540. Transitor tipe MOSFET ini memiliki keunggulan *losses power* yang lebih kecil jika dibanding transistor BJT. Skema rangkaian H-bridge dan sistem minimum dari ATMega16 ditunjukkan pada Gambar 4.5.



Gambar 4.5 *Sistem Minimum ATMega16 dan H-Bridge*

Rangkaian di atas juga dilengkapi dengan optoisolator tipe PIC 817 yang berfungsi sebagai pengaman. Jika ada arus bocor pada H-bridge maka tidak akan membuat sistem minimun menjadi rusak. Sebelum kita membuat program aplikasi untuk pengontrolan motor, kita harus membuat *truth table* rangkaian H-Bridge untuk program yang akan kita rencanakan guna menghindari kondisi-kondisi yang tidak kita inginkan. Untuk lebih jelas mengenai cara kerja dari rangkaian H-bridge, perhatikan Tabel 4.1.

Tabel 4.1 *Truth Table Rangkaian H-Bridge*

Dir A	PWM (A)	Dir B	PWM (B)	Arah Motor
0	0/1	0	0/1	berhenti
1	0	1	0	berhenti
1	0	1	1	maju
1	1	1	0	mundur
1	1	1	1	Terlarang

Dari tabel di atas, Dir (A/B) akan di-AND-kan dengan data PWM untuk membuat tipe-tipe gerakan (maju, mundur, dan berhenti) pada motor. Juga ada kondisi terlarang atau kondisi yang wajib dihindari pada proses pengontrolan, karena pada kondisi tersebut akan terjadi kerusakan pada MOSFET. Berikut ini listing program dari pengaturan kecepatan motor DC dengan menggunakan software Code Vision AVR.

Motor DC.c:

```
// Program motor DC
// By : e-Technology Center 2010.
Chip type           : ATmega16/32 Smart AVR RObotics
Program type        : Application
Clock frequency     : 8.000000 MHz
*****************************************************/

#include <mega16.h>
#include <stdio.h>
#include <stdlib.h>

#define dir_kn  PORTD.3 //mendefinisikan PortD.3 sbg dir_kn
#define dir_kr  PORTD.2    //mendefinisikan PortD.2dir_kr
#define pwm_kn  OCR1A      //mendefinisikan OCR1A  sbg pwm
#define pwm_kr  OCR1B
#define hidup 1      //mendefinisikan hidup aktif high
#define mati 0

void maju (unsigned char kec1,unsigned char kec2)
{
 dir_kn=dir_kr=hidup;// mengaktifkan kedua direction
 pwm_kn=kec1;        // memberi nilai pwm_kn
 pwm_kr=kec2;        // memberi nilai pwm_kr
```

```
}
void stop(void)
{
 dir_kn=dir_kr=mati;          // menonaktifkan kedua direction
}
void belok_kiri(unsigned char kec1,unsigned char kec2)
{
 dir_kn=hidup;                // mengaktifkan direction motor
kanan
 dir_kr=mati;          // menonaktifkan direction motor kiri
 pwm_kn=kec1;          // memberi nilai pwm_kn
 pwm_kr=kec2;          // memberi nilai pwm_kr
}
void belok_kanan(unsigned char kec1,unsigned char kec2)
{
 dir_kn=mati;          // menonaktifkan direction motor kanan
 dir_kr=hidup;               // mengaktifkan direction motor
kiri
 pwm_kn=kec1;          // memberi nilai pwm_kn
 pwm_kr=kec2;          // memberi nilai pwm_kr
}
void main(void)
{
PORTD=0x0C;
DDRD=0x3C;    // mensetting PinD (4,5) sebagai output PWM
TCCR1A=0xF1;          // mensetting PWM 8 bit
TCCR1B=0x03;          // mensetting clock PWM 125 kH
TCNT1H=0x00;
TCNT1L=0x00;
ICR1H=0x00;
ICR1L=0x00;
OCR1AH=0x00;
OCR1AL=0x00;
OCR1BH=0x00;
OCR1BL=0x00;

TIMSK=0x00;
ACSR=0x80;
SFIOR=0x00;
while (1)
      {
      maju(255,255);        //memanggil fungsi maju full
//speed dengan  kecepatan kanan dan kiri maksimum (255).
      belok_kiri(150,0);  // memanggil fungsi belok_kiri
// dengan kec kanan=150 dan kec kiri =0
      belok_kanan(0,100); // memanggil fungsi belok_kanan
// dengan kec kiri=100 dan kec kanan =0
      stop();         // memanggil fungsi berhenti
```



```
      };
}
```

4.2 MOTOR DC BRUSHLESS

Motor *Brushless Direct Current* (BLDC) merupakan motor yang sering digunakan dalam dunia industri, misalnya permobilan, otomasi medis, dan peralatan instrumentasi. Motor BLDC tidak menggunakan sikat (*brush*) untuk pergantian medan magnet (komutasi), tetapi dilakukan secara *elektronic commutated*. Menggunakan motor BLDC mempunyai lebih banyak keuntungan dibanding menggunakan motor DC dan motor induksi biasa. Keuntungan itu antara lain:

1. Kecepatan yang lebih baik untuk melawan karakteristik tenaga putaran
2. Tanggapan dinamis tinggi
3. Efisiensi tinggi
4. Tahan lama sehingga usia pakainya lebih lama
5. Nyaris tanpa suara bila dioperasikan
6. *Speed range* yang lebih luas

Perbedaan utama antara motor DC Brushless dengan motor DC magnet permanen terletak pada pembangkitan medan magnet untuk menghasilkan daya gerak. Pada motor DC magnet permanen, medan magnet yang dikontrol terletak di rotor dan medan magnet tetap berada di stator sedangkan pada motor DC brushless adalah kebalikan daripada motor DC magnet permanen, yaitu medan magnet tetap berada pada rotor sementara medan magnet yang dikontrol berada di stator.

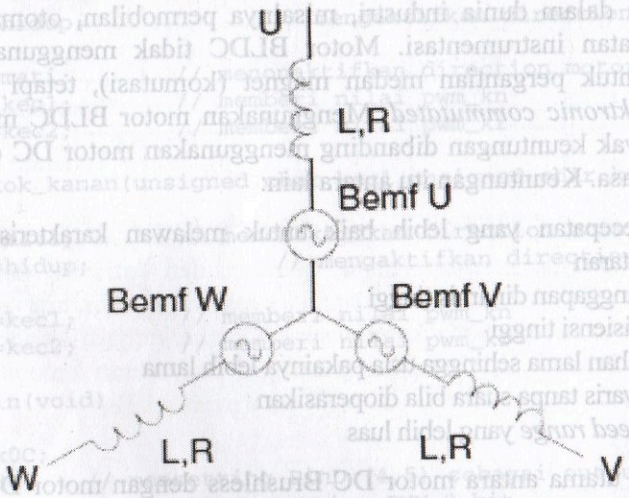
4.2.1 Konstruksi dan Cara Kerja Motor DC Brushless

Konstruksi sederhana dari motor DC brushless terdiri dari medan magnet tetap yang berada pada rotor yang dikelilingi oleh tiga medan magnet gulungan pada stator dan dilengkapi dengan sensor hall.

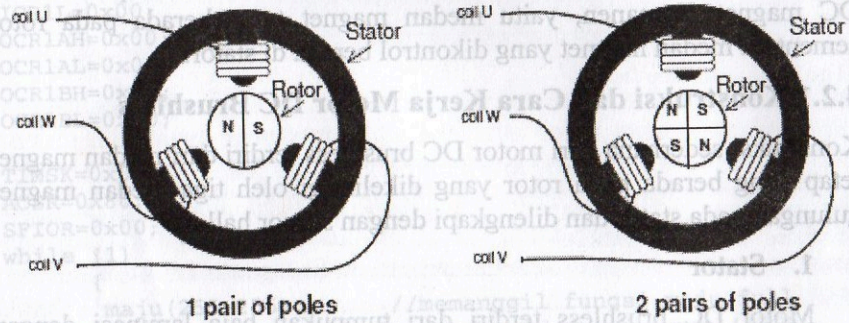
1. Stator

Motor DC brushless terdiri dari tumpukan baja laminasi dengan lilitan yang ditempatkan pada slot. BLDC motor mempunyai tiga gulungan stator yang dihubungkan secara bintang. Masing-masing lilitan dibangun dengan banyak koil yang saling berhubungan sehingga membentuk suatu gulungan. Satu atau lebih koil ditempat-

kan dalam slot dan mereka saling berhubungan untuk membuat suatu gulungan. Masing-masing lilitan ini dibagi-bagikan di atas batas luar stator untuk membentuk suatu bilangan genap kutub. Untuk lebih jelas mengenai susunan tiga gulungan stator U, V, W yang disusun secara bintang, lihat Gambar 4.6.



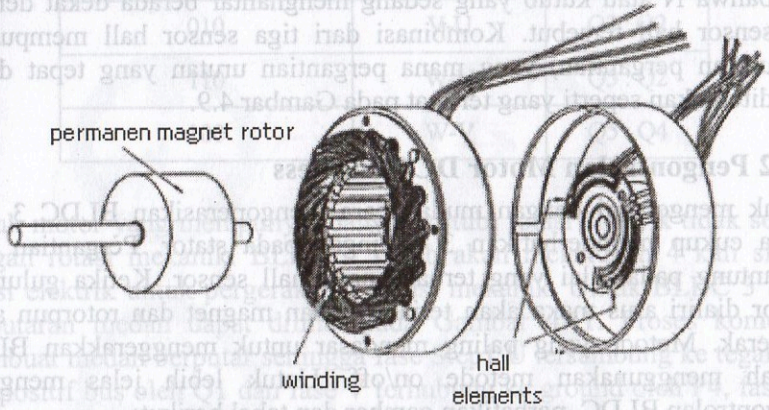
Gambar 4.6 *Konstruksi Dasar Stator*



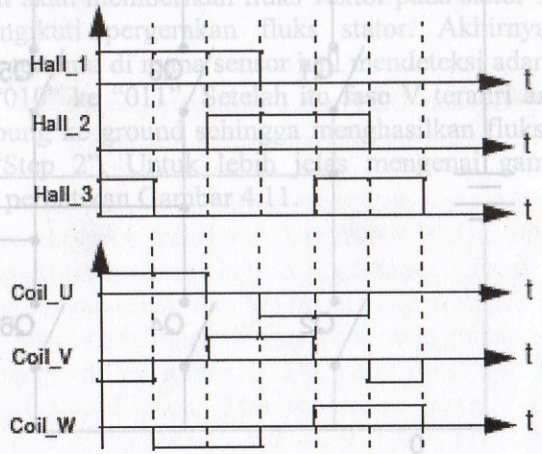
Gambar 4.7 *Konstruksi Dasar Rotor [4]*

2. Rotor

Rotor pada motor DC Brushless terdiri dari magnet yang berjumlah ganjil. Jumlah kutub magnet di rotor juga memengaruhi pergerakan dan torsi yang dihasilkan oleh motor. Semakin banyak kutub maka pergerakan motor akan lebih halus dan *ripple* torsinya kecil. Hal ini karena jumlah step dari rotor tersebut kecil. Bentuk fisik dari stator pada motor DC brushless dapat dilihat pada Gambar 4.6.



Gambar 4.8 *Konstruksi Motor DC Brushless [5]*



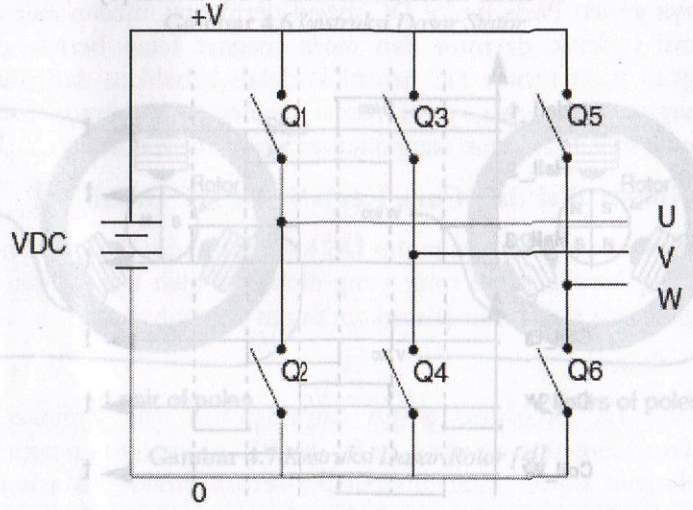
Gambar 4.9 *Kombinasi Sensor Hall*

3. Sensor Hall

Untuk memperkirakan posisi rotor maka motor dilengkapi dengan tiga sensor yang disebut *hall sensors*. Biasanya sensor hall ini dipasang pada suatu papan atau PCB dengan posisi yang sudah tetap, kira-kira selisih 120° antara sensor yang satu dengan yang lain. Jika kutub magnetis rotor lewat dekat sensor hall maka kutub magnetis itu akan memberi isyarat 0 (*low*) atau 1 (*high*). Hal itu menandakan bahwa N atau kutub yang sedang menghantar berada dekat dengan sensor hall tersebut. Kombinasi dari tiga sensor hall mempunyai urutan pergantian, yang mana pergantian urutan yang tepat dapat ditentukan seperti yang terlihat pada Gambar 4.9.

4.2.2 Pengontrolan Motor DC Brushless

Untuk mengetahui dengan mudah cara mengoperasikan BLDC 3 fase Anda cukup memperhatikan 3 gulungan pada stator. Pergantian fase tergantung pada nilai yang terbaca pada hall sensor. Ketika gulungan motor dialiri arus maka akan terjadi medan magnet dan rotorpun akan bergerak. Metode yang paling mendasar untuk menggerakkan BLDC adalah menggunakan metode on/off. Untuk lebih jelas mengenai pengontrolan BLDC, perhatikan gambar dan tabel berikut:

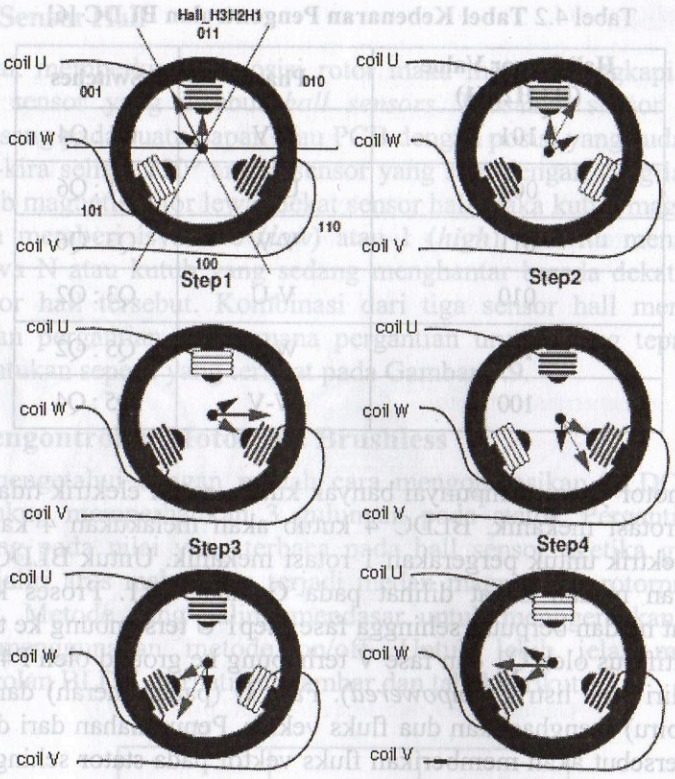


Gambar 4.10 *Kontrol BLDC*

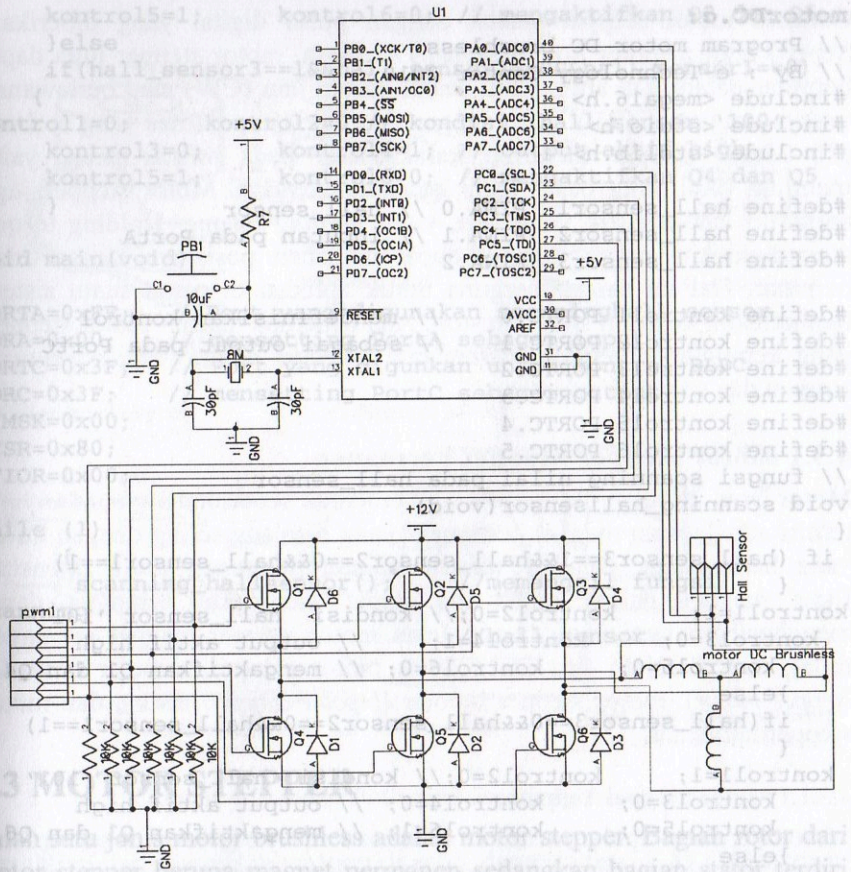
Tabel 4.2 **Tabel Kebenaran Pengontrolan BLDC [6]**

Hall Sensor Value (H3,H2,H1)	Phase	Switches
101	U-V	Q1 : Q4
001	U-W	Q1 : Q6
011	V-W	Q3 : Q6
010	V-U	Q3 : Q2
110	W-U	Q5 : Q2
100	W-V	Q5 : Q4

Untuk motor yang mempunyai banyak kutub, rotasi elektrik tidak sesuai dengan rotasi mekanik. BLDC 4 kutub akan melakukan 4 kali siklus rotasi elektrik untuk pergerakan 1 rotasi mekanik. Untuk BLDC 3 fase, perputaran medan dapat dilihat pada Gambar 4.11. Proses komutasi membuat medan berputar sehingga fase Step1 U tersambung ke tegangan DC positif bus oleh Q1 dan fase V terhubung ke ground oleh P4, fase W tak teraliri arus listrik (*unpowered*). Fase U (panah merah) dan fase V (panah biru) menghasilkan dua fluks vektor. Penjumlahan dari dua fluks vektor tersebut akan memberikan fluks vektor pada stator sehingga rotor mencoba mengikuti pergerakan fluks stator. Akhirnya rotor akan mencapai titik tertentu di mana sensor hall mendeteksi adanya perubahan dari kondisi "010" ke "011". Setelah itu fase V teraliri arus listrik dan fase W terhubung ke ground sehingga menghasilkan fluks vektor stator baru, yaitu "Step 2". Untuk lebih jelas mengenai gambaran proses commutation, perhatikan Gambar 4.11.



Gambar 4.11 *Proses Commutation dan Rotation*



Gambar 4.12 *Rangkaian Kontrol BLDC*

Dari gambar 4.12, hal yang paling mendasar yang harus diperhatikan untuk membuat kontrol sederhana dari motor BLDC adalah perubahan nilai yang terjadi pada sensor hall. Mengacu pada Tabel 4.2, kita dapat menentukan gulungan (U, V dan W) mana yang akan diaktifkan. Transistor yang digunakan pada rangkaian tersebut menggunakan MOSFET tipe N. Metode pengontrolan motor BLDC pada rangkaian tersebut adalah pengontrolan ON/OFF saja. Mikrokontroler yang digunakan adalah ATMega 16 buatan ATMEL. Berikut ini listing program pengontrolan motor BLDC sederhana menggunakan software Code Vision AVR.

motorDC.c:

```
// Program motor DC brushless
// By : e-Technology Center 2010.
#include <mega16.h>
#include <stdio.h>
#include <stdlib.h>

#define hall_sensor1  PINA.0 // hall_sensor
#define hall_sensor2  PINA.1 // inputan pada PortA
#define hall_sensor3  PINA.2

#define kontrol1 PORTC.0     // mendefinisikan kontrol
#define kontrol2 PORTC.1     // sebagai output pada PortC
#define kontrol3 PORTC.2
#define kontrol4 PORTC.3
#define kontrol5 PORTC.4
#define kontrol6 PORTC.5
// fungsi scanning nilai pada hall_sensor
void scanning_hallsensor(void)
{
 if (hall_sensor3==1&&hall_sensor2==0&&hall_sensor1==1)
    {
kontrol1=1;      kontrol2=0;// kondisi  hall_sensor '101'
  kontrol3=0;      kontrol4=1;      // output aktif high
     kontrol5=0;      kontrol6=0;  // mengaktifkan Q1 dan Q4
    }else
    if(hall_sensor3==0&&hall_sensor2==0&&hall_sensor1==1)
    {
 kontrol1=1;      kontrol2=0;// kondisi  hall_sensor '101'
     kontrol3=0;      kontrol4=0;  // output aktif high
     kontrol5=0;      kontrol6=1;  // mengaktifkan Q1 dan Q6
    }else
 if(hall_sensor3==0&&hall_sensor2==1&&hall_sensor1==1)
    {
 kontrol1=0;      kontrol2=0;// kondisi  hall_sensor '011'
     kontrol3=1;      kontrol4=0;  // output aktif high
     kontrol5=0;      kontrol6=1;  // mengaktifkan Q3 dan Q6
    }else
    if(hall_sensor3==0&&hall_sensor2==1&&hall_sensor1==0)
    {
kontrol1=0;      kontrol2=1;// kondisi  hall_sensor '010'
     kontrol3=1;      kontrol4=0;  // output aktif high
     kontrol5=0;      kontrol6=0;  // mengaktifkan Q2 dan Q3
     }else
     if(hall_sensor3==1&&hall_sensor2==1&&hall_sensor1==0)
    {
kontrol1=0;      kontrol2=1;// kondisi  hall_sensor '110'
     kontrol3=0;      kontrol4=0;  // output aktif high
```



```
     kontrol5=1;      kontrol6=0;  // mengaktifkan Q2 dan Q5
     }else
     if(hall_sensor3==1&&hall_sensor2==0&&hall_sensor1==0)
    {
kontrol1=0;      kontrol2=0;// kondisi  hall_sensor '100'
     kontrol3=0;      kontrol4=1;  // output aktif high
     kontrol5=1;      kontrol6=0;  // mengaktifkan Q4 dan Q5
     }
}
void main(void)
{
PORTA=0xFF;   //Port yang digunakan scanninghall-sensor
DDRA=0x00;    // mensetting PortA sebagai input
PORTC=0x3F;   // Port yang digunkan untuk kontrol BLDC
DDRC=0x3F;    // mensetting PortC sebagai output
TIMSK=0x00;
ACSR=0x80;
SFIOR=0x00;

while (1)
     {
      scanning_hallsensor();     //memanggil fungsi
scanning
                                 //hall_sensor
     };
}
```

4.3 MOTOR STEPPER

Salah satu jenis motor brushless adalah motor stepper. Bagian rotor dari motor stepper berupa magnet permanen sedangkan bagian stator terdiri atas beberapa lilitan. Adanya beberapa kumparan pada bagian stator ini mengakibatkan motor stepper dapat berputar setiap step. Besar derajat putar per step (resolusi) yang dimiliki masing-masing motor stepper adalah 0,72° per step. Ini merupakan resolusi terkecil. Nilai 90° per step merupakan resolusi terbesar.

Prinsip kerja motor stepper adalah gerak putarnya dapat dikontrol secara digital. Bila sinyal *clock* diberikan pada rangkaian translator maka salah satu kumparan pada stator akan tersuplai arus selama selang waktu tertentu sehingga motor stepper akan bergerak satu step sesuai spesifikasinya. Bila setiap lilitan pada bagian stator tersuplai arus selama selang waktu tertentu secara berurutan (sekuensial) maka motor stepper akan berputar.

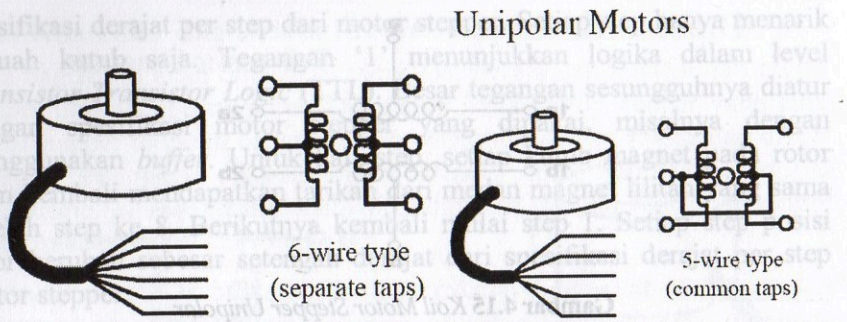
Gerakan motor stepper sesuai dengan pulsa digital yang diberikan. Seperti halnya motor konvensional DC biasa, motor stepper juga dapat berputar dalam dua arah, yaitu searah jarum jam (CW) atau berlawanan arah jarum jam (CCW), yaitu dengan memberikan polaritas yang berbeda [7]. Motor stepper memiliki karakteristik pada *torque-speed,* yaitu memiliki torsi yang besar pada kecepatan rendah. Motor stepper juga memiliki karakteristik yang lain, yaitu *holding torque*. Holding torque memungkinkan motor stepper dapat menahan posisinya ketika tidak berputar. Hal ini sangat berguna untuk aplikasi di mana suatu sistem memerlukan keadaan start dan stop. Ada beberapa macam motor stepper, yang kemudian dibedakan dalam dua kategori besar, yaitu *permanent magnet* dan *variable reluctance*.

4.3.1 Motor Stepper Magnet Permanen

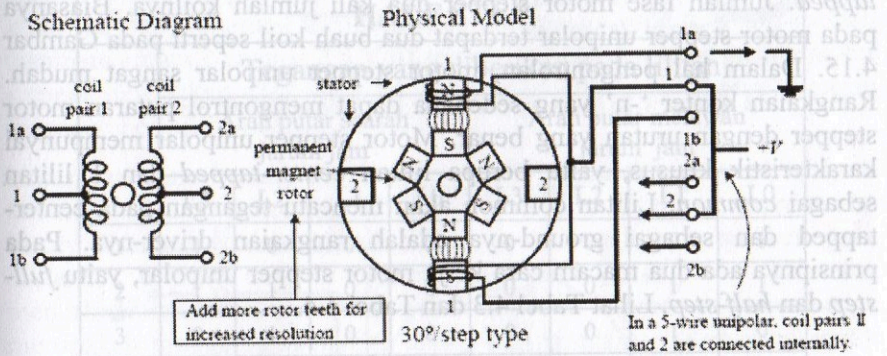
Motor jenis ini memiliki rotor yang berbentuk seperi kaleng bundar yang terdiri atas lapisan magnet permanen yang diselang-seling dengan kutub yang berlawanan. Dengan adanya magnet permanen maka intensitas fluks magnet dalam motor ini dapat meningkat sehingga dapat menghasilkan torsi yang besar. Jenis motor stepper magnet permanen dibagi lagi menjadi tiga jenis, yaitu motor stepper unipolar (*unipolar stepper motor*), motor stepper bipolar (*bipolar stepper motor*), dan motor stepper universal.

4.3.1.1 Motor Stepper Unipolar

Motor stepper unipolar memiliki susunan stator yang mirip dengan motor stepper variable reluctance, hanya saja motor stepper jenis ini menggunakan rotor dengan magnet permanen dan memiliki susunan lilitan dalam yang berbeda. Gambar 4.14 menunjukkan step per 30° pada motor stepper unipolar. Stepper ini terdiri dari 4 kutub (2 pasang lilitan) stator dengan *center tap* di antara pasang lilitan dan 6 rotor magnet permanen bergigi. Center tap mungkin dihubungkan secara internal dan dikeluarkan sebagai 1 kabel atau secara terpisah dengan 2 kabel. Bagian center tap biasanya dihubungkan dengan kutub positif dari tegangan sumber.

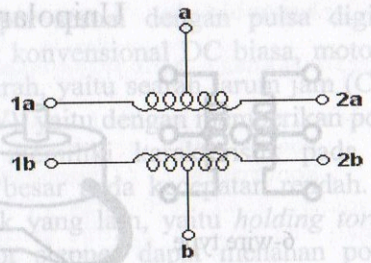


Gambar 4.13 *Tipe-tipe Motor Stepper Unipolar [8]*



Gambar 4.14 *Model Fisik dari Motor Stepper Unipolar*

Ketelitian dari magnet permanen di rotor dapat sampai 1,8° untuk setiap stepnya. Ketika arus mengalir melalui tap tengah pada lilitan pertama maka hal itu akan menyebabkan kutub pada stator bagian atas menjadi kutub utara sedangkan kutub stator pada bagian bawah menjadi kutub selatan. Kondisi ini akan menyebabkan rotor mendapat gaya tarik menuju kutub-kutub ini dan ketika arus yang melalui lilitan 1 dihentikan dan lilitan 2 diberi arus maka rotor akan bergerak lagi menuju kutub-kutub ini. Sampai di sini rotor sudah berputar sampai 30° atau 1 step.



Gambar 4.15 *Koil Motor Stepper Unipolar*

Motor stepper unipolar dapat dikenali dengan adanya lilitan *center-tapped*. Jumlah fase motor stepper dua kali jumlah koilnya. Biasanya pada motor stepper unipolar terdapat dua buah koil seperti pada Gambar 4.15. Dalam hal pengontrolan, motor stepper unipolar sangat mudah. Rangkaian konter '-n' yang sedehana dapat mengontrol putaran motor stepper dengan urutan yang benar. Motor stepper unipolar mempunyai karakteristik khusus, yaitu berupa lilitan *center-tapped* dan 1 lilitan sebagai *common*. Lilitan common akan mencatu tegangan pada center-tapped dan sebagai ground-nya adalah rangkaian driver-nya. Pada prinsipnya ada dua macam cara kerja motor stepper unipolar, yaitu *full-step* dan *half-step*. Lihat Tabel 4.3 dan Tabel 4.4.

Tabel 4.3 *Kontrol Full Step*

***FULL*STEP**								
	Tegangan yang diberikan pada lilitan							
	Arah putar searah jarum jam				Arah putar melawan jarum jam			
	L3	L2	L1	L0	L3	L2	L1	L0
1	1	0	0	0	0	0	0	1
2	0	1	0	0	0	0	1	0
3	0	0	1	0	0	1	0	0
4	0	0	0	1	1	0	0	0

Pada fullstep, suatu titik pada sebuah kutub magnet di rotor akan kembali mendapat tarikan medan magnet stator pada lilitan yang sama setelah step ke-4. Berikutnya dapat diberikan lagi mulai dari step 1. Setiap step, rotor bergerak searah atau berlawanan dengan jarum jam sebesar

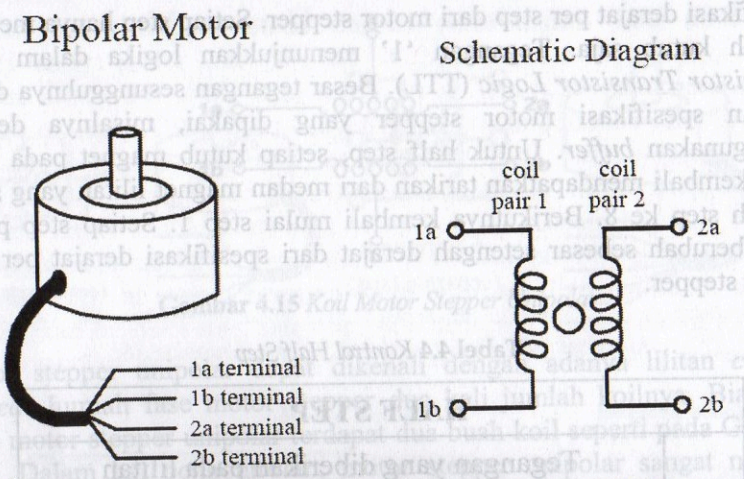
spesifikasi derajat per step dari motor stepper. Setiap step hanya menarik sebuah kutub saja. Tegangan '1' menunjukkan logika dalam level *Transistor Transistor Logic* (TTL). Besar tegangan sesungguhnya diatur dengan spesifikasi motor stepper yang dipakai, misalnya dengan menggunakan *buffer*. Untuk half step, setiap kutub magnet pada rotor akan kembali mendapatkan tarikan dari medan magnet lilitan yang sama setelah step ke 8. Berikutnya kembali mulai step 1. Setiap step posisi rotor berubah sebesar setengah derajat dari spesifikasi derajat per step motor stepper.

Tabel 4.4 *Kontrol Half Step*

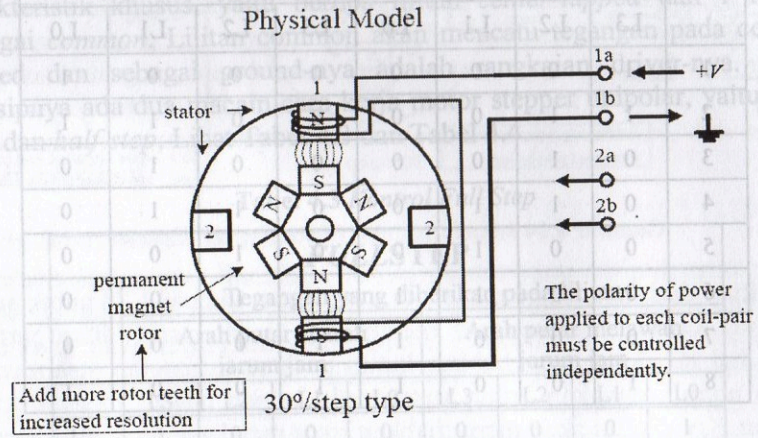
HALF STEP								
	Tegangan yang diberikan pada lilitan							
	Arah putar searah jarum jam				Arah putar melawan jarum jam			
	L3	L2	L1	L0	L3	L2	L1	L0
1	1	0	0	0	0	0	0	1
2	1	1	0	0	0	0	1	1
3	0	1	0	0	0	0	1	0
4	0	1	1	0	0	1	1	0
5	0	0	1	0	0	1	0	0
6	0	0	1	1	1	1	0	0
7	0	0	0	1	1	0	0	0
8	1	0	0	1	1	0	0	1

4.3.1.2 Motor Stepper Bipolar

Motor stepper ini mirip dengan motor stepper unipolar, tetapi sepasang lilitan statornya tidak memiliki center tap. Ini berarti tegangan sumber diberikan pada ujung lilitan yang berbeda. Tegangan suplai diberikan ke ujung lilitan, kemudian ujung lilitan yang berlawanan harus diberi polaritas yang berlawanan. Diagram skematik motor stepper bipolar ditunjukan pada Gambar 4.16 dan model fisik motor stepper bipolar ditunjukkan pada Gambar 4.17.



Gambar 4.16 *Diagram Skematik dari Motor Stepper Bipolar*



Gambar 4.17 *Model Fisik dari Motor Stepper Bipolar*

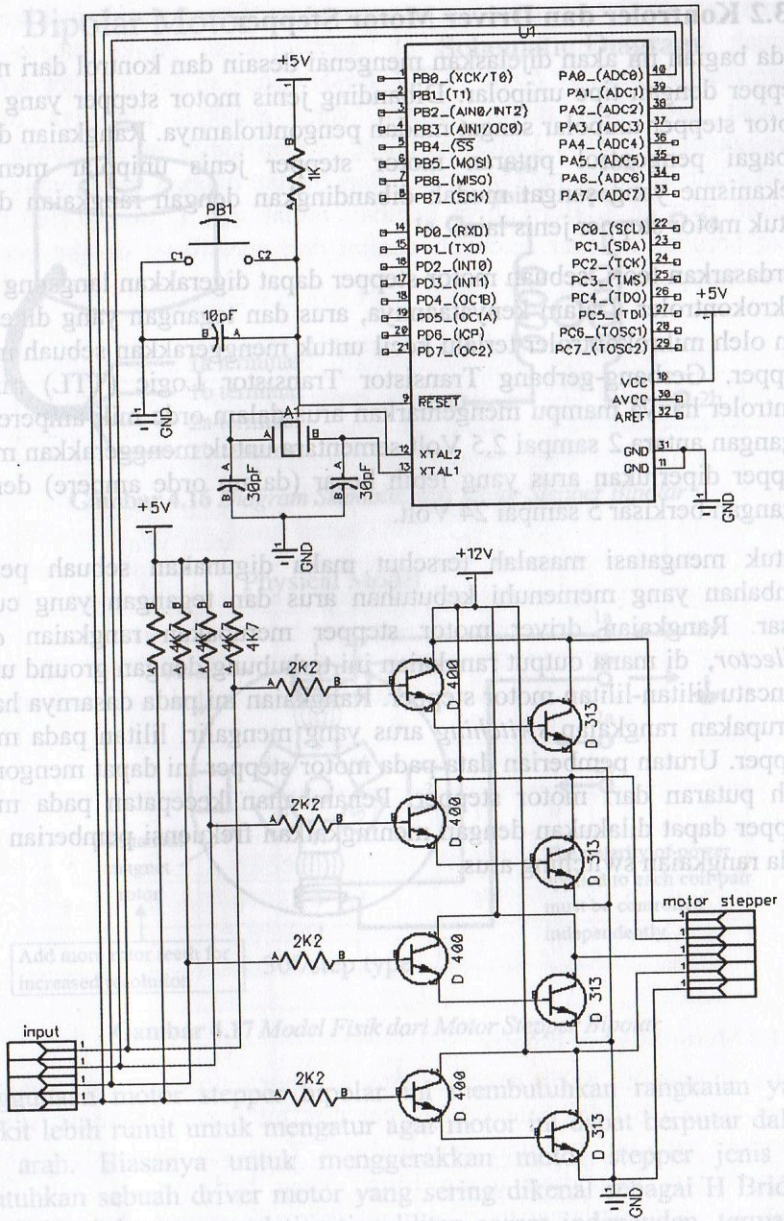
Penggunaan motor stepper bipolar ini membutuhkan rangkaian yang sedikit lebih rumit untuk mengatur agar motor ini dapat berputar dalam dua arah. Biasanya untuk menggerakkan motor stepper jenis ini dibutuhkan sebuah driver motor yang sering dikenal sebagai H Bridge. Rangkaian ini mengontrol tiap-tiap lilitan secara independen, termasuk dengan polaritasnya.

4.3.2 Kontroler dan Driver Motor Stepper

Pada bagian ini akan dijelaskan mengenai desain dan kontrol dari motor stepper dengan tipe unipolar. Dibanding jenis motor stepper yang lain, motor stepper unipolar sangat mudah pengontrolannya. Rangkaian driver sebagai pengontrol putaran motor stepper jenis unipolar memiliki mekanisme yang sangat mudah dibandingkan dengan rangkaian driver untuk motor stepper jenis lain.

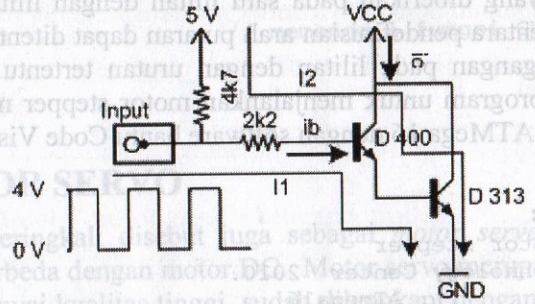
Berdasarkan teori, sebuah motor stepper dapat digerakkan langsung oleh mikrokontroler. Dalam kenyataannya, arus dan tegangan yang dikeluarkan oleh mikrokontroler terlalu kecil untuk menggerakkan sebuah motor stepper. Gerbang-gerbang Transistor Transistor Logic (TTL) mikrokontroler hanya mampu mengeluarkan arus dalam orde mili-ampere dan tegangan antara 2 sampai 2,5 Volt sementara untuk menggerakkan motor stepper diperlukan arus yang lebih besar (dalam orde ampere) dengan tegangan berkisar 5 sampai 24 Volt.

Untuk mengatasi masalah tersebut maka digunakan sebuah peranti tambahan yang memenuhi kebutuhan arus dan tegangan yang cukup besar. Rangkaian driver motor stepper merupakan rangkaian *open collector*, di mana output rangkaian ini terhubung dengan ground untuk mencatu lilitan-lilitan motor stepper. Rangkaian ini pada dasarnya hanya merupakan rangkaian *switching* arus yang mengaliri lilitan pada motor stepper. Urutan pemberian data pada motor stepper ini dapat mengontrol arah putaran dari motor stepper. Penambahan kecepatan pada motor stepper dapat dilakukan dengan meningkatkan frekuensi pemberian data pada rangkaian switching arus.



Gambar 4.18 *Driver Motor Stepper*

Rangkaian pada Gambar 4.18 menggunakan transistor D400 dan D313 yang disusun menurut cara Darlington. Pada rangkaian tersebut digunakan 4 susun, terdiri dari dua buah transistor untuk mengontrol gerakan satu buah motor stepper. Transistor tipe D400 dan D313 digunakan karena transistor ini mampu dilalui arus yang cukup besar, mengingat untuk mengendalikan motor stepper dibutuhkan arus yang cukup besar. Susunan Darlington digunakan untuk menghasilkan penguatan yang besar untuk output mikrokontroler. Karena H_{fe} min transistor D313 = 25 dan H_{fe} transistor D400 = 40 maka H_{fe} total = 40x25 = 1000.



Gambar 4.19 *Cara Kerja Transistor Darlington*

Perhitungan I:

$$4 + 2,\ 2KI_1 + 0,7 + 0,7 = 0$$

$$I_1 = \frac{4 - 0,7 - 0,7}{2,2\ K} = 1,12\ \text{mA}$$

Perhitungan II:

$$-5 + 4,7KI_2 + 2,2KI_2 + 0,7 + 0,7 = 0$$

$$I_2 = \frac{5 - 0,7 - 0,7}{4,7\,K + 2,2\,K} = 0,76\,mA$$

$$I_b = I_1 + I_2 = 1,12\,mA + 0,76\,mA = 1,88\,mA$$

Dengan demikian susunan Darlington mampu menghasilkan I_c minimal:

$$I_c = Hf_e * I_b = 1000 * 1,88A$$

Jadi rangkaian driver stepper tersebut mampu mengalirkan I_c minimal 1,88 A untuk kondisi H_{fe} minimum.

Ada 2 parameter yang harus didefinisikan untuk pengendalian motor stepper, yaitu kecepatan dan arah putaran. Kecepatan dapat diatur dengan menghitung perbandingan waktu/delay yang diberikan untuk mencatu tegangan pada lilitan-lilitan dan selang waktu yang dibutuhkan antara tegangan step yang diberikan pada satu lilitan dengan lilitan lain yang berurutan, sementara pendefinisian arah putaran dapat ditentukan dengan memberikan tegangan pada lilitan dengan urutan tertentu. Berikut ini contoh listing program untuk menjalankan motor stepper menggunakan mikrokontroler ATMega 16 dengan software bantu Code Vision AVR.

Stepper.c :

```
// Program motor stepper
// By : e-Technology Center  2010.
Chip type           : ATmega16
Program type        : Application
Clock frequency     : 8.000000 MHz
*****************************************************/
#include <mega16.h>
#include <delay.h>
#include <stdlib.h>
#include <stdio.h>
void CW (void)        //fungsi untuk gerak motor stepper
{                   //serah jarum jam
 PORTA=0b11111110;    // memberikan output ke PORTA
 delay_ms(10);              // pemberian delay
 PORTA=0b11111101;    // output aktif low
 delay_ms(10);
 PORTA=0b11111011;    // menggerser nilai output yang aktif
 delay_ms(10);
 PORTA=0b11110111;
 delay_ms(10);
}
void CCW (void) // fungsi untuk gerak motor stepper
{               // berlawanan jarum jam
 PORTA=0b11110111;    // memberikan output ke PORTA
 delay_ms(10);              // pemberian delay
 PORTA=0b11111011;    // output aktif low
```



```
 delay_ms(10);
 PORTA=0b11111101;    // menggerser nilai output yang aktif
 delay_ms(10);
 PORTA=0b11111110;
 delay_ms(10);
}
void main(void)
{
PORTA=0x0F;
DDRA=0x0F;            // menseting PORTA sebagai output
ACSR=0x80;
SFIOR=0x00;
while (1)
      {
       CW();          // memenggil fungsi CW
      };
}
```

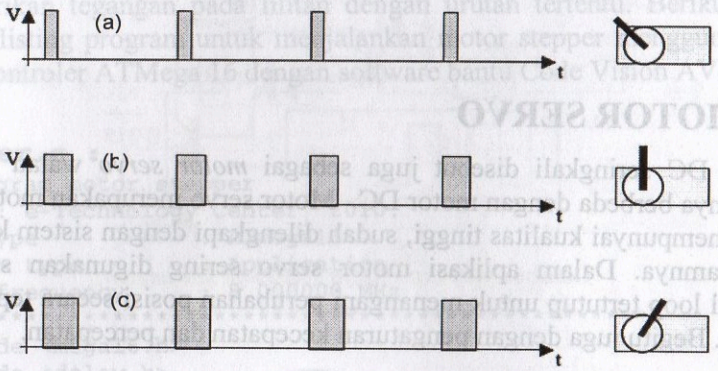
4.4 MOTOR SERVO

Motor DC seringkali disebut juga sebagai *motor servo* walau dalam realitanya berbeda dengan motor DC. Motor servo merupakan motor DC yang mempunyai kualitas tinggi, sudah dilengkapi dengan sistem kontrol di dalamnya. Dalam aplikasi motor servo sering digunakan sebagai kontrol loop tertutup untuk menangani perubahan posisi secara tepat dan akurat. Begitu juga dengan pengaturan kecepatan dan percepatan.

Gambar 4.20 *Model Fisik Motor Servo [9]*

Bentuk fisik dari motor servo dapat dilihat pada Gambar 4.20. Sistem pengkabelan motor servo terdiri dari 3 bagian, yaitu Vcc, Gnd, dan Kontrol (PWM). Penggunaan PWM pada motor servo berbeda dengan penggunaan PWM pada motor DC. Pada motor servo, pemberian nilai PWM akan membuat motor servo bergerak pada posisi tertentu dan kemudian berhenti (kontrol posisi). Pengaturannya dapat dilakukan dengan menggunakan delay pada setiap perpindahan dari posisi awal menuju posisi akhir.

Motor servo dibedakan menjadi 2, yaitu *continuous servo motor* dan *uncontinuous servo motor*. Motor servo kontinu dapat berputar penuh 360° sehingga memungkinkannya untuk melakukan gerak rotasi.



Gambar 4.21 *Cara Pengontrolan Motor Servo*

Prinsip utama pengontrolan motor servo adalah pemberian nilai PWM pada kontrolnya. Perubahan *duty cycle* akan menentukan perubahan posisi dari motor servo. Untuk lebih jelasnya, perhatikan Gambar 4.21. Frekuensi PWM yang digunakan pada pengontrolan motor servo selalu 50 Hz sehingga pulsa dihasilkan setiap 20 ms. Lebar pulsa menentukan posisi servo yang dikehendaki. Sebagai contoh, lebar 0,7 ms akan memutar disk ke posisi paling kiri (-120°) seperti tampak pada Gambar 4.21 (a) dan lebar 1,7 ms akan merotasi disk ke posisi paling kanan (120°) seperti tampak pada Gambar 4.21 (b).

Seperti motor stepper, motor servo merupakan solusi yang baik dan sederhana untuk dunia robotika. Namun motor servo memiliki kekurangan, yaitu tidak dapat memberikan umpan balik keluar. Ketika

memberikan sinyal PWM pada sebuah servo, kita tidak tahu kapan servo akan mencapai posisi yang dikehendaki. Motor Servo HS805 memiliki torsi hingga 20 Kg, GWS 04 10 kg, servo GWS S35, dan S03 memiliki torsi yang lebih rendah. Beberapa tipe lain yang memiliki daya tahan tinggi adalah Hitec-422 dan Hitec-311. Motor servo kontinu dapat berputar 360 derajat, sedangkan motor servo standar hanya dapat berputar sekitar 180 derajat.

Referensi:

[1] Austin Hughes (2006), "*Electric Motor and Drives*", Senior Fellow, School of Electronic and Electrical Engineering, University of Leeds.

[2] EE505 Electrical Engineering Lab, Spring 2007, project paper, 'Lab2.
DC Motor Control using a Microcontroller'.

[3] Pitowarno, Endro (2006), "*Robotika Desain dan Kontrol*", Andi Offset Yogyakarta.

[4] http://www.atmel.com/dyn/resources/prod_documents/doc8138.pdf

[5] T.J.E. Miller (1989), "*Brushless permanent magnet and reluctance motor drive*", Oxford.

[6] Philips (2007), "*Brushless DC motor control using the LPC2141*".

[7] Schuler, CA & McName, WL (1993), *Modern Industrial Electronics,* McGraw Hill, New York.

[8] Scherz, Paul (2000), *Practical Electronics for Investors,* Toronto: McGraw-Hill.

[9] Thomas Bräunl (2003), "*Embedded Robotics, Mobile Robot Design
and Applications with Embedded Systems*" Springer.

BAB V
PEMROGRAMAN DASAR ROBOT

Pokok bahasan pada bab ini:

1. Mengenal Mikrokontroler
2. Memrogram Robot Line Follower
3. Memrogram Robot Avoider
4. Memrogram Robot Tank

5.1 MENGENAL MIKROKONTROLER

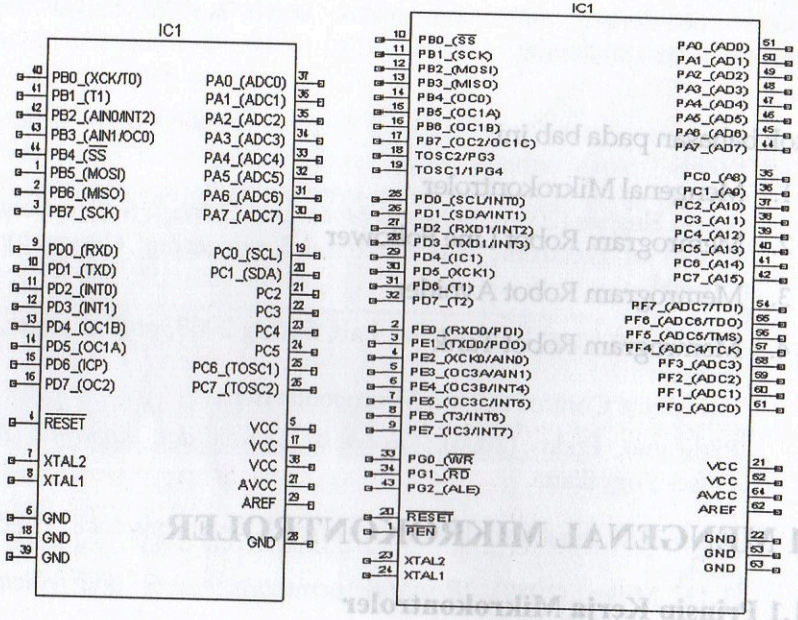
5.1.1 Prinsip Kerja Mikrokontroler

Pemrograman mikrokontroler merupakan dasar dari prinsip pengontrolan kerja robot, di mana orientasi dari penerapan mikrokontroler adalah untuk mengendalikan suatu sistem berdasarkan informasi input yang diterima, yang kemudian diproses oleh mikrokontroler, dan dilakukan aksi pada bagian output sesuai program yang telah ditentukan sebelumnya.

Mikrokontroler merupakan pengontrol utama perangkat elektronika saat ini, termasuk robot tentunya. Mikrokontroler yang terkenal dan mudah didapatkan di Indonesia saat ini ialah 89S51, AVR ATmega 8535, Atmega16, Atmega32, dan Atmega128. Beberapa merek lain yang terkenal, misalnya PIC 16F877 dan Basic Stamp 2.

Mikrokontroler memiliki beberapa port yang dapat digunakan sebagai I/O (*input/output*). Gambar berikut ini merupakan susunan kaki standar

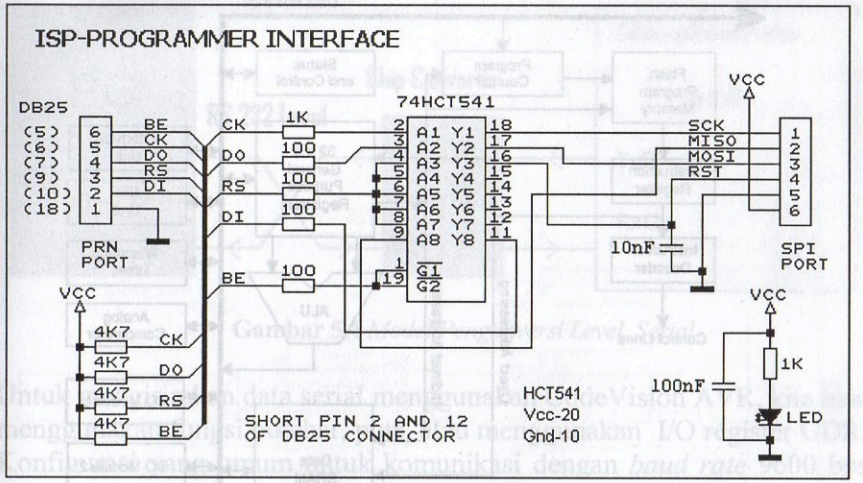
40 pin DIP mikrokontroler AVR ATmega 8535. Sedangkan ATmega128 memiliki kemampuan yang lebih besar, yaitu 128 KB ISP Flash Memory, 4 K EEPROM, 4 K internal SRAM, dan memiliki 5 port I/O.



(a) (b)

Gambar 5.1 *Mikrokontroler ATmega8535 (a) dan ATMega128 (b)*

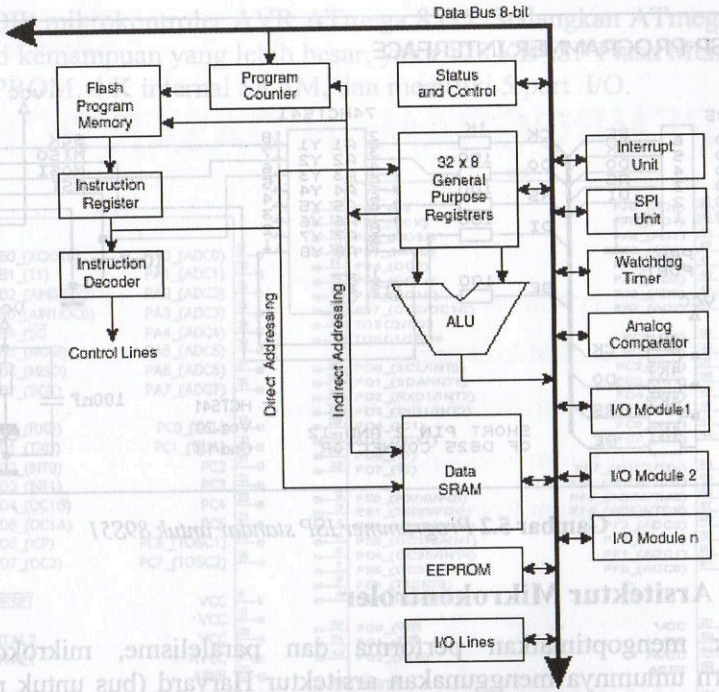
Untuk membangun programmer melalui Parallel Port PC dapat dilakukan dengan mudah, yaitu dengan merangkai beberapa komponen yang ada pada skema Gambar 5.2. Untuk softwarenya dapat menggunakan berbagai software gratis ISP yang dapat Anda download di situs penulis.



Gambar 5.2 *Programmer ISP standar untuk 89S51*

5.1.2 Arsitektur Mikrokontroler

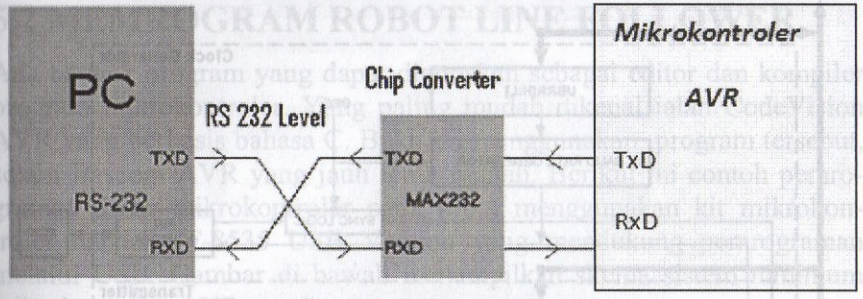
Untuk mengoptimalkan performa dan paralelisme, mikrokontroler modern umumnya menggunakan arsitektur Harvard (bus untuk memori dan program dan data terpisah). Jika dilihat lebih detail, pada bagian pemroses mikrokontroler ini terdapat unit CPU utama untuk memastikan eksekusi program. CPU juga dapat mengakses memori, melakukan kalkulasi, pengontrolan, dan penanganan interupsi, sehingga dapat dihasilkan performa yang tinggi. Hal ini karena instruksi pada memori program dieksekusi dengan *single level pipelining*, sehingga pada saat sebuah instruksi dieksekusi maka instruksi berikutnya dapat diakses dari memori program. Konsep ini memungkinkan instruksi-instruksi dieksekusi pada setiap siklus *clock*.



Gambar 5.3 *Arsitektur Mikrokontroler AVR*

Fast-access Register File berisi 32x8 bit register keperluan umum dengan waktu akses hanya membutuhkan sebuah siklus detak. Teknik ini membuat operasi ALU (*Arithmetic Logic Unit*) hanya membutuhkan 1 siklus. Lebih dahsyat lagi, 6 dari 32 register dapat digunakan sebagai 3 buah 16 bit *indirect address register pointer* untuk pengalamatan ruang data. Satu address pointer dapat digunakan sebagai address pointer untuk *look up table* pada memori flash program. Setiap alamat memori program berisi instruksi sebesar 16 atau 32 bit.

Pada mikrokontroler AVR ATmega 16, pin PD0 dan PD1 digunakan untuk komunikasi serial *USART (Universal Synchronous and Asynchronous Serial Receiver and Transmitter)* yang mendukung komunikasi *full duplex* (komunikasi 2 arah). Gambar berikut ini menampilkan model hubungan antara mikrokontroler dengan PC melalui format serial.

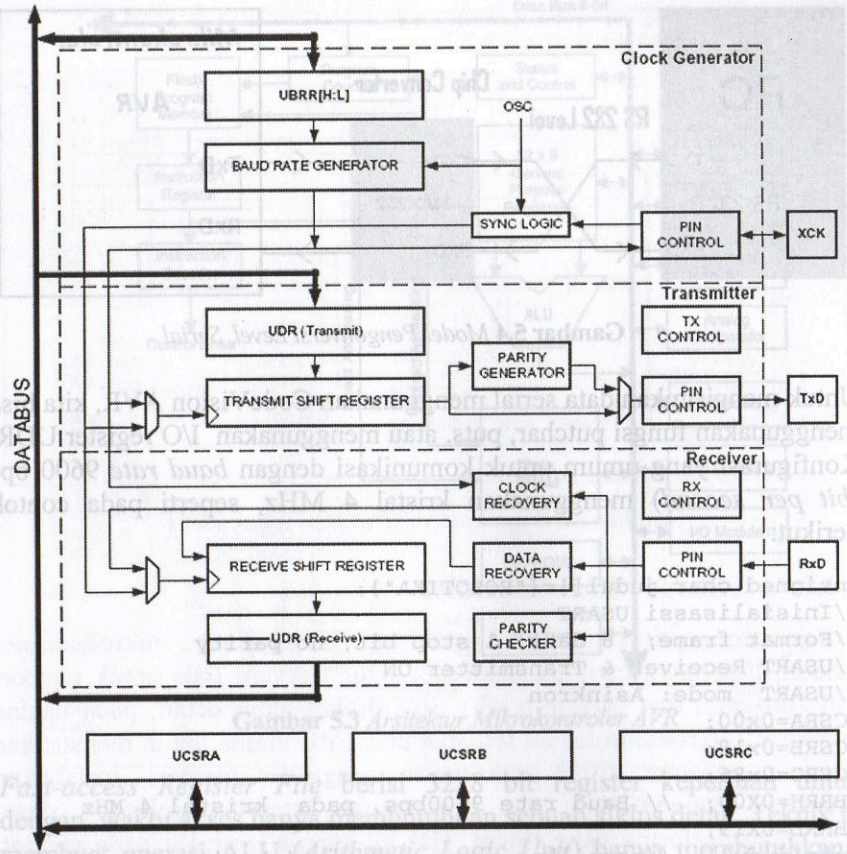


Gambar 5.4 *Model Pengonversi Level Serial*

Untuk mengirimkan data serial menggunakan CodeVision AVR, kita bisa menggunakan fungsi putchar, puts, atau menggunakan I/O register UDR. Konfigurasi yang umum untuk komunikasi dengan *baud rate* 9600 bps (*bit per second*) menggunakan kristal 4 MHz, seperti pada contoh berikut:

```
unsigned char judul[]={"ROBOTIKA"};
//Inisialisassi USART
//Format frame;  8 data, 1 stop bit, no parity
//USART Receiver & Transmitter ON
//USART  mode: Asinkron
UCSRA=0x00;
UCSRB=0x18;
UCSRC=0x86;
UBRRH=0X00;  // Baud rate 9600bps, pada  kristal 4 MHz
UBRRL=0X19;
puts(judul); //tampilkan pesan dan ganti baris
UDR='O';
```

Jika menggunakan sistem *polling* maka flag RXC*(USART Receive Complete, UCSRA.7)* akan diperiksa. Jika flag RXC = 1 berarti ada data pada I/O register UDR yang diterima dan belum dibaca. Bila sebaliknya maka ada data yang diterima dan sudah dibaca, belum ada data baru. Diagram blok USART dapat dilihat pada Gambar 5.5.

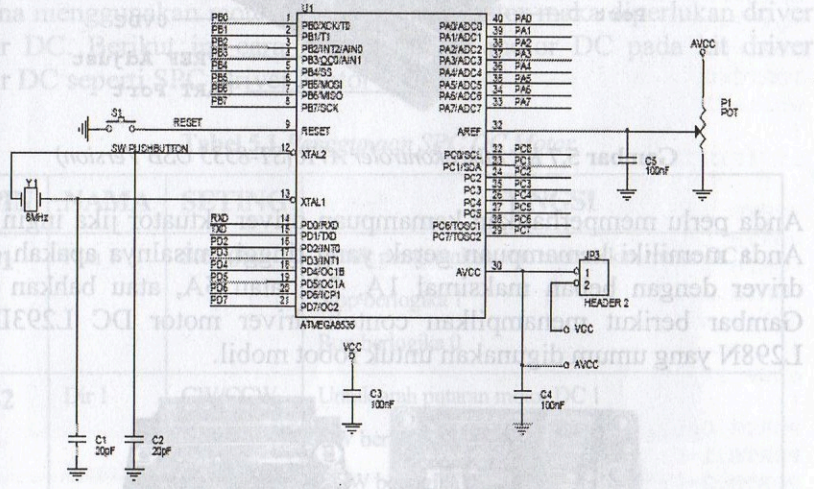


Gambar 5.5 *Diagram Blok USART*

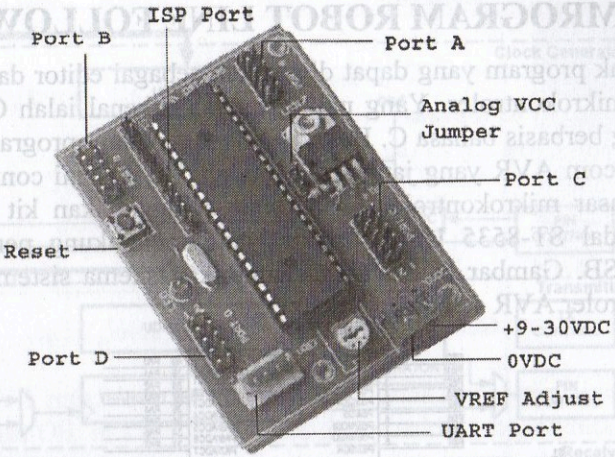
Pada gambar di atas terdapat 3 buah register, yaitu UCSRA, UCSRB, dan UCSRC. *Clock generation logic* membangkitkan clock untuk *transmitter* (pengirim) dan *receiver* (penerima). USART ini mendukung 4 mode operasi clock, yaitu *Normal Asynchronous, double speed asynchronous, master synchronous dan slave synchronous*. Bit UMSEL pada *USART Control And Status Register C* (UCSRC) memilih operasi asinkron atau sinkron tersebut.

5.2 MEMROGRAM ROBOT LINE FOLLOWER

Ada banyak program yang dapat digunakan sebagai editor dan kompiler program mikrokontroler. Yang paling mudah dikenal ialah CodeVision AVR yang berbasis bahasa C. Buku ini menggunakan program tersebut, selain Bascom AVR yang jauh lebih mudah. Berikut ini contoh pemrograman dasar mikrokontroler pada robot menggunakan kit mikrokontroler handal ST-8535 USB Version yang mendukung pemrograman melalui USB. Gambar di bawah menampilkan skema sistem minimum mikrokontroler AVR standar.

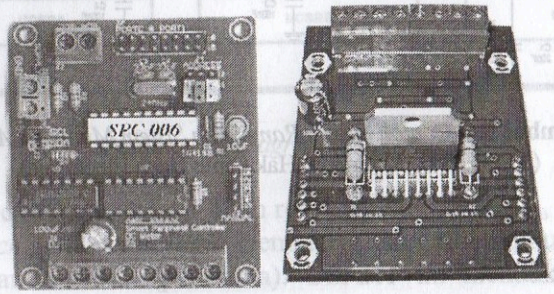


Gambar 5.6 *Contoh Skema Rangkaian Sistem Minimum AVR* (ST-8535 USB version, Hak Cipta Delta Electronic)



Gambar 5.7 *Kit Mikrokontroler AVR (ST-8535 USB Version)*

Anda perlu memperhatikan kemampuan driver aktuator jika ingin robot Anda memiliki kemampuan gerak yang tinggi, misalnya apakah cukup driver dengan beban maksimal 1A, 2A, atau 5A, atau bahkan lebih. Gambar berikut menampilkan contoh driver motor DC L293D dan L298N yang umum digunakan untuk robot mobil.

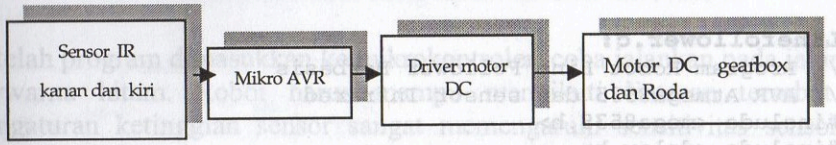


(a) (b)

Gambar 5.8 *Driver Motor DC 1A(1) dan 2A (b)*

Sebagai contoh, pertama cobalah memprogram robot line follower, yaitu robot yang mengikuti garis menggunakan sensor garis berbasis sinar inframerah. Sensor terpasang pada Port C.0 dan C.1, sedangkan driver

motor DC dipasang pada port B.0, B.1, B.2, dan B.3. Diagram blok dari robot line follower sederhana dapat dilihat pada Gambar 5.9.



Gambar 5.9 *Diagram Blok Robot Line Follower*

Karena menggunakan motor DC sebagai aktuator maka diperlukan driver motor DC. Berikut ini cara menggerakkan motor DC pada kit driver motor DC seperti SPC Driver Motor DC:

Tabel 5.1 *Penggunaan SPC DC Motor*

PIN	NAMA	SETING	FUNGSI
S1	Run 1	Stop /Run	Untuk menjalankan atau mematikan motor DC 1 Stop berlogika 1 Run berlogika 0
S2	Dir 1	CW/CCW	Untuk arah putaran motor DC 1 CW berlogika 1 CCW berlogika 0
S3	Run 2	Sto/Run	Untuk menjalankan atau mematikan motor DC 2 Stop berlogika 1 Run berlogika 0
S4	Dir 2	CW/CCW	Untuk arah putaran motor DC 2 CW berlogika 1 CCW berlogika 0

Setelah rangkaian dirakit, isilah program mikrokontroler tersebut menggunakan Codevision AVR dengan menggunakan kode berikut:

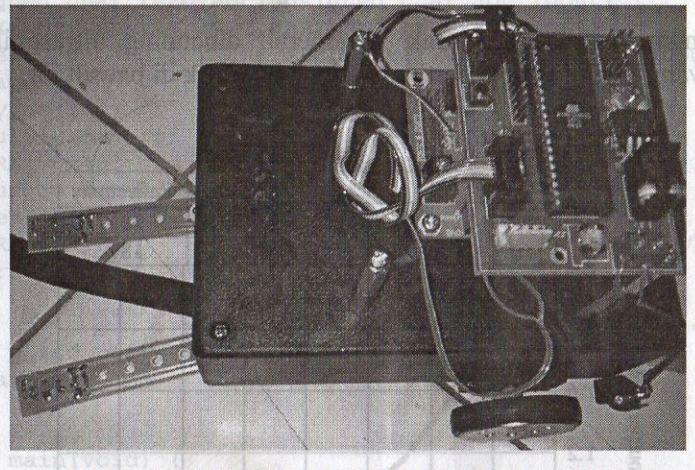
Linefollower.c:

```
// Program Robot Line Follower Berbasis
// AVR Atmega8535 dan sensor Infrared
#include <mega8535.h>
#include <delay.h>

void main(void)
{
DDRB=255;
PORTB=255;
DDRC=0;
PORTC=0;
while(1)
{
if (PINC.0==0 && PINC.1==0)      //maju
{
PORTB.0=0;
PORTB.1=0;
PORTB.2=0;
PORTB.3=1;
delay_ms(20);
}
else if (PINC.0==1 && PINC.1==0)// belok kanan
{
PORTB.0=0;
PORTB.1=0;
PORTB.2=0;
PORTB.3=0;
delay_ms(100);
}
else  if (PINC.0==0 && PINC.1==1) //belok kiri
{
PORTB.0=0;
PORTB.1=1;
PORTB.2=0;
PORTB.3=1;
delay_ms(100);
}

else      //mundur
{
PORTB.0=0;
PORTB.1=1;
PORTB.2=0;
```



```
PORTB.3=0;
delay_ms(20);
}}}
```

Setelah program dimasukkan ke mikrokontroler, coba jalankan pada jalur berwarna hitam. Robot harus mampu mengikuti lintasan tersebut. Pengaturan ketinggian sensor sangat memengaruhi sensitivitas sensor garis. Untuk mengatur kecepatan putar motor dapat digunakan teknik PWM (*Pulse Width Modulation*). Hasil robot dapat dilihat pada Gambar 5.10.



Gambar 5.10 *Robot Line Follower yang Sukses Dijalankan*

5.3 MEMROGRAM ROBOT AVOIDER

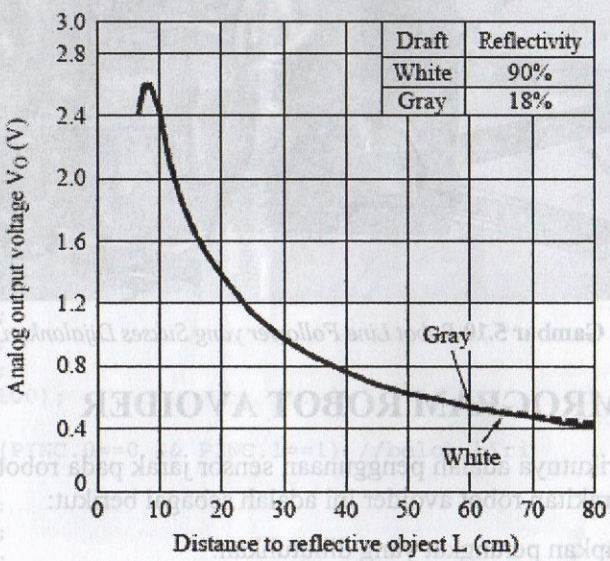
Contoh berikutnya adalah penggunaan sensor jarak pada robot. Langkah-langkah perakitan robot avoider ini adalah sebagai berikut:

1. Siapkan perangkat yang dibutuhkan:
   a. Sistem mikrokontroler AVR
   b. Driver DC Motor
   c. Roda trolley
   d. Sensor jarak Sharp GP2D12

Bahasa pemrograman yang digunakan dapat berbasis USB, port paralel, dan serial. Hubungkan keluaran sensor Sharp GP2D12 di port A.0. Berikut ini diagram blok sistemnya:

Gambar 5.11 *Diagram Blok Robot Avoider*

Kekurangan dari sensor jarak inframerah dibanding ultrasonik ialah output yang tidak linear seperti tampak pada grafik di bawah ini.



Gambar 5.12 *Grafik Antara Jarak Terukur dengan Tegangan Output Sensor*

RobotAvoider.c:

```
// Percobaan baca data sensor jarak Sharp GP2D12
// Untuk Robot Avoider (Penghindar halangan)
// SPC DC Motor di Port B.0,B.1, B.2 dan B.3
```



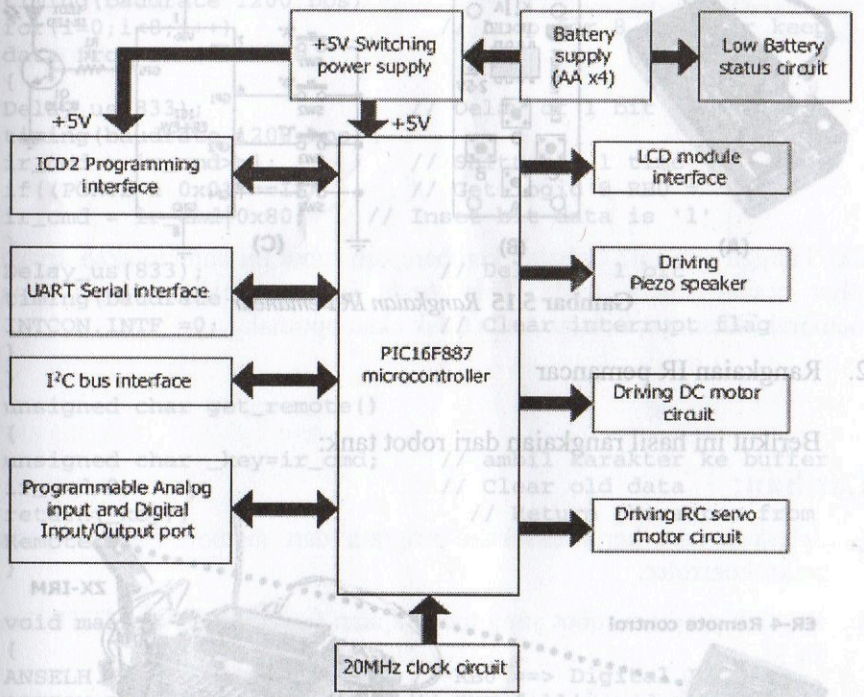
```
// Sensor Jarak Sharp GP2D12 di Port A.0
// pasang LCD 2x16 di port C

#include <mega8535.h>
#include <stdio.h>
#include <delay.h>
// include the LCD driver routines
#include <lcd.h>

#define ADC_VREF_TYPE 0x60
flash unsigned char string1[]={"data  adc:  %d; "};
// Baca 8 bit terpenting
unsigned char read_adc(unsigned char adc_input) {
ADMUX=adc_input | (ADC_VREF_TYPE & 0xff);
// Delay needed for the stabilization of the ADC input
voltage
delay_us(10);
// Mulai konversi ADC
ADCSRA|=0x40;
// Tunggu konversi ADC selesai
while ((ADCSRA & 0x10)==0);
ADCSRA|=0x10;
return ADCH;
}
#asm
.equ __lcd_port=0x15 ;PORTC
#endasm


void main(void) {
// Inisialisasi Port B dan D sebagai output
PORTB=0xFF;
DDRB=0xFF;
PORTD=0xFF;
DDRD=0xFF;
// initialize the LCD for
// 2 lines & 16 columns
lcd_init(16);

// USART initialization
// Communication Parameters: 8 Data, 1 Stop, No Parity
// USART Receiver: On
// USART Transmitter: On
// USART Mode: Asynchronous
// USART Baud Rate: 9600
UCSRA=0x00;
UCSRB=0x18;
UCSRC=0x86;
```

```
UBRRH=0x00;
UBRRL=0x19;
// ADC initialization
// ADC Clock frequency: 31,250 kHz
// ADC Voltage Reference: AVCC pin
// ADC Auto Trigger Source: Free Running
// Menggunakan 8 bit data
ADMUX=ADC_VREF_TYPE & 0xff;
ADCSRA=0xA7;
SFIOR&=0x1F;
while (1)
      {
     PORTD=read_adc(0);      // Baca data sensor di
                             //PA.0
     printf(string1,PORTD); //Tampilkan di
                            // Hyperterminal PC
      if (PORTD<100)
      {
      PORTB=10;  // Maju
      lcd_gotoxy(0,1);
      // tampilkan pesan
      lcd_putsf("Maju");
      delay_ms(100;
      }
      else
      {
      PORTB=0;  // Mundur
      delay_ms(100);
      PORTB=1; //Belok kanan
      lcd_gotoxy(0,1);
      // tampilkan pesan
      lcd_putsf("Belok Kanan");
      delay(ms(50);

      }
};
}
```

Sensor jarak mengeluarkan tegangan analog yang sesuai dengan jarak yang terukur, meskipun kekurangan sensor inframerah ini ialah tidak linear. Oleh mikrokontroler tegangan analog tersebut diubah menjadi data digital untuk kemudian nilainya dibandingkan dengan program, apakah nanti harus maju, belok, atau mundur. Setelah itu mikrokontroler akan memberikan sinyal tertentu ke driver motor DC agar menggerakkan motor DC sesuai dengan yang telah kita tentukan di program.

5.4 MEMROGRAM ROBOT TANK

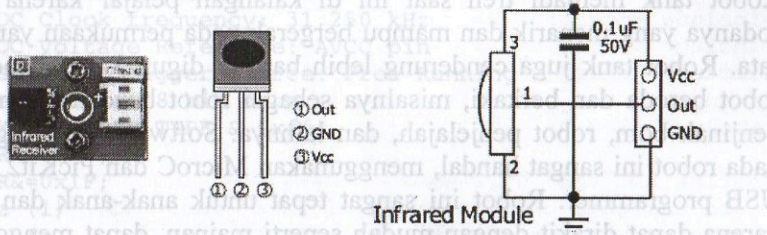
Robot tank menjadi tren saat ini di kalangan pelajar karena bentuk rodanya yang menarik dan mampu bergerak pada permukaan yang tidak rata. Robot tank juga cenderung lebih banyak digunakan dibandingkan robot beroda dan berkaki, misalnya sebagai robot bencana alam, robot penjinak bom, robot penjelajah, dan lainnya. Software yang digunakan pada robot ini sangat handal, menggunakan MicroC dan PicKit2 sebagai USB programmer. Robot ini sangat tepat untuk anak-anak dan pelajar karena dapat dirakit dengan mudah seperti mainan, dapat menggunakan body dan roda tank dari Tamiya. Kit mikrokontroler robot ini menggunakan mikrokontroler PIC 16F877 yang terkenal cepat dan kaya akan fitur pengontrolan.



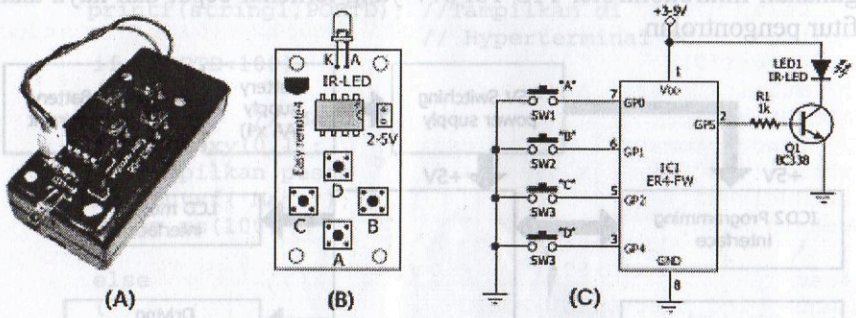
Gambar 5.13 *Diagram Blok Robot Standar Berbasis 16F877*

Untuk mencoba menggerakkan motor tank yang dikendalikan oleh remote IR, komponen yang dibutuhkan dan sudah ada di dalam paket roboPICA2.0 adalah:

1. Modul penerima IR



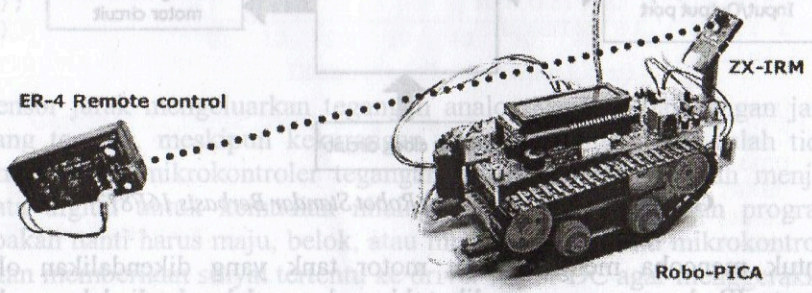
Gambar 5.14 *Rangkaian IR Penerima Standar*



Gambar 5.15 *Rangkaian IR Pemancar*

2. Rangkaian IR pemancar

Berikut ini hasil rangkaian dari robot tank:



Gambar 5.16 *Robot Tank Dikendalikan Remote*

Remote.c:

```
//Program robot tank

//Hak Cipta Inex 2010
#include <motor.h>
char *text = "ER-4 Remote";    // Pesan
unsigned char ir_cmd=0;
void interrupt()
{
unsigned char i;                    // counter
if(INTCON.INTF)                     // Cek interrupt flag RB0
(Falling edge)
{
Delay_us(416);                    // Delay 1/2 of 1 bit
timing(baudrate 1200 bps)
for(i=0;i<8;i++)                   // Loop for 8 time for keep
data from ER-4 Remote
{
Delay_us(833);                  // Delay of 1 bit
timing(baudrate 1200 bps)
ir_cmd = ir_cmd>>1;             // Shitt bit 1 time
if((PORTB & 0x01)==1)           // Get logic @ RB0 = '1'?
ir_cmd = ir_cmd|0x80;     // Inset bit data is '1'
}
Delay_us(833);                    // Delay of 1 bit
timing(baudrate 1200 bps)
INTCON.INTF =0;                   // Clear interrupt flag
}
}
unsigned char get_remote()
{
unsigned char _key=ir_cmd;     // ambil karakter ke buffer
ir_cmd=0;                      // Clear old data
return(_key);                    // Return character from
Remote
}

void main()
{
ANSELH.F4=0;                  // RB0 ==> Digital IO
OPTION_REG.INTEDG = 0;        // INT falling edge
INTCON.INTE =1;               // Enable INT/PB0
INTCON.GIE =1;                // Enable Global interrupt

Lcd_Init(&PORTD);             // Initialize LCD connected to
PORTD
```

```
Lcd_Cmd(Lcd_CLEAR);              // Clear display
Lcd_Cmd(Lcd_CURSOR_OFF);         // Turn cursor off
Lcd_Out(1, 1, text);             // Print text to LCD, 2nd row,
1st column
Sound_Init(&PORTC,0);
Sound_Play(2500,400);
while(1)                                  // Infinite loop
{
char key=get_remote();                    // Get Remote
if((key=='a')||(key=='A'))                // Button A press?
{ Backward(255);Delay_ms(50); }
else if((key=='b')||(key=='B'))           // Button B press?
{ S_Right(255);Delay_ms(50);  }
else if((key=='c')||(key=='C'))           // Button C press?
{ S_Left(255);Delay_ms(50);   }
else if((key=='d')||(key=='D'))           // Button D press?
{ Forward(255);Delay_ms(50);}
else Motor_Stop();
}

}
```

Ok! Tunggu apalagi? Cobalah kembangkan robot ini dan ciptakan kreasi robot sesuai selera Anda. Bila Anda hanya membaca saja tanpa mempraktikkannya, belajar Anda tidak akan optimal.

Latihan:

1. Jelaskan cara kerja eksekusi program dan memori internal pada mikrokontroler.
2. Buatlah program robot yang menerapkan fasilitas interupsi INT0 dan INT1.
3. Buatlah rangkaian sistem minimum robot menggunakan mikrokontroler AVR Atmega32 dan driver motor DC L 298 dengan penampil LCD untuk indikator pergerakan robot.

4. Pelajari berbagai sensor jarak ultrasonik seperti PING dan SRF04, serta modul komunikasi wireless untuk *embedded system* seperti Xbee.

Referensi:

[1]. Braunl, Thomas (2003) *Embedded Robotics*, Springer-Verlag berlin Heidelberg

[2]. Budiharto, Widodo (2009), *Membuat Robot Cerdas edisi revisi*, Elex Media Komputindo.

[3]. Budiharto, Widodo (2008), *Membuat Robot Humanoid*, Elex Media Komputindo

[4]. www.atmel.com

BAB VI
NAVIGASI ROBOT

Pokok bahasan pada bab ini:

1. Navigasi Indoor dan Outdoor Robot
2. Motion Planning pada Robot

6.1 PENDAHULUAN

Pada saat bergerak, robot membutuhkan navigasi yang tepat. Begitu juga dengan manusia. Jika berjalan tanpa informasi navigasi yang tepat maka gerak manusia sama saja dengan robot yang bergerak tanpa sensor, dapat menabrak benda-benda di sekitarnya. Navigasi robot mobil menggunakan computer vision dapat diklasifikasikan menjadi bagian indoor dan outdoor. Navigasi outdoor lebih sulit karena umumnya berarea lebih luas, penuh halangan, dan lintasan yang dilalui robot tidak terstruktur. Strategi perencanaan gerak (*motion planning*) juga perlu diperhatikan untuk mencapai hasil yang optimal.

6.2 NAVIGASI ROBOT INDOOR

Salah satu teknik navigasi indoor pada robot ialah dengan menggunakan peta (*map environment*). Pada model navigasi indoor, biasanya urutan image digunakan untuk menentukan persepsi lingkungan yang penting. Pengetahuan tentang lingkungan sebelumnya dapat digunakan sebagai sumber untuk klasifikasi navigasi indoor. Navigasi berbasis peta lingkungan ini juga dapat diklasifikasikan menjadi:

1. Navigasi dengan map

   Di mana model geometris lingkungan yang dibuat oleh pengguna digunakan.

2. Navigasi berdasarkan pembuatan map

   Sensor robot mobil digunakan untuk membangun peta lingkungan yang dapat digunakan untuk merencanakan pergerakan robot

3. Navigasi tanpa map

   Sistem ini tidak menggunakan representasi lingkungan, tetapi lebih pada mengenal obyek yang dapat dilintasi. Perintah pergerakan dihasilkan menggunakan informasi fitur lingkungan. Elemen–elemen ini, misalnya dinding, pintu, meja, dan lainnya. Oleh karena itu posisi relatif tidak terlalu penting, tetapi navigasi ini merujuk ke elemen tersebut. Strategi paling penting pada navigasi ini adalah:

   a. *Navigasi berbasis optical flow*
      Riset pertama dilakukan oleh Santos-Victor dan berdasarkan persepsi visual dari serangga [1]. Sistem persepsi lebah menggunakan fitur pergerakan, di samping informasi jarak. Hal ini sama dengan tampilan perspektif ganda dan persepsi *optical flow* yang menghasilkan performa penghindaran tubrukan yang lebih baik

   b. *Strategi pengenalan objek dengan database*
      Metode ini terdiri dari pembuatan sebuah database dengan fitur-fitur image penting yang dapat diasosiasikan dengan perintah dan kontrol robot. Image-image yang penting berisi atribut yang relevan yang digunakan oleh neural network untuk menentukan posisi akhir dari pergerakan robot. Representasi path dengan mengeset image-image yang berbeda dapat memberikan informasi perintah kontrol arah pergerakan robot, sebagai fungsi dari image yang beda yang diperoleh berhubungan dengan trayektori yang dipilih.

   c. *Strategi pengenalan objek tanpa database*
      Navigasi tanpa map dapat didasarkan pencarian obyek, dan navigasi simbolik diusulkan. Informasi perintah harus menyertakan landmark untuk mengizinkan path dan pengenalan obyek seperti "pergi ke pintu", "pergi ke meja berikutnya", dan seterusnya. Metode machine vision yang diusulkan terdiri dari sistem trinokular yang digunakan untuk deteksi landmark untuk merencanakan pergerakan robot.

6.3 NAVIGASI OUTDOOR

Metode navigasi outdoor umumnya menyertakan landmark dan deteksi penghalang, pembuatan map, dan estimasi posisi. Metode ini dapat diklasifikasikan sebagai fungsi dari:

1. Navigasi outdoor terstruktur

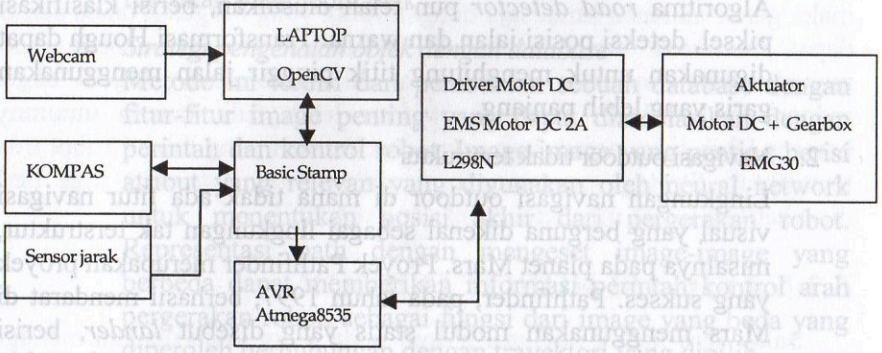
   Sistem vision outdoor pertama kali menggunakan dasar penghindar halangan. Jalanan sering digunakan sebagai area riset navigasi outdoor dengan lingkungan terstruktur dengan variabel berupa lebar jalan, variasi cahaya dan bayangan. Proyek riset terkenal untuk navigasi outdoor dilakukan oleh Carniege-Melon University [4] dengan proyek bernama Robot Navlab yang menggunakan neural network. Navlab menggunakan sistem machine vision untuk mendeteksi dan menghidari halangan, di mana pemrosesan image didasarkan atas metode *space-resolution*, di mana image resolusi tinggi digunakan untuk mendeteksi tekstur dan resolusi rendah untuk menentukan warna. Algoritma *road detector* pun telah diusulkan, berisi klasifikasi piksel, deteksi posisi jalan dan warna. Transformasi Hough dapat digunakan untuk menghitung titik pinggir jalan menggunakan garis yang lebih panjang.

2. Navigasi outdoor tidak terstruktur

   Lingkungan navigasi outdoor di mana tidak ada fitur navigasi visual yang berguna dikenal sebagai lingkungan tak terstruktur, misalnya pada planet Mars. Proyek Pathfinder merupakan proyek yang sukses. Pathfinder, pada tahun 1997, berhasil mendarat di Mars menggunakan modul statis yang disebut *lander*, berisi *multi-spectral stereo vision system*. *The rover* menggunakan chip Intel 8085 untuk tujuan *path selection* dan *rover localization*.

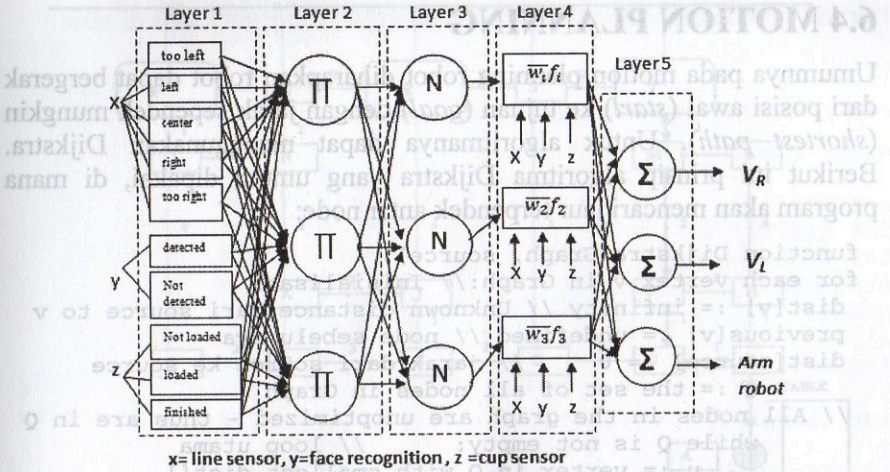
Gambar 6.1 *Cycab, Navigasi Outdoor Menggunakan Kamera*

Sebagai contoh, diagram blok di bawah ini menampilkan navigasi menggunakan vision pada robot Srikandi 1. Robot ini akan berjalan mengikuti garis.



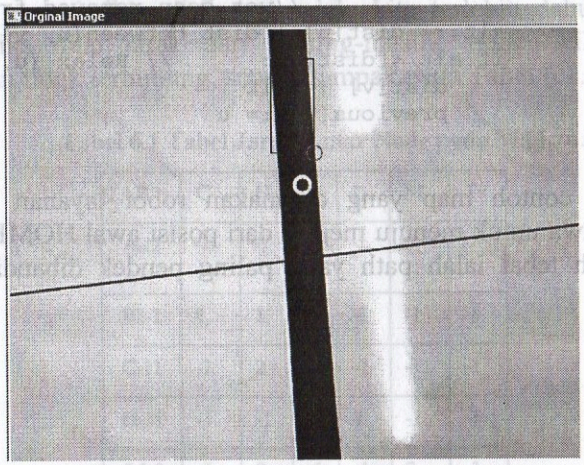
Gambar 6.2 *Model Navigasi Robot Berbasis Vision*

Sebagai kontroler navigasi digunakan ANFIS controller dengan arsitektur sebagai berikut:



Gambar 6.3 *Arsitektur ANFIS untuk Navigasi Robot [6]*

Berikut ini contoh hasil deteksi lintasan menggunakan vision, sebagai input panduan buat kontroler untuk navigasi robot:



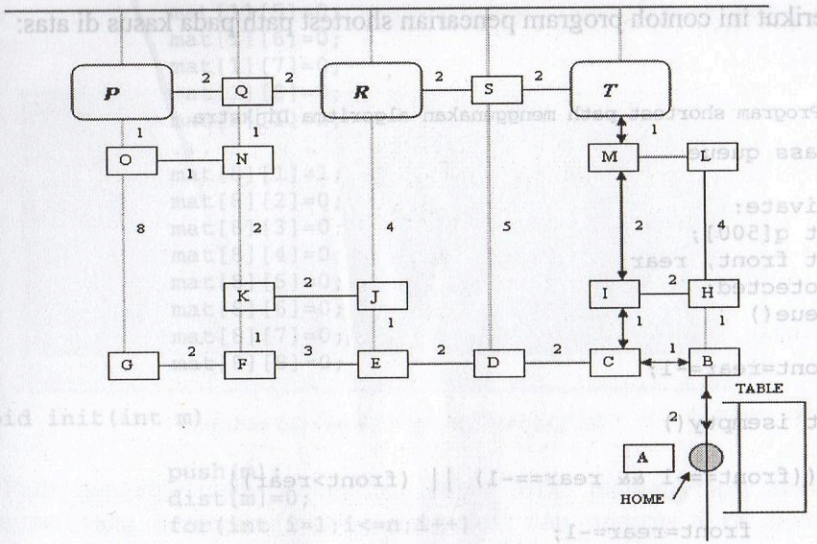
Gambar 6.4 *Lintasan yang Dideteksi Oleh Kamera*

6.4 MOTION PLANNING

Umumnya pada motion planning robot diharapkan robot dapat bergerak dari posisi awal (*start*) ke tujuan (*goal*) dengan jarak sependek mungkin (*shortest path*). Untuk algoritmanya dapat menggunakan Dijkstra. Berikut ini prinsip algoritma Dijkstra yang umum dipakai, di mana program akan mencari jaur terpendek antar node:

```
function Dijkstra(Graph, source):
for each vertex v in Graph:// Inisialisasi
 dist[v] := infinity // Unknown distance dari source to v
 previous[v] := undefined // node sebelumnya
 dist[source] := 0      // jarak dari source ke source
     Q := the set of all nodes in Graph
// All nodes in the graph are unoptimized - thus are in Q
      while Q is not empty:       // loop utama
          u := vertex in Q with smallest dist[]
          if dist[u] = infinity:
              break   // all remaining vertices are
                      // inaccessible from source
          remove u from Q
          for each neighbor v of u:        // where v has not
                                  //yet been removed from Q.
              alt := dist[u] + dist_between(u, v)
              if alt < dist[v]:       // Relax (u,v,a)
                  dist[v] := alt
                  previous[v] := u
return dist[]
```

Berikut ini contoh map yang digunakan robot layanan Srikandi 1. Terlihat bahwa untuk menuju meja T dari posisi awal HOME, path yang digaris lebih tebal ialah path yang paling pendek dibandingkan path lainnya.



Gambar 6.5 *Map untuk Robot Layanan*

Untuk mewujudkan path terpendek, kita terlebih dahulu membuat tabel matriks nilai jarak dari masing-masing node. Nilai -1 dapat menandakan tidak terhubung, seperti tampak pada Tabel 6.1.

Tabel 6.1 Tabel Jarak Antar Node pada Map

TAB	C	H	I	L	M	T
A-12	-1	-1	-1	-1		-1
B2-1	1	1	-1	-1	-1	-1
C-11	-1	2	1	-1	-1	-1
H-11	-1	-1	2	4	-1	-1
I-1-1	-1	2	-1	-1	2	-1
L-1-1	-1	3	-1	-1	1	-1
M-1-1	-1	-1	2	-1	-1	1
T1-1	-1	-1	-1	-1	-1	-1

Berikut ini contoh program pencarian shortest path pada kasus di atas:

```
//Program shortest path menggunakan algoritma Dijkstra
class queue
{
private:
int q[500];
int front, rear;
protected:
queue()
{
front=rear=-1;
}
int isempty()
{
if((front==-1 && rear==-1) || (front>rear))
{
       front=rear=-1;
       return 1;
       }
       return 0;
       }
void push(int a)
{
if(isempty())
front++;
q[++rear]=a;
};
int del()
{
return q[front++];
}
};
class dj :public queue
{
       private:
       int mat[10][10], dist[10], path[10];
       public:
       int n;
       void input()
{
n=8;
              mat[1][1]=0;
              mat[1][2]=2;
              mat[1][3]=0;
              mat[1][4]=0;
```

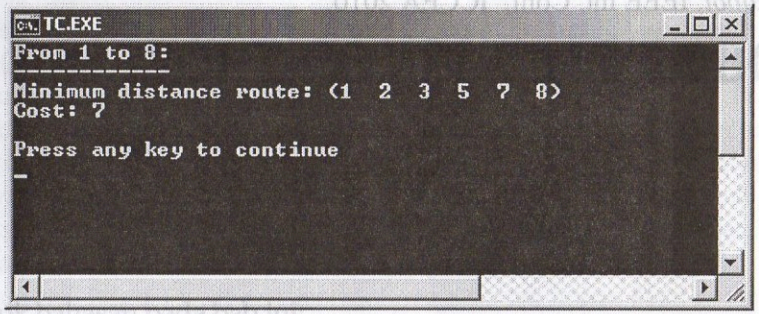


```
              mat[1][5]=0;
              mat[1][6]=0;
              mat[1][7]=0;
              mat[1][8]=0;
              mat[2][1]=2;
              ...
              mat[8][1]=1;
              mat[8][2]=0;
              mat[8][3]=0;
              mat[8][4]=0;
              mat[8][5]=0;
              mat[8][6]=0;
              mat[8][7]=0;
              mat[8][8]=0;
}
void init(int m)
{
              push(m);
              dist[m]=0;
              for(int i=1;i<=n;i++)
              {
                     if(i!=m)
                     {
                            push(i);
                            dist[i]=infi;
                     }
                     path[i]=0;
              }
       }
void min_dist(int m)
       {
              int v, w;
              init(m);
              while(!(isempty()))
              {
                     v=del();
                     for(int i=1;i<=n;i++)
                     {
                            if(mat[v][i]!=0)
                            {
                                   w=i;
              if((dist[v]+mat[v][w])<(dist[w]))
              {
              dist[w]=dist[v]+mat[v][w];
               path[w]=v;
                            }
                            }
                     }
```

```
        }
    }
void disp_path(int m)
    {
        int p=path[m];
        if(p==0)
        return;
        disp_path(p);
      printf ("  %d",m);
    }
void disp_dist(int m)
{
printf("Cost: %d",dist[m]);
}
};
void main()
{

    int c=0;
    dj a;
    a.input();
    cout<<"\n\nPress any key to continue"<<endl;
    getch();
    clrscr();
    for(int i=1;i<=a.n;i++)
    {
        a.min_dist(i);
        for(int j=1;j<=a.n;j++)
        {
            if(i!=j)
            {
                if(++c==10)
                {
    cout<<"\n\nPress any key to continue"<<endl;
    getch();
    clrscr();c=0;
                }
                    if (i ==1 && j==8) {
        cout<<"From "<<i<<" to "<<j<<":"<<endl;
        cout<<"------------"<<endl;
        cout<<"Minim route: ("<<i;
        a.disp_path(j);
        cout<<")"<<endl;
        a.disp_dist(j);
}
                }
        }
    }
```



```
cout<<"\n\nPress any key to exit"<<endl;
getch();
}
```



Gambar 6.6 *Hasil Program Shortest Path*

Pada pengembangan selanjutnya, robot harus mampu mendeteksi halangan yang bergerak (*moving obstacles*) dan memprediksi kapan halangan tersebut akan dilewati. Untuk hal ini dapat digunakan probabilitas Bayesian.

Referensi:

[1] K. Fujimura and H. Slamet, *Motion planning in a dynamic domain.* In Proc. Of the IEEE Int. Conf on Robotics and Automation, pages 324-330, Cincinatti, OH(USA), May 1990.

[2] C. Dorf, Richard (2000), *The Electrical Engineering Handbook, CRC Press LLC.*

[3] D'souza, A. Vijayakuma and Schall, S (2001). *Learning Inverse Kinematics*. Proc IEEE/RSJ International Conference Intelligent Robots and System, 298-302.

[4] Stefan Florczyk (2005), *Robot Vision, Video-Bsed Indoor Exploration with Autonomous and Mobile Robots*. Wiley VCH Verlag GmbH, Weinheim.

[5] J.P. van den Berg, and M. H. Overmars, *Roadmap-based motion planning in dynamic environments*, IEEE Transactions on Robotics, vol. 21, pp.885-897,2005.

[6] Widodo Budiharto, Achmad Jazidie and Djoko Purwanto, *Indoor Navigation using Adaptive Neuro Fuzzy Controller for Servant Robot*, IEEE Int. Conf. ICCEA 2010.

BAB VII
SOFT COMPUTING ROBOT

Pokok bahasan pada bab ini:

1. Soft Computing
2. Implementasi Neural Network pada Robot
3. ANFIS dan Implementasinya

7.1 PENDAHULUAN

7.1.1 Soft Computing

Soft computing memainkan peran penting dalam dunia robotika saat ini. Prof. Lotfi A. Zadeh adalah seorang pakar kecerdasan buatan terkemuka yang mengusulkan istilah ini. Beliau mendefinisikan soft computing sebagai penggabungan fuzzy logic, neuro computing dan evolutionary computation serta probabilistic computing ke dalam suatu bidang multidisiplin [1]. Tujuan utama soft computing ialah terbentuknya *high machine intelligence quotient*, yaitu suatu sistem yang mampu mengolah informasi secara cerdas. SC memiliki toleransi terhadap ketidakakuratan, ketidakpastian, dan kebenaran parsial untuk mencapai *robustness*.

Pada dasarnya teknik soft computing terdiri dari fuzzy logic, neuro computing atau artificial neural network (ANN), evolutionary computation dan probabilistic computing. Pada bab ini pembahasan soft computing difokuskan pada neural network dan ANFIS karena keduanya cukup banyak digunakan pada navigasi robot cerdas [5].

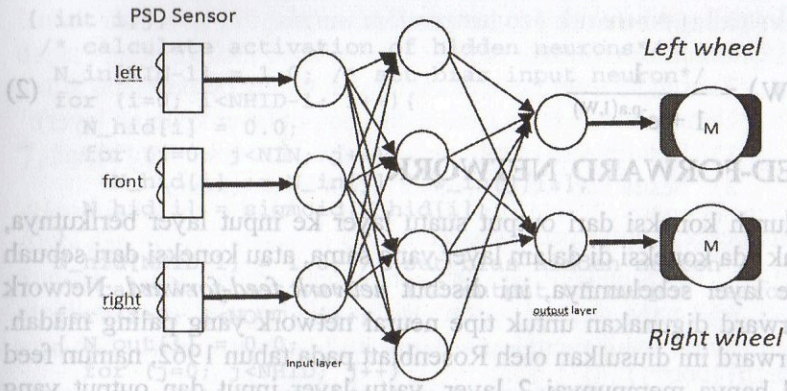
7.1.2 Artificial Neural Network

Neural network ialah suatu bidang kecerdasan buatan yang sedang berkembang pesat saat ini. Dengan neural network Anda dapat membuat sistem Anda menjadi cerdas, di mana sistem dapat belajar (*learning*) dari kondisi yang dialami untuk memutuskan aksi yang akan dilakukan. Neural network atau lengkapnya Artificial Neural Network (ANN) dibuat dari sebuah unit individu yang disebut *neuron* yang terhubung dengan yang lain melalui berbagai penghubung [Braun, 2003]. Neural network ialah paradigma yang berbeda untuk komputasi, di mana konsep mesin Von Neumann berdasarkan pada abstrasi pemrosesan (memori), sedangkan pada neural network berdasarkan arsitektur paralel dari *animal brain*. Setiap neuron mempunyai sebuah input, sebuah node pemroses dan sebuah output, di mana setiap koneksi dari 1 neuron ke neuron lain diasosiasikan dengan sebuah bobot atau *weight*.

7.2 NEURAL NETWORK PADA ROBOTIKA

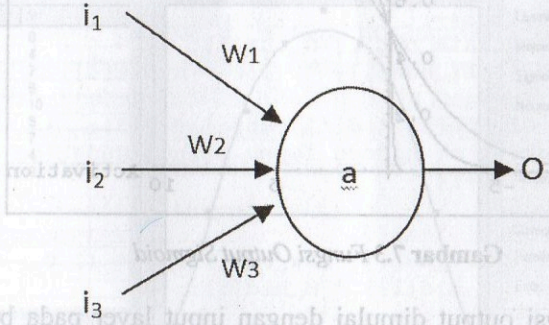
Jaringan neural dibuat dari sebuah neuron yang saling terhubung, yang tersusun pada sebuah layer. Output dari 1 layer neuron terhubung ke input dari layer berikutnya. Layer pertama dari neuron disebut sebagai input layer, misalnya sensor. Layer berikutnya disebut *hidden layer*, dan layer terakhir disebut layer output, misalnya aktuator robot. Seluruh layer neuron di antara input dan output layer disebut hidden layer. Untuk aplikasi praktis, sebuah hidden layer sudah memadai.

Contoh penerapan neural network pada robot mobil, seperti gambar di bawah ini, merupakan setup untuk NN yang mengemudikan robot mobil yang bebas tubrukan melalui sebuah penghalang (*maze*). Karena kita menggunakan 3 sensor dan 2 motor output, serta membuat 4 hidden neuron maka akan dihasilkan 3 + 4 + 2 neuron, sehingga total menjadi 9 neuron.



Gambar 7.1 *Setup NN pada Robot Mobil*

Pemrosesan pada sebuah neural network dilakukan secara paralel untuk seluruh neuron. Tiap neuron secara terus-menerus mengevaluasi atau membaca input, menghitung nilai aktivasinya, dan memproduksi atau menulis nilai output. Fungsi aktivasi dari sebuah neuron a(I,W) ialah jumlah seluruh input, yaitu tiap input dikalikan dengan *associated weight*. Output neuron ditentukan oleh fungsi output o(I,W).



Gambar 7.2 *Representasi weight, input dan output*

Dengan fungsi aktivasi:

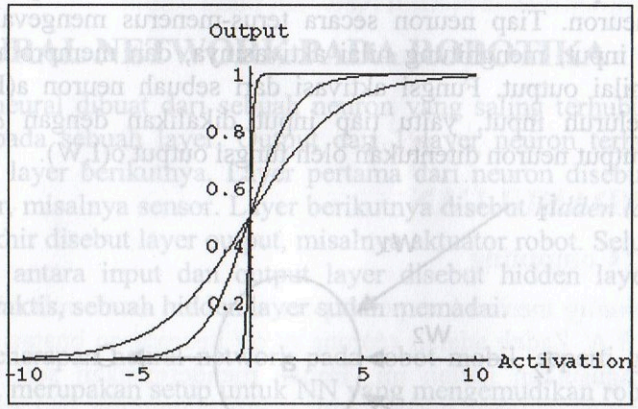
$$a(I,W) = \sum_{k=1}^{n} ik.wk \qquad (1)$$

output:

$$0(I,W) = \frac{1}{1 + e^{-p.a(I,W)}} \quad (2)$$

7.3 FEED-FORWARD NETWORK

Jika seluruh koneksi dari output suatu layer ke input layer berikutnya, dan tidak ada koneksi di dalam layer yang sama, atau koneksi dari sebuah layer ke layer sebelumnya, ini disebut *network feed-forward*. Network feed forward digunakan untuk tipe neural network yang paling mudah. Feed forward ini diusulkan oleh Rosenblatt pada tahun 1962, namun feed forward hanya mempunyai 2 layer, yaitu layer input dan output yang disebut *perceptron*. Kita menggunakan fungsi *sigmoid* untuk karakteristik pembelajaran pada sistem cerdas kita.



Gambar 7.3 *Fungsi Output Sigmoid*

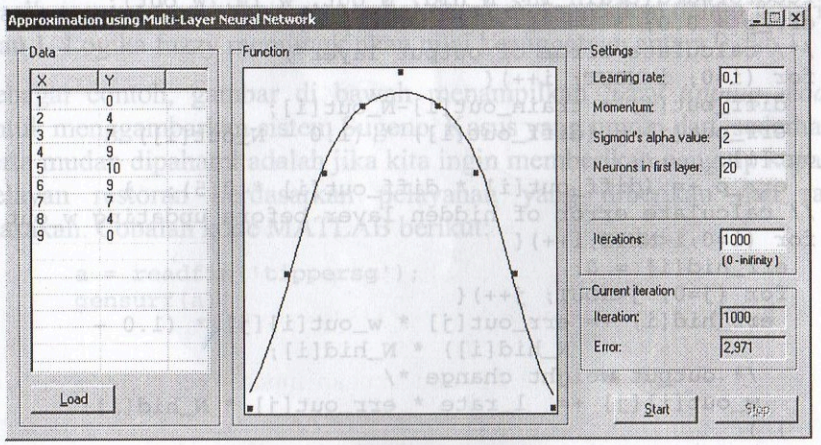
Perhitungan fungsi output dimulai dengan input layer pada bagian kiri network. Untuk layer input ada 1 nilai input per input neuron. Berikut contoh eksekusi feed forward pada neuron yang sederhana [6]:

```
double sigmoid(double x) {
      return 1.0 / (1.0 + (double)exp(-x));
}

void feedforward(double N_in[NIN], double N_hid[NHID],
double N_out[NOUT], double w_in[NIN][NHID], double
w_out[NHID][NOUT])
```



```
{ int i,j;
 /* calculate activation of hidden neurons*/
  N_in[NIN-1] = 1.0; /* set bias input neuron*/
  for (i=0; i<NHID-1; i++){
    N_hid[i] = 0.0;
    for (j=0; j<NIN; j++)
      N_hid[i] += N_in[j] * w_in[j][i];
    N_hid[i] = sigmoid(N_hid[i]);
  }
  N_hid[NHID-1] = 1.0; /* set bias hidden neuron*/
  /* calculate activation and output of output neurons*/
  for (i=0; i<NOUT; i++)
  { N_out[i] = 0.0;
    for (j=0; j<NHID; j++)
      N_out[i] += N_hid[j] * w_out[j][i];
    N_out[i] = sigmoid(N_out[i]);
  }
}
```

Gambar di bawah adalah contoh lain kemampuan NN untuk aproksimasi data:



Gambar 7.4 *Contoh Aproksimasi Nilai Menggunakan NN*

Berbagai konsep pembelajaran pada neural network telah diusulkan, di antaranya teknik *supervised* dan *unsupervised*. Penerapan yang paling terkenal ialah *backpropagation*. Algoritma backpropagation dapat diurutkan sebagai berikut:

1. Inisialisasi network dengan *random weight.*
2. Untuk seluruh *training cases*:
   a. Buat input training ke network dan hitung output
   b. Untuk seluruh layer (mulai dari output layer ke input layer):
      1). Bandingkan output network dengan ouput yang benar
      2). Sesuaikan weight (bobot) pada layer tersebut

Berikut ini penerapannya pada program:

```
double backprop(double train_in[NIN], double
train_out[NOUT],double l_rate,double w_in[NIN][NHID], double
w_out[NHID][NOUT])
/* returns error for current pattern*/
{ int i,j;
  double err_p; /*error for this pattern*/
  double N_out[NOUT],err_out[NOUT];  // generated network
                                     //output
  double diff_out[NOUT];  // diff output to desired
  double N_hid[NHID], err_hid[NHID];
  //re-calc network, difference and hidden output
  feedforward(train_in, N_hid, N_out, w_in, w_out);
  err_p = 0.0;
  /*/ calculate error of output layer*/
  for (i=0; i<NOUT; i++){
    diff_out[i] = train_out[i]-N_out[i];
    err_out[i] = (diff_out[i]) * (1.0 - N_out[i]) *
N_out[i];
    err_p += (diff_out[i] * diff_out[i] * 0.5);  }
  // calculate error of hidden layer before updating w_out
  for (i=0;i<NHID;i++){
    err_hid[i] = 0;
    for (j=0; j<NOUT; j++){
     err_hid[i] += err_out[j] * w_out[i][j] * (1.0 -
                   N_hid[i]) * N_hid[i];
      /* output weight change */
      w_out[i][j] +=  l_rate * err_out[j] * N_hid[i];
    } }
  /*/ update w_in*/
  for (i=0; i<NIN; i++)
    for (j=0; j<NHID; j++)
      w_in[i][j] += l_rate * err_hid[j] * train_in[i];
return err_p;}
```


Untuk mencoba progam neural network ini secara langsung pada robot dapat Anda lakukan pada robot EyeCon. Silahkan kunjungi situs penulis untuk demo kode dan penjelasan detail tentang robot ini.

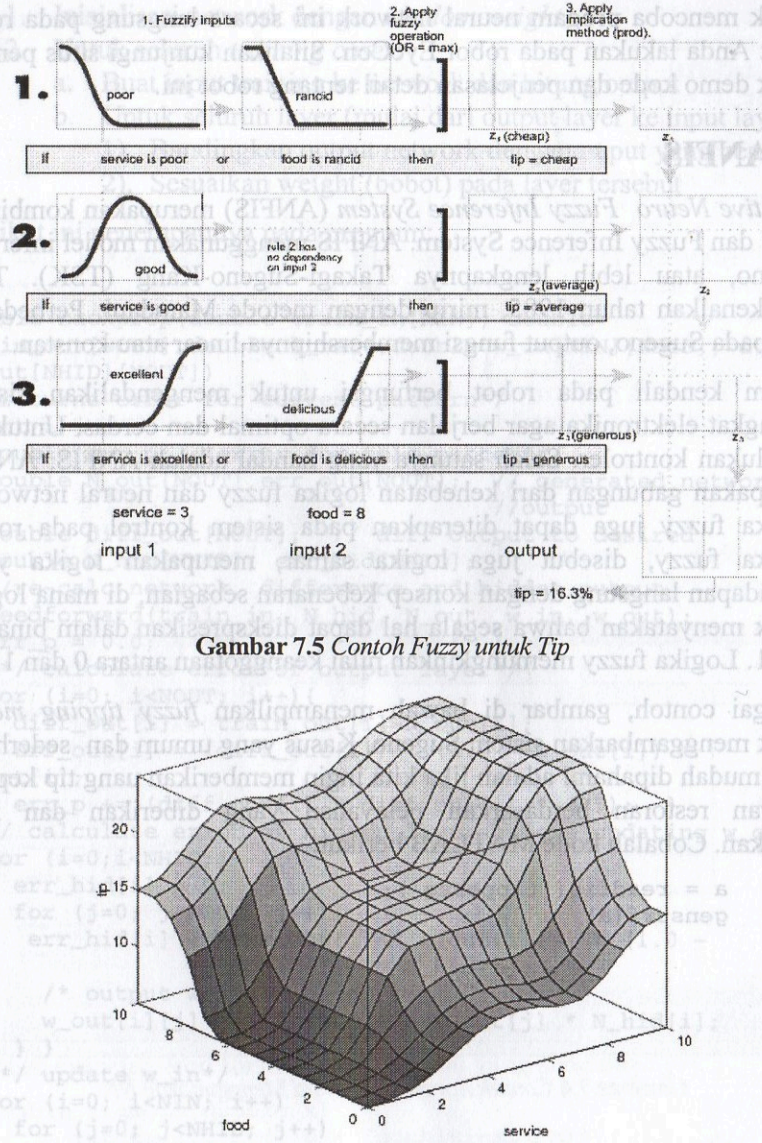
7.4 ANFIS

Adaptive Neuro Fuzzy Inference System (ANFIS) merupakan kombinasi ANN dan Fuzzy Inference System. ANFIS menggunakan model inferensi Sugeno, atau lebih lengkapnya Takagi-Sugeno-Kang (TSK). TSK diperkenalkan tahun 1985, mirip dengan metode Mamdani. Perbedaannya, pada Sugeno, output fungsi membershipnya linear atau konstan.

Sistem kendali pada robot berfungsi untuk mengendalikan sistem perangkat elektronika agar berjalan secara optimal dan cerdas. Untuk itu diperlukan kontroler. Salah satunya yang handal adalah ANFIS. ANFIS merupakan gabungan dari kehebatan logika fuzzy dan neural networks. Logika fuzzy juga dapat diterapkan pada sistem kontrol pada robot. Logika fuzzy, disebut juga logika samar, merupakan logika yang berhadapan langsung dengan konsep kebenaran sebagian, di mana logika klasik menyatakan bahwa segala hal dapat diekspresikan dalam binari 0 atau 1. Logika fuzzy memungkinkan nilai keanggotaan antara 0 dan 1.

Sebagai contoh, gambar di bawah menampilkan *fuzzy tipping model* untuk menggambarkan sistem Sugeno. Kasus yang umum dan sederhana serta mudah dipahami adalah jika kita ingin memberikan uang tip kepada pelayan restoran berdasarkan pelayanan yang diberikan dan rasa masakan. Cobalah kode MATLAB berikut:

```
a = readfis('tippersg');
gensurf(a)
```



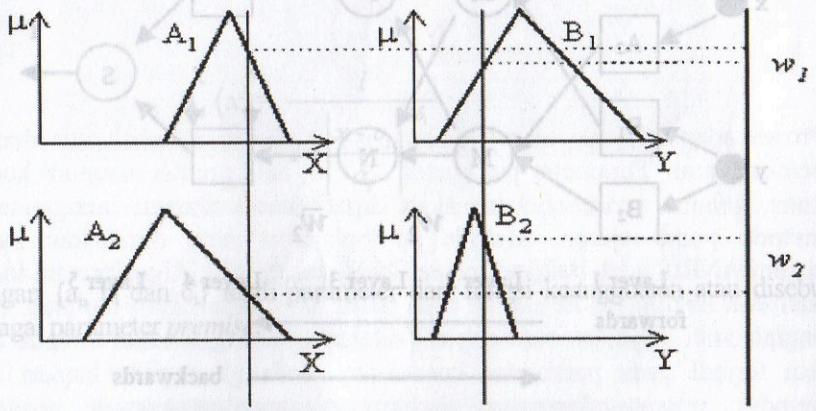
Gambar 7.5 *Contoh Fuzzy untuk Tip*

Gambar 7.6 *Hasil Pembangkitan Grafik untuk Kasus Tipper*

Dari Gambar 7.6 terlihat bahwa pemberian tip sangat mulus dan tidak kaku. ANFIS menggabungkan mekanisme *fuzzy inference system* yang digambarkan dalam arsitektur neural networks. Metode ini jauh lebih cepat dibandingkan *backpropagation* pada neural networks. Dengan hanya beberapa puluh *epoch* saja, proses pembelajaran dapat selesai. Basis aturan yang digunakan ialah:

Basis aturan:

1. If x is A1 and Y is B1 then z1 = p1*x + q1*y + r1
2. If x is A2 and Y is B2 then z2=p2*x + q2*y + r2



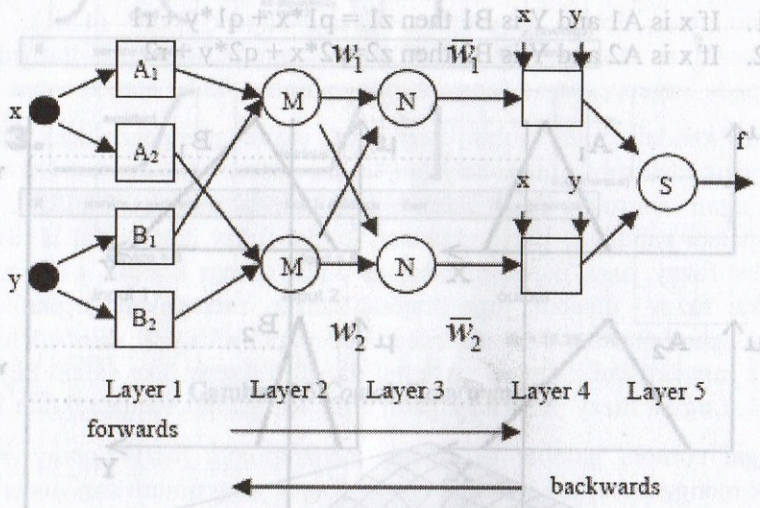
$$f = \frac{w_1 f_1 + w_2 f_2}{w_1 + w_2} = \overline{w}_1 f_1 + \overline{w}_2 f_2 \quad (3)$$

Gambar 7.7 *Output ANFIS*

Biasanya fungsi keanggotaan fuzzy input (premis) yang digunakan ialah fungsi generalized-Bell yang dirumuskan sebagai:

$$gbell(x,a,b,c,) = \frac{1}{1+\left|\frac{x-c}{a}\right|^{2b}} \quad (4)$$

Salah satu bentuk struktur yang sudah sangat dikenal dari sistem ANFIS terlihat pada Gambar 7.8. Dalam struktur ini, sistem inferensi fuzzy yang diterapkan adalah inferensi fuzzy model Takagi-Sugeno-Kang.



Gambar 7.8 *Struktur ANFIS [2]*

Seperti terlihat pada Gambar 7.8, sistem ANFIS terdiri dari 5 lapisan. Lapisan yang disimbolkan dengan kotak adalah lapisan yang bersifat adaptif, sedangkan yang disimbolkan dengan lingkaran bersifat tetap. Setiap keluaran dari masing-masing lapisan disimbolkan dengan Ol,i dengan i adalah urutan simpul dan l menunjukkan urutan lapisannya. Berikut ini adalah penjelasan untuk setiap lapisan:

Lapisan 1:

Berfungsi untuk membangkitkan derajat keanggotaan:

$$O_{1i} = \mu_{A_i}(x) \text{ i} = 1,2$$

$$O_{1i} = \mu_{B_i}(y) \text{ i} = 1,2$$

Dengan x dan y adalah masukan bagi simpul ke-i:

$$\mu_A(x) = \frac{1}{1+\left|\frac{x-c_i}{a_i}\right|^{2\delta_i}}$$

dengan $\{a_i, b_i$ dan $c_i\}$ ialah parameter dari fungsi keanggotaan atau disebut sebagai parameter *premise*.

Lapisan 2:

Berfungsi untuk membangkitkan *firing-strength* dengan mengalikan setiap sinyal masukan.

$$O_{2,1} = \omega_i = \mu_{A_i}(x) \; x \; \mu_{B_i}(y) \; 1 = 1,2$$

Lapisan 3:

Menormalkan *firing strength:*

$$O_{3,i} = \overline{w_i} = \frac{w_i}{w_1 + w_2} \; 1 = 1,2$$

Lapisan 4:

Menghitung keluaran kaidah berdasarkan parameter *consequent* $\{p_i, q_i$ dan $r_i\}$:

$$O_{4i} = \overline{w_i}\, f_i = \overline{w_i}\, (p_i x + q_i y + r_i)$$

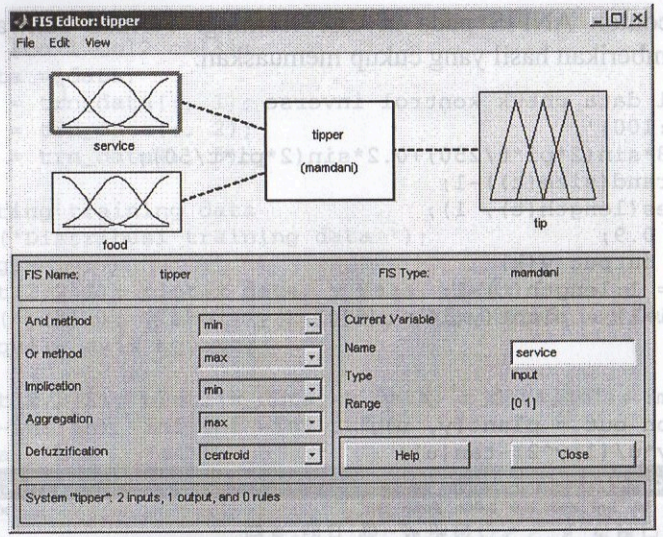
Lapisan 5:

Menghitung sinyal keluaran ANFIS dengan menjumlahkan semua sinyal yang masuk:

$$O_{s.i} = \sum_i \overline{w_i}\, f_i = \frac{\sum_i w_i f}{\sum_i w_i}$$

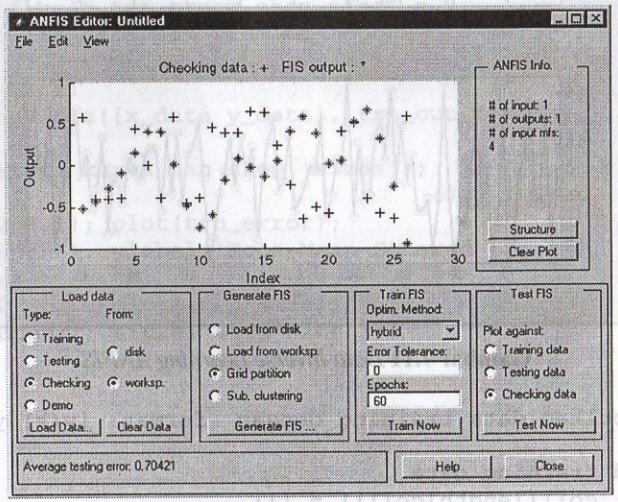
Proses adaptasi yang terjadi dalam sistem ANFIS dikenal juga dengan pembelajaran. Parameter-parameter ANFIS, baik premis maupun konsekuen, selama proses belajar akan diperbaharui dengan menggunakan metode pembelajaran. Metode pembelajaran yang digunakan dalam sistem ANFIS adalah algoritma pembelajaran hibrid. Algoritma ini terdiri dari dua bagian, yaitu bagian arah maju dan bagian arah mundur. Pada bagian arah maju, proses adaptasi dilakukan menggunakan metode LSE dan terjadi pada parameter konsekuen. Sedangkan pada bagian arah mundur, proses adaptasi dilakukan menggunakan metode *gradient-descent* dan terjadi pada parameter premise.

Untuk memelajari fuzzy dan ANFIS, paling mudah dilakukan melalui MATLAB. Ketiklah fuzzy pada command prompt MATLAB maka akan tampil FIS editor. Setelah itu bukalah file **tipper.fis** pada folder MATLAB sebagai demo fuzzy inference system untuk pemberian tip pada pelayanan rumah makan:



Gambar 7.9 *FIS Editor dengan Tipper.fis*

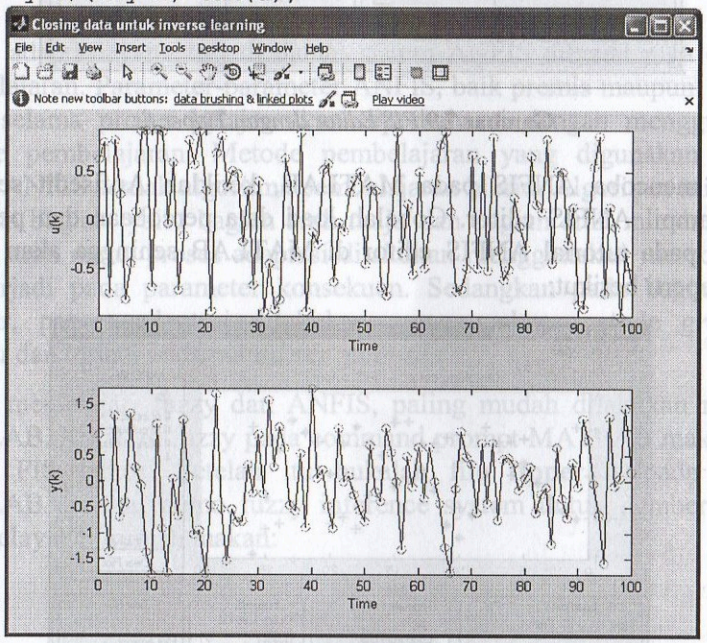
Untuk mencoba ANFIS pada MATLAB, ketiklah Anfisedit sehingga akan tampil ANFIS editor. Cobalah load data percobaan dan pelatihan seperti pada tutorial ANFIS editor di MATLAB sehingga akan tampil hasil seperti berikut:



Gambar 7.10 *Hasil Checking ANFIS Editor*

Berikut contoh ANFIS pada inverse learning. Terlihat bahwa sistem dapat memberikan hasil yang cukup memuaskan:

```
% Ambil data untuk kontrol inverse
t = (0:100)';
u = 0.8*sin(2*pi*t/250)+0.2*sin(2*pi*t/50);
u = 2*rand(size(t))-1;
y = ones(length(t), 1);
y(1) = 0.9;
% Cari output y(k)
for i = 1:length(t)-1;
    y(i+1) = plant(y(i), u(i));
end

plant.m:
function out = plant(y, u);
out = y*u/(1+y^2)-tan(u);
```

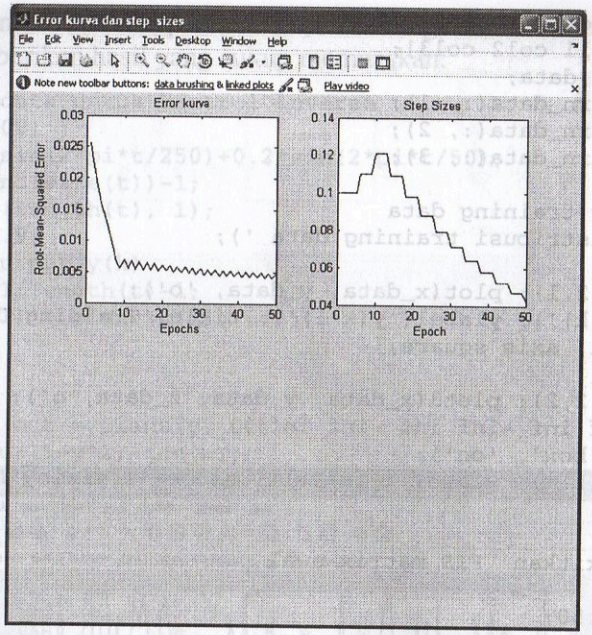


Gambar 7.11 *Data Inverse Learning ANFIS*

```
% Mengumpulkan data training dengan  format: [y(k) y(k+1);
u(k)]
% forward training data
col1 = y; col1(length(col1)) = [];
col2 = y; col2(1) = [];
```

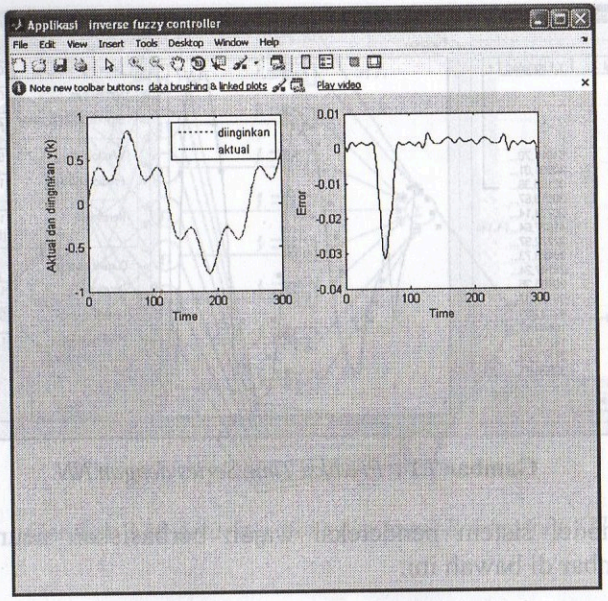


```
col3 = u; col3(length(col3)) = [];
data = [col1 col2 col3];
trn_data = data;
x_data = trn_data(:, 1);
y_data = trn_data(:, 2);
z_data = trn_data(:, 3);

% plotting training data
genfig('Distribusi training data ');
blackbg;
subplot(2,2,1); plot(x_data, y_data, 'o');
xlabel('y(k)'); ylabel('y(k+1)'); title('Training Data');
axis equal; axis square;

subplot(2,2,2); plot3(x_data, y_data, z_data,'o');
axis([-inf inf -inf inf -inf inf]);
set(gca, 'box', 'on');
xlabel('y(k)'); ylabel('y(k+1)'); title('Training Data');
drawnow

% membangkitkan  FIS matrix awal
mf_n = 3;
epoch_n = 50;
ss = 0.1;
mf_type = 'gbellmf';
in_fismat = genfis1(trn_data, mf_n, mf_type);

% mulai  training
[trn_out_fismat trn_error step_size] = ...
    anfis(trn_data, in_fismat, [epoch_n 0 ss], [1,1,1,1]);

% hitung hasil
z_hat = evalfis([x_data y_data], trn_out_fismat);

genfig('Error kurva dan step  sizes');
blackbg;
subplot(2,2,1); plot(trn_error);
xlabel('Epoh'); ylabel('Root-Mean-Squared Error');
title('Error kurva');
subplot(2,2,2); plot(step_size);
xlabel('Epoh'); title('Step Size');
drawnow
```



Gambar 7.12 *Kurva Kesalahan dan Step Sizes*

```
% verifikasi kontroler yang diperoleh
t = (0:300)';
u = 2*rand(size(t))-1;
desired_y = 0.6*sin(2*pi*t/250)+0.2*sin(2*pi*t/50);
y = ones(length(t), 1);
y(1) = 0.0; % this is not important

for i = 1:length(t)-1;
      u(i) = evalfis([y(i) desired_y(i+1)],trn_out_fismat);
   y(i+1) = plant(y(i), u(i));
end

genfig('Applikasi   inverse fuzzy controller');
blackbg;
subplot(2,2,1); plot(t, desired_y, ':', t, y, '-');
xlabel('Time'); ylabel('Aktual dan diinginkan y(k)');
legend('diinginkan ', 'aktual');
subplot(2,2,2); plot(t, desired_y-y);
xlabel('Time'); ylabel('Error');
```

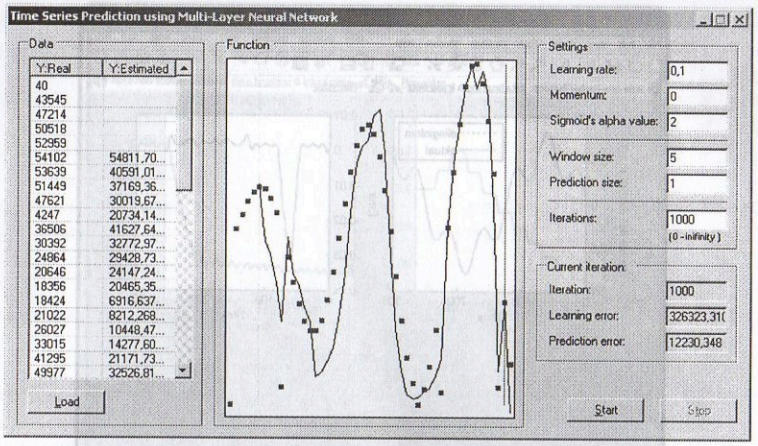



Gambar 7.13 *Hasil Kontroler ANFIS[2]*

Untuk mengimplementasikan FIS dari MATLAB ke bahasa C, Anda dapat mengompilasi program **fismain.c** dan **fis.c**. Silakan lihat petunjuk lengkapnya pada bagian tutorial MATLAB. Untuk mencoba ANFIS dalam bahasa C, coba kode dari Roger Jang (pencipta ANFIS) dengan file **anfis.tar.gz** dan NFIDENT dari ilmuwan lainnya dengan nama file **nfident.tar.gz** yang dapat diunduh di situs penulis. Anda memerlukan kesungguhan dan kesabaran di dalam mengimplementasikan sistem ini.

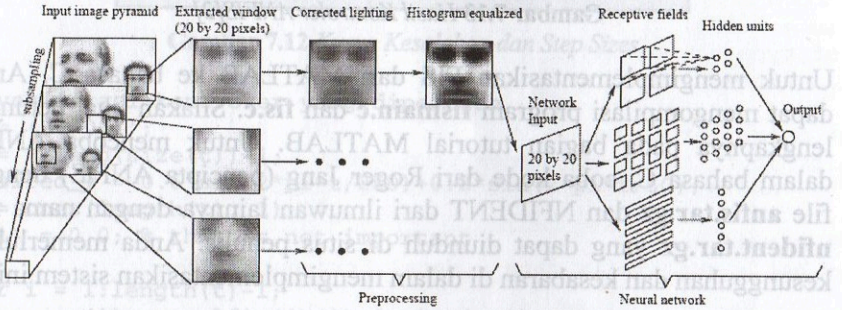
Latihan:

Buatlah program neural network untuk prediksi Time Series seperti contoh berikut [7]:



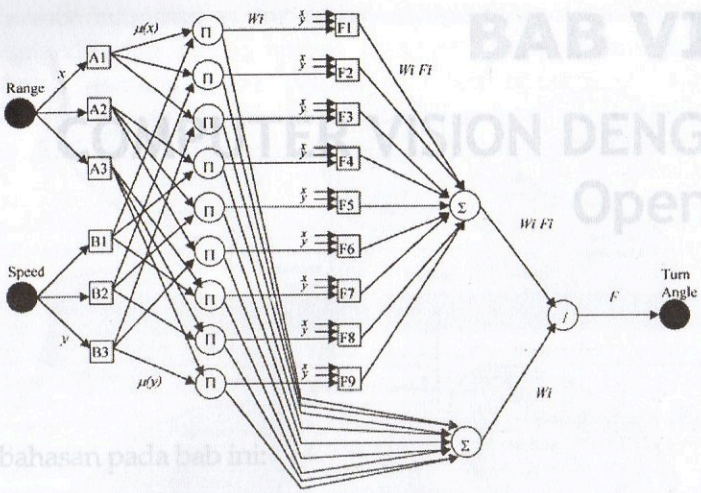
Gambar 7.14 *Prediksi Time Series dengan NN*

Buatlah model sistem pendeteksi wajah berbasiskan neural network seperti gambar di bawah ini:



Gambar 7.15 *Deteksi Wajah dengan NN [4]*

Buatlah program kontroler robot ANFIS dengan input data menggunakan sensor jarak ultrasonik seperti pada contoh berikut:



Gambar 7.16 *Arsitektur Kontroler untuk Robot Mobil [3]*

Referensi:

[1]. Zadeh, Lotfi A. (1997), "What is Soft Computing", Springer Verlag Germany.

[2] J.S. Jang (1997), *Neuro Fuzzy And Soft Computing*, Prentice Hall Publisher, Upper Saddle River, NJ.

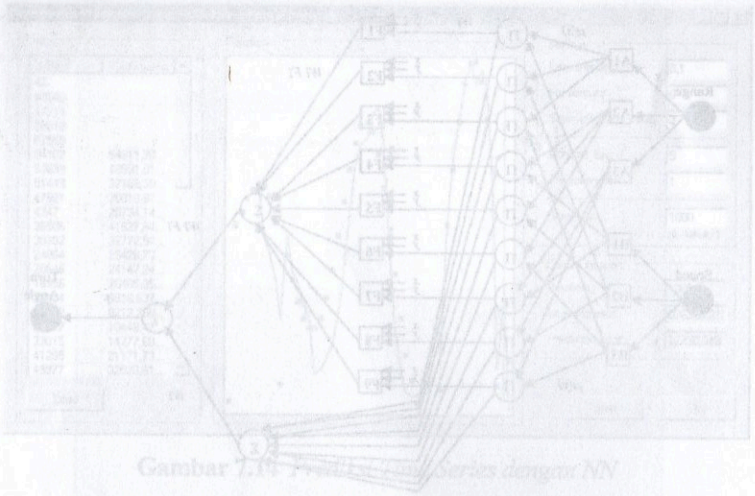
[3] Petru Rusu (2003), *Beaviour Based Neuro Fuzzy controller for Mobile Robot Navigation*, IEEE Transaction on Instrumentation and Measurement, pp. 1335-1340.

[4] Henry A Roley (1998), *Neural Network Based Face Detection*, PAMI.

[5] Cuesta F dan Olleor A (2005). *Intelligent Mobile Robot Navigation*, Springer Verlag, Berlin.

[6] http://robotics.ee.uwa.edu.au/eyebot

[7] http://www.codeproject.com



BAB VIII
COMPUTER VISION DENGAN OpenCV

Pokok bahasan pada bab ini:

1. Peranan Computer Vision
2. Mengenal OpenCV
3. Kamera dan Video
4. Pengolahan Citra dengan OpenCV

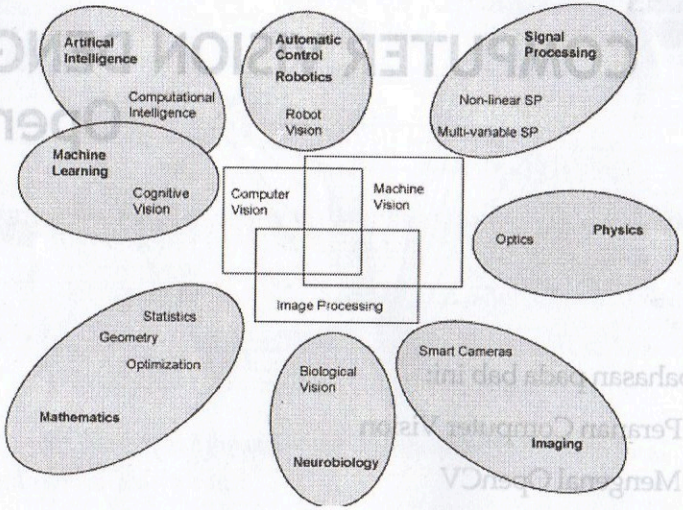
8.1 PENDAHULUAN

Computer vision merupakan teknologi paling penting dalam pengembangan robot yang interaktif di masa yang akan datang. Computer vision merupakan bidang pengetahuan yang berfokus pada bidang sistem kecerdasan buatan dan berhubungan dengan akuisisi dan pemrosesan images.

Contoh penerapan computer vision pada dunia riset dan industri ialah:

1. Interaksi manusia dan robot (*human robot interaction*),
2. Pengontrolan proses industri,
3. Pendeteksi nomor plat kendaraan,
4. Organisasi informasi indexing database,
5. Modeling obyek atau lingkungan.

Gambar di bawah menampilkan relasi antara computer vision dengan bidang riset lainnya:



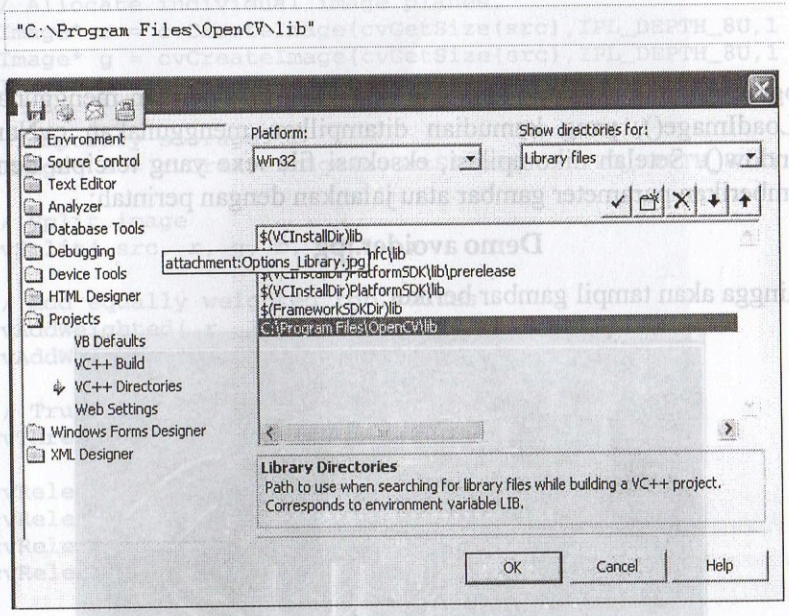
Gambar 8.1 *Hubungan Computer Vision dengan Bidang Lainnya*

Pada penerapannya, computer vision menggunakan *digital image procesing* untuk algoritma komputer dalam pemrosesan image pada image digital.

8.2 PROGRAM OpenCV

OpenCV ialah program open source berbasiskan C++ yang saat ini banyak digunakan sebagai program computer vision. Salah satu penerapannya ialah pada robotika. Dengan OpenCV maka Anda dapat membuat interaksi antara manusia dan robot (*human robot interaction*). Misalnya, wajah dari manusia dideteksi oleh kamera (webcam), yang kemudian diproses oleh komputer, untuk kemudian diproses oleh robot guna melakukan aksi tertentu, misalnya mengikuti/mengenal wajah orang tersebut. Kesemuanya itu membutuhkan OpenCV sebagai program utama antara webcam dan pengolahnya, yaitu komputer. Silakan kunjungi situs opencv.org untuk download atau mengetahui berita terbaru tentang software ini.

Anda membutuhkan editor dan kompiler Visual Studio.Net 2008 untuk mengedit dan mengompilasi program OpenCV. Anda terlebih dahulu harus mengonfigurasi Visual C++.Net tersebut di mana file library dan source-nya harus disertakan (lihat tutorialnya di internet). Beberapa file library juga harus ditambahkan pada input linker di Visual C++.



Gambar 8.2 *Konfigurasi File Library*

Sebagai contoh, buatlah program Win32 console application untuk menampilkan sebuah gambar di Windows. Berikut ini contohnya:

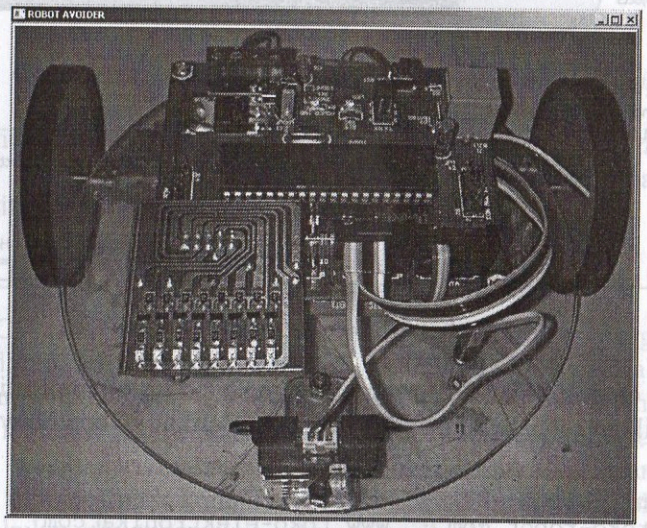
```
// Demo Program menampilkan gambar Robot
// By Mr.  Widodo 2010
// www.toko-robot.com     www.toko-elektronika.com

#include "stdafx.h"
#include "conio.h"
#include <cv.h>
#include <highgui.h>
int main(int argc, char** argv) //fungsi utama
{
//load image dari parameter yang diberikan
```

```
IplImage* img =cvLoadImage(argv[1]);
//judul window
cvNamedWindow ("ROBOT AVOIDER",CV_WINDOW_AUTOSIZE);
//tampilkan gambar
cvShowImage("ROBOT AVOIDER",img);
cvWaitKey(0);
cvReleaseImage(&img); //release resource
cvDestroyWindow("ROBOT AVOIDER");
}
```

Program di atas akan me-*load* file .jpg yang kita berikan menggunakan cvLoadImage(), yang kemudian ditampilkan menggunakan cvNamed Window(). Setelah dikompilasi, eksekusi file .exe yang tercipta dengan memberikan parameter gambar atau jalankan dengan perintah:

Demo avoider.jpg

sehingga akan tampil gambar berikut:

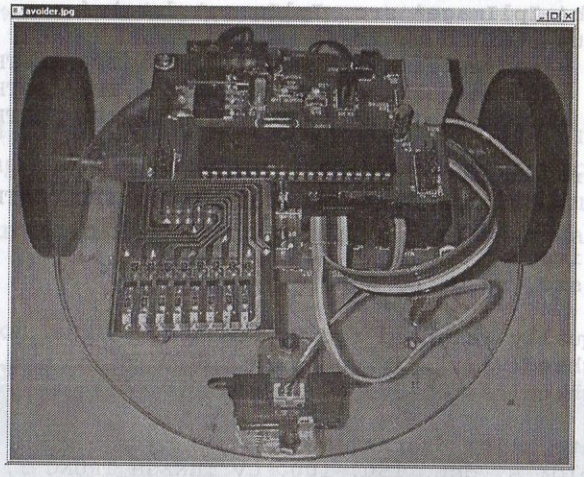


Gambar 8.3 *Tampilan Hasil Load Gambar*

Kadang citra yang kita miliki perlu kita olah untuk tujuan tertentu yang dikenal dengan istilah *pengolahan citra*, misalnya proses penjumlahan nilai RGB pada program di bawah ini:


```
//Program penjumlahan RGB
#include "stdafx.h"
#include <stdio.h>
#include <cv.h>
#include <highgui.h>

void sum_rgb( IplImage* src, IplImage* dst ) {
  // Allocate individual image planes.
IplImage* r = cvCreateImage(cvGetSize(src),IPL_DEPTH_8U,1 );
IplImage* g = cvCreateImage(cvGetSize(src),IPL_DEPTH_8U,1 );
IplImage* b = cvCreateImage(cvGetSize(src),IPL_DEPTH_8U,1 );

  // Temporary storage.
IplImage* s = cvCreateImage(cvGetSize(src),IPL_DEPTH_8U,1 );

  // Split image
  cvSplit( src, r, g, b, NULL );

  // Add equally weighted rgb values.
  cvAddWeighted( r, 1./3., g, 1./3., 0.0, s );
  cvAddWeighted( s, 2./3., b, 1./3., 0.0, s );

  // Truncate nilai diatas 100.
  cvThreshold( s, dst, 150, 100, CV_THRESH_TRUNC );

  cvReleaseImage( &r );
  cvReleaseImage( &g );
  cvReleaseImage( &b );
  cvReleaseImage( &s );
}

int main(int argc, char** argv)
{
  // Buat jendela
  cvNamedWindow( argv[1], 1 );
  // Load the image from the given file name.
  IplImage* src = cvLoadImage( argv[1] );
  IplImage* dst = cvCreateImage( cvGetSize(src), src->depth,
1);
  sum_rgb( src, dst);

  // Tampilkan jendela
  cvShowImage( argv[1], dst );
  // Idle until the user hits the "Esc" key.
  while( 1 ) { if( (cvWaitKey( 10 )&0x7f) == 27 ) break; }

  // Bersihkan jendela dan memori
```

```
    cvDestroyWindow( argv[1] );
    cvReleaseImage( &src );
    cvReleaseImage( &dst );
}
```



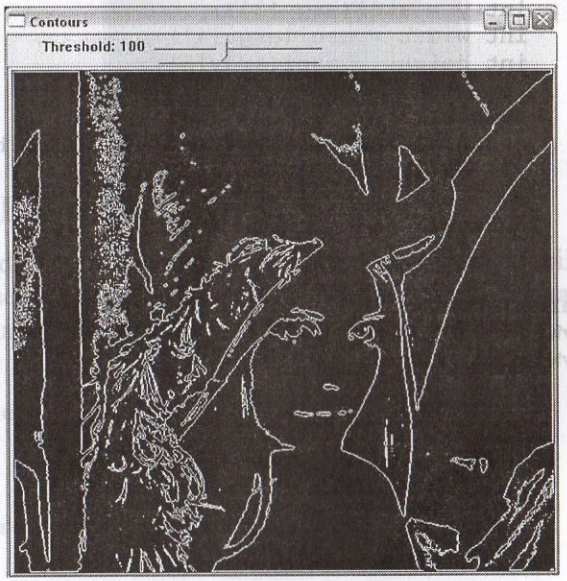
Gambar 8.4 *Hasil Penjumlahan RGB*

Pada pengolahan citra dasar, biasanya *edge detection*, *threshold* dan *contour* sangat dibutuhkan. Berikut ini contohnya:

```
//Program  membuat countour dan variasi threshold
#include <cv.h>
#include <highgui.h>
IplImage* image = 0;
int thresh = 100;
void on_trackbar(int)
{
IplImage* gray = cvCreateImage( cvGetSize(image), 8, 1 );
CvMemStorage* storage = cvCreateMemStorage(0);
CvSeq* contours = 0;
cvCvtColor( image, gray, CV_BGR2GRAY );
cvThreshold( gray, gray, thresh, 255, CV_THRESH_BINARY );
cvFindContours( gray, storage, &contours );
cvZero( gray );
if( contours )
cvDrawContours( gray, contours, cvScalarAll(255),
cvScalarAll(255), 100 );
```



```
cvShowImage( "Contours", gray );
cvReleaseImage( &gray );
cvReleaseMemStorage( &storage );
}
int main( int argc, char** argv )
{
IplImage* image;
if( argc != 2 || !(image = cvLoadImage(argv[1])) )
return -1;
cvNamedWindow( "Contours", 1 );
cvCreateTrackbar( "Threshold", "Contours",
&thresh, 255, on_trackbar );
on_trackbar(0);
cvWaitKey();
return 0;
}
```



Gambar 8.5 *Program Countour pada Lena.jpg*

Pada pengolahan citra, untuk mengambil sebagian daerah penting dari suatu image, biasa dikenal sebagai ROI *(region of interest)*, dapat dilakukan dengan menggunakan fungsi berikut:

```
cvSetImageROI(src, cvRect(x,y,width,height));
```

Berikut ini contoh program ROI:

ROI.cpp:

```
#include "stdafx.h"
#include <cv.h>
#include <highgui.h>

int main(int argc, char** argv)
{

    IplImage* src;
    cvNamedWindow("Contoh awal", CV_WINDOW_AUTOSIZE);
    cvNamedWindow("Contoh akhir", CV_WINDOW_AUTOSIZE);
    if( argc == 7 && ((src=cvLoadImage(argv[1],1)) != 0 ))
    {
            int x = atoi(argv[2]);
            int y = atoi(argv[3]);
            int width = atoi(argv[4]);
            int height = atoi(argv[5]);
            int add = atoi(argv[6]);
            cvShowImage( "Contoh awal", src);
            cvSetImageROI(src, cvRect(x,y,width,height));
            cvAddS(src, cvScalar(add),src);
            cvResetImageROI(src);
            cvShowImage( "Contoh akhir",src);
            cvWaitKey();
    }
  cvReleaseImage( &src );
  cvDestroyWindow("Contoh awal");
  cvDestroyWindow("Contoh akhir");
  return 0;
}
```

Gambar 8.6 *Area ROI pada image*

8.3 KAMERA DAN VIDEO

Jika Anda ingin mendeteksi webcam dan mengambil image dari webcam tersebut, gunakan demo kode berikut ini, di mana fungsi yang umum digunakan adalah:

```
pCapture = cvCaptureFromCAM( CV_CAP_ANY );
```

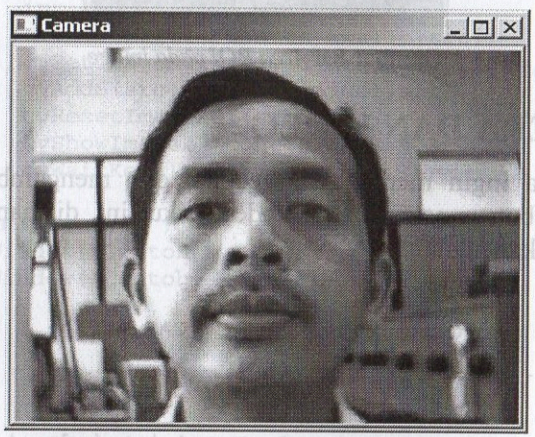
atau

```
CvCapture* cvCreateCameraCapture( int index );
```

Kamera.cpp:

```
//Menampilkan image menggunakan Webcam
#include "stdafx.h"
#include "cv.h"
#include "highgui.h"
int main( int argc, char** argv ) {
```

```
    cvNamedWindow( "Camera", CV_WINDOW_AUTOSIZE );
    CvCapture* capture;
    if (argc==1) {
        capture = cvCreateCameraCapture( 0 );
    } else {
        capture = cvCreateFileCapture( argv[1] );
    }
    assert( capture != NULL );

    IplImage* frame;
    while(1) {
        frame = cvQueryFrame( capture );
        if( !frame ) break;
        cvShowImage( "Camera", frame );
        char c = cvWaitKey(10);
        if( c == 27 ) break;
    }
    cvReleaseCapture( &capture );
    cvDestroyWindow( "Camera" );
}
```



Gambar 8.7 *Hasil Penggunaan Webcam*

Program berikut ini akan menggambar kotak dan mendeteksi ukuran kotak dengan menampilkan posisi x dan y kotak tersebut. Digunakan mouse callback untuk mengetahui apakah melakukan klik kanan atau kiri mouse dengan menggunakan rutin berikut:

```
void my_mouse_callback( int event, int x, int y, int flags,
void* param )
```



```
{
  IplImage* image = (IplImage*) param;
  switch( event ) {
    case CV_EVENT_MOUSEMOVE: {
      if( drawing_box ) {
        box.width  = x-box.x;
        box.height = y-box.y;
      }
    }
    break;
    case CV_EVENT_LBUTTONDOWN: {
      printf( " x: %i   y: %i ",  box.x, box.y);
```

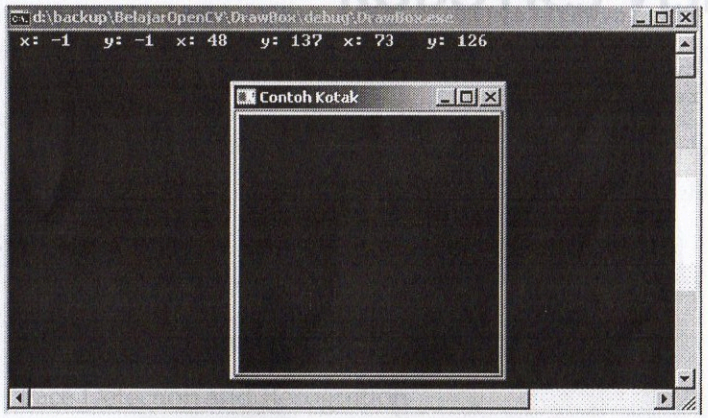
DrawoBox.cpp:

```
// Program menggambar kotak dan menampilkan posisinya.

#include "stdafx.h"
#include <cv.h>
#include <highgui.h>

void my_mouse_callback(
   int event, int x, int y, int flags, void* param
);

CvRect box;
bool drawing_box = false;

void draw_box( IplImage* img, CvRect rect ) {
  cvRectangle (
    img,
    cvPoint(box.x,box.y),
    cvPoint(box.x+box.width,box.y+box.height),
    cvScalar(0xff,0x00,0x00)     /* merah */
  );
}

int main( int argc, char* argv[] ) {

box = cvRect(-1,-1,0,0);
IplImage* image = cvCreateImage(
cvSize(200,200),IPL_DEPTH_8U,3);
cvZero( image );
IplImage* temp = cvCloneImage( image );
cvNamedWindow( "Contoh Kotak" );
cvSetMouseCallback(
    "Contoh Kotak",
```

```
    my_mouse_callback,
    (void*) image
  );

while( 1 ) {
    cvCopyImage( image, temp );
    if( drawing_box ) draw_box( temp, box );
    cvShowImage( "Contoh Kotak", temp );

    if( cvWaitKey( 15 )==27 ) break;
  }
  cvReleaseImage( &image );
  cvReleaseImage( &temp );
  cvDestroyWindow( "Contoh Kotak" );
}

// mouse callback.
void my_mouse_callback(
   int event, int x, int y, int flags, void* param )
{

  IplImage* image = (IplImage*) param;

  switch( event ) {
    case CV_EVENT_MOUSEMOVE: {
      if( drawing_box ) {
        box.width  = x-box.x;
        box.height = y-box.y;
      }
    }
    break;
    case CV_EVENT_LBUTTONDOWN: {
      printf( " x: %i   y: %i ",  box.x, box.y);
      drawing_box = true;
      box = cvRect( x, y, 0, 0 );
    }
    break;
    case CV_EVENT_LBUTTONUP: {
      drawing_box = false;
      if( box.width<0  ) {
        box.x+=box.width;
        box.width *=-1;
      }
      if( box.height<0 ) {
        box.y+=box.height;
        box.height*=-1;
      }
      draw_box( image, box );
```



```
    }
    break;
  }
}
```



Gambar 8.8 *Hasil Program Gambar Kotak*

LATIHAN:

1. *Buatlah program untuk menerapkan Kalmann Filter.*
2. *Buatlah program yang menampilkan teks pada layar monitor seperti contoh di bawah ini.*

```
#include <cv.h>
#include <highgui.h>
#include <string.h>
#include <stdio.h>

CvPoint pt1,pt2;
CvFont font;

int main( int argc, char** argv )
{
while(1)
{
IplImage* hist1 = cvCreateImage( cvSize(640,480), 8, 3 );
pt1.x=100;pt1.y=60;
```

```
cvInitFont( &font, CV_FONT_HERSHEY_COMPLEX, 0.5, 0.5, 0.0,
1, CV_AA );
cvPutText( hist1, "Mr. Widodo", pt1, &font , CV_RGB(0,0,255)
);
        cvNamedWindow( "Demo Text", 1 );
        cvShowImage( "Demo Text", hist1 );
        if( (cvWaitKey(10) & 255) == 27 ) break;
        }
        cvDestroyWindow("Demo Text");
        return 0;
}
```

BAB IX
ROBOTICS VISION

Pokok bahasan pada bab ini:

1. Konsep Robotics Vision
2. Penerapan cvHaar
3. Face Detection and Recognition

9.1 PENDAHULUAN

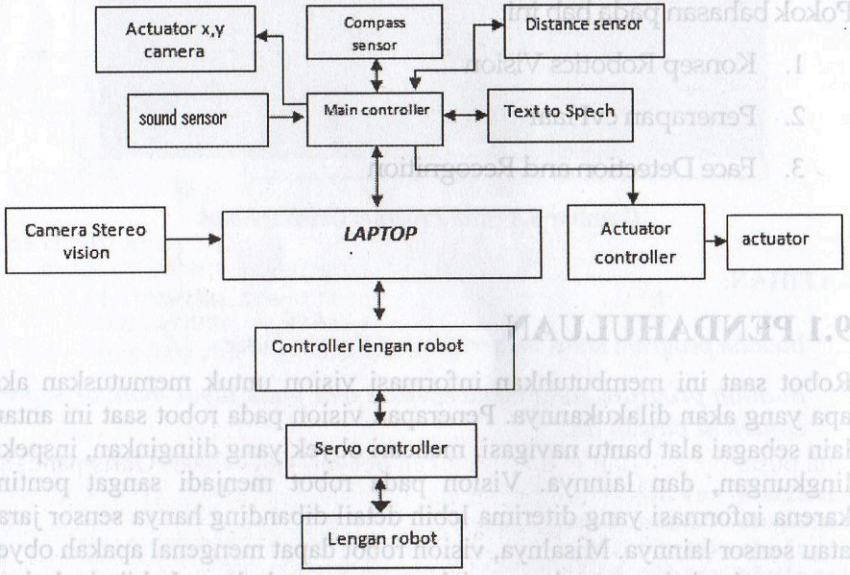
Robot saat ini membutuhkan informasi vision untuk memutuskan aksi apa yang akan dilakukannya. Penerapan vision pada robot saat ini antara lain sebagai alat bantu navigasi, mencari obyek yang diinginkan, inspeksi lingkungan, dan lainnya. Vision pada robot menjadi sangat penting karena informasi yang diterima lebih detail dibanding hanya sensor jarak atau sensor lainnya. Misalnya, vision robot dapat mengenal apakah obyek yang terdeteksi merupakan wajah orang atau bukan. Lebih jauh lagi, sistem vision yang canggih pada robot membuat robot dapat membedakan wajah A dengan wajah B secara akurat. Bab ini akan menjelaskan teknik yang umum digunakan untuk membantu robot melakukan deteksi dan pengenalan obyek dan wajah.

9.2 HUMAN ROBOT INTERACTION

Perkembangan pesat teknologi robot telah menuntut hadirnya robot cerdas yang mampu melengkapi dan membantu pekerjaan manusia. Salah satu harapan dari perkembangan teknologi vision pada robot ini ialah

robot dapat berinteraksi dengan manusia bahkan dapat menggantikan beberapa fungsi penting dalam hidup, misalnya robot pengganti (*surrogate*) yang dapat berbelanja ke pasar. Dengan robot yang mampu berbelanja tentunya kita dapat menurunkan persentase kecelakaan pada manusia.

Pada tahun 2020 diprediksi hampir semua pekerjaan rumah tangga sudah dapat dikerjakan oleh *service robot.* Adalah tantangan besar bagi kita di dalam mengembangkan service robot tersebut. Gambar di bawah menampilkan model robot umum yang dapat berinteraksi dengan manusia.

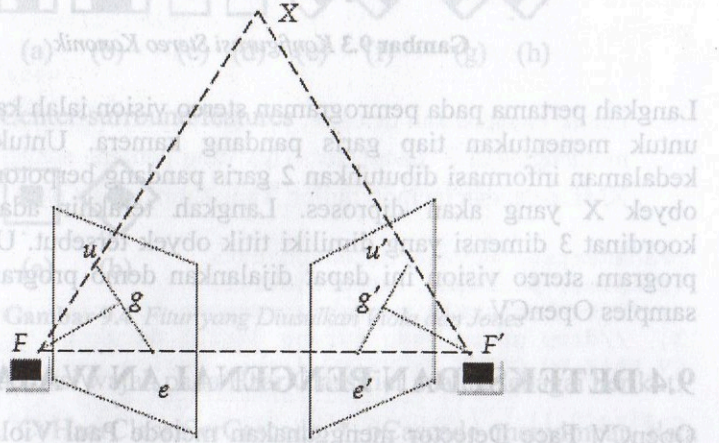


Gambar 9.1 *Arsitektur Robot yang Mampu Berinteraksi dengan Manusia*

Robot yang mampu berinteraksi dengan manusia *(humanoid robot)* idealnya harus mampu berperilaku seperti manusia, antara lain berbicara, bergerak, memiliki tangan atau manipulator dan mampu mengenali wajah. Untuk mendukung hal di atas dapat digunakan seperangkat PC/laptop dengan mikrokontroler serta modul elektronika pendukung. Namun demikian saat ini sudah banyak diterapkan FPGA berbasis Linux dan laptop serta modul pendukung berbiaya murah untuk mewujudkannya.

9.3 STEREO VISION

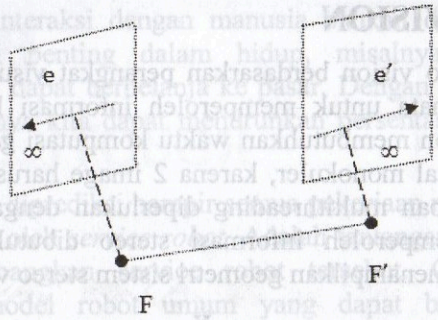
Geometri stereo vision berdasarkan perangkat visual manusia yang menggunakan 2 mata untuk memperoleh informasi kedalaman suatu obyek. Stereo vision membutuhkan waktu komputasi ganda dibandingkan perangkat visual monokuler, karena 2 image harus dianalisis. Oleh karena itu, penerapan multithreading diperlukan dengan menggunakan thread. Untuk memperoleh informasi stereo dibutuhkan 2 kamera. Gambar di bawah menampilkan geometri sistem stereo vision.



Gambar 9.2 *Geometri dari stereovision*

Dua pusat optik F dan F' diasosiasikan dengan garis dasar. Garis pandang miik F dan F' berpotongan pada titik X dan menghasilkan bidang triangular yang memotong setiap bidang image pada garis epipolar g dan g'. Proyeksi u dan u' dihasilkan dari titik obyek X. Seluruh posisi X yang mungkin terletak pada garis FX untuk image kiri dan F'X untuk image kanan lalu direfleksikan pada bidang image kamera kiri untuk F'X dengan garis g dan bidang image kamera kanan untuk garis F'X dengan garis g'. Epipole e dan e' juga terletak pada garis g dan g' yang berpotongan pada garis dasar.

Gambar di bawah menampilkan konfigurasi stereo vision yang disebut *canonical.* Garis dasar (baseline) sesuai dengan sumbu koordinat horizontal. Konfigurasi tersebut mengakibatkan 2 epipole e dan e' tak terbatas.



Gambar 9.3 *Konfigurasi Stereo Kanonik*

Langkah pertama pada pemrograman stereo vision ialah kalibrasi kamera untuk menentukan tiap garis pandang kamera. Untuk memperoleh kedalaman informasi dibutuhkan 2 garis pandang berpotongan pada titik obyek X yang akan diproses. Langkah terakhir adalah pengujian koordinat 3 dimensi yang dimiliki titik obyek tersebut. Untuk mencoba program stereo vision ini dapat dijalankan demo program pada folder samples OpenCV.

9.4 DETEKSI DAN PENGENALAN WAJAH

OpenCV Face Detector menggunakan metode Paul Viola dan Michael Jones, sedangkan OpenCV Face Tracker menggunakan algoritma CamShift. Metode face detector ini mengombinasikan:

1. Fitur rectangular sederhana yang disebut fitur Haar
2. Integral image untuk deteksi fitur yang cepat
3. Metode machine learning AdaBoost
4. Sebuah pengklasifikasi cascade untuk mengkombinasikan banyak fitur secara efisien.

Fitur yang digunakan Viola dan Jones menggunakan bentuk gelombang Haar. Bentuk gelombang Haar ialah sebuah gelombang kotak. Pada 2 dimensi, gelombang kotak ialah pasangan persegi yang bersebelahan, 1 terang dan 1 gelap. Haar ditentukan oleh pengurangan piksel rata-rata daerah gelap dari piksel rata-rata daerah terang. Jika perbedaan di atas threshold (diset selama learning), fitur tersebut dikatakan ada. Kata cascade pada classifier berarti total classifier terdiri dari classifier yang lebih simple yang diaplikasikan secara subsekuen ke ROI .

1. Edge features

(a) (b) (c) (d)

2. Line features

(a) (b) (c) (d) (e) (f) (g) (h)

3. Center-surround features

(a) (b)

Gambar 9.4 *Fitur yang Diusulkan Viola dan Jones*

Implementasi deteksi wajah pada Haar Classifier adalah sebagai berikut:

1. Variabel CvHaarClassifierCascade * pCascade menyimpan data dari file XML. Untuk me-load data XML ke pCascade, Anda dapat menggunakan fungsi cvLoad(). cvLoad ialah fungsi umum untuk me-load data dari file yang membutuhkan hingga 3 parameter input. Jika Anda membuat kode pada C, set parameter sisanya menjadi 0. Jika menggunakan C++, hilangkan parameter yang tidak digunakan.
2. Sebelum mendeteksi wajah pada images, Anda membutuhkan objek CvMemStorage. Detektor akan mendaftar wajah yang terdeteksi ke buffer. Yang harus Anda kerjakan ialah membuatnya.

```
pStorage=CvCreateMemStorage(0);
```

dan me-*release*-nya ketika telah selesai.

```
cvReleaseMemStorage(&pStorage);
```

3. Jika Anda sering me-load data dari file, tentu ada kemungkinan salah path. Sebaiknya berikan pengecekan untuk memastikan file di-load dengan benar.

```
if(!pInpImg || !pStorage || !pCascade)
{
printf ("Inisialisasi gagal \n");
}
exit (-1);
}
```

4. Untuk menjalankan detektor, panggil obyek cvHaarDetect. Fungsi ini membutuhkan 7 parameter. Tiga yang pertama adalah pointer image, XML data, dan memori buffer. Yang sisanya diset pada default C++.

```
pFaceRectSeq =cvHaarDetectObjects
(pInpImg, pCascade, pStorage,1.1,
//tingkatkan skala pencarian dengan 10% tiap passing
3,  //drop group yang kurang dari 3 deteksi
CV_HAAR_DO_CANNY_PRUNNING //skip region yang tidak berisi
                          //wajah
cvSize(0,));  //gunakan XML default untuk skala pencarian
              //terkecil.
```

5. Untuk membuat display Window, gunakan cvNamedWindow seperti berikut:

```
cvNamedWindow ("Haar Window", CV_WINDOW_AUTOSIZE);
```

Untuk memasukkan image ke display, panggil fungsi cvShowImage() dengan nama yang telah dibuat pada window dan nama image yang ingin ditampilkan.

HaarDetection.cpp

```
//Deteksi wajah dengan Haar
#include "stdafx.h"
#include "cv.h"
```



```
#include "highgui.h"

CvHaarClassifierCascade* load_object_detector( const char*
cascade_path )
{
    return (CvHaarClassifierCascade*)cvLoad( cascade_path );
}

void detect_and_draw_objects( IplImage* image,

CvHaarClassifierCascade* cascade,int do_pyramids )
{
    IplImage* small_image = image;
    CvMemStorage* storage = cvCreateMemStorage(0);
    CvSeq* faces;
    int i, scale = 1;
    if( do_pyramids )
    {
    small_image = cvCreateImage( cvSize(image-
    >width/2,image->height/2), IPL_DEPTH_8U, 3 );
    cvPyrDown( image, small_image, CV_GAUSSIAN_5x5 );
    scale = 2;
    }

    /* gunakan fastest variant */
    faces = cvHaarDetectObjects( small_image, cascade,
storage, 1.2, 2, CV_HAAR_DO_CANNY_PRUNING );

    /* gambar kotak */
    for( i = 0; i < faces->total; i++ )
    {
        /* extract the rectangles only */
        CvRect face_rect = *(CvRect*)cvGetSeqElem( faces, i
);

cvRectangle( image,
cvPoint(face_rect.x*scale,face_rect.y*scale),

cvPoint((face_rect.x+face_rect.width)*scale,
(face_rect.y+face_rect.height)*scale),
                        CV_RGB(255,0,0), 3 );
    }

    if( small_image != image )
        cvReleaseImage( &small_image );
    cvReleaseMemStorage( &storage );
}
```

```
int main( int argc, char** argv )
{
    IplImage* image;
    if( argc==3 && (image = cvLoadImage( argv[1], 1 )) != 0
)
    {
        CvHaarClassifierCascade* cascade =
        load_object_detector(argv[2]);
        detect_and_draw_objects( image, cascade, 1 );
        cvNamedWindow( "test", 0 );
        cvShowImage( "test", image );
        cvWaitKey(0);
        cvReleaseHaarClassifierCascade( &cascade );
        cvReleaseImage( &image );
    }

    return 0;
}
```

Jalankan dengan perintah:

Haardetection.exe fira.JPG haarcascade_frontal_Face.xml

Gambar 9.5 *Hasil Deteksi Wajah Imut*

Jika ingin mendeteksi wajah pada video maka dapat digunakan contoh berikut:

Video.cpp:

```
//Program deteksi wajah pada video AVI
#include "cv.h"
#include "highgui.h"
int main( int argc, char** argv )
{
static CvMemStorage* storage = 0;
static CvHaarClassifierCascade* cascade = 0;
CvCapture* capture = 0;
int optlen = strlen("--cascade=");
if( argc != 3 || strncmp( argv[1], "--cascade=", optlen ))
return -1;
cascade = (CvHaarClassifierCascade*)cvLoad( argv[1] + optlen
);
capture = cvCaptureFromAVI( argv[2] );
if( !cascade || !capture ) return -1;
storage = cvCreateMemStorage(0);
cvNamedWindow( "Video", 1 );
for(;;) {
IplImage* frame = cvQueryFrame( capture ), *img;
CvSeq* faces;
if( !frame )
break;
img = cvCloneImage(frame); img->origin = 0;
if( frame->origin )
cvFlip(img,img);
cvClearMemStorage( &storage );
faces = cvHaarDetectObjects( img, cascade, storage,
1.1, 2, CV_HAAR_DO_CANNY_PRUNING, cvSize(20, 20) );
for( int i = 0; i < (faces ? faces->total : 0); i++ ) {
CvRect* r = (CvRect*)cvGetSeqElem( faces, i );
cvRectangle( img, cvPoint(r->x,r->y),
cvPoint(r->x+r->width,r->y+r->height), CV_RGB(255,0,0), 3 );
}
cvShowImage( "Video", img );
cvReleaseImage( &img );
if( cvWaitKey(10) >= 0 ) break;
}
cvReleaseCapture( &capture );
return 0;
}
```

$./facedetect --
cascade=opencv/data/haarcascades/haarcascade_frontalface_alt2.xml video.avi

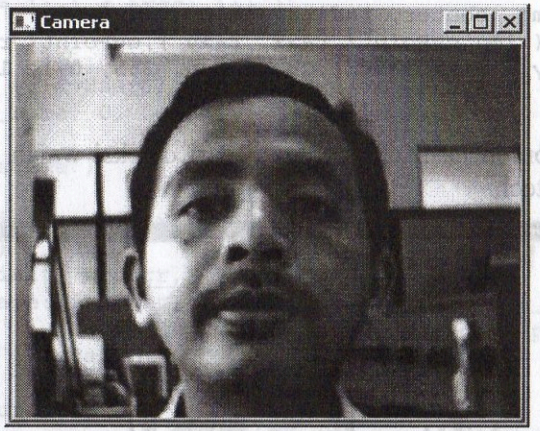
9.5 KAMERA DAN VIDEO

Jika Anda ingin mendeteksi webcam dan mengambil streaming video dari webcam tersebut, gunakan demo kode berikut ini, di mana fungsi yang umum digunakan ialah:

```
pCapture = cvCaptureFromCAM( CV_CAP_ANY );

          atau

CvCapture* cvCreateCameraCapture( int index );
```

Kamera.cpp:

```
//Menampilkan image menggunakan Webcam
#include "stdafx.h"
#include "cv.h"
#include "highgui.h"

int main( int argc, char** argv ) {
    cvNamedWindow( "Camera", CV_WINDOW_AUTOSIZE );
    CvCapture* capture;
    if (argc==1) {
     capture = cvCreateCameraCapture( 0 ); //kamera pertama
    } else {
        capture = cvCreateFileCapture( argv[1] );
    }
    assert( capture != NULL );

    IplImage* frame;
    while(1) {
        frame = cvQueryFrame( capture );
        if( !frame ) break;
        cvShowImage( "Camera", frame );
        char c = cvWaitKey(10);
        if( c == 27 ) break;
    }
    cvReleaseCapture( &capture );
    cvDestroyWindow( "Camera" );
}
```

Gambar 9.6 *Hasil Penggunaan Webcam*

Kadangkala input image mengandung noise. Oleh sebab itu pada robotics vision, jika kita ingin memfilter input image untuk mereduksi noise lalu mencari edge detection pada image input, kita dapat menggunakan operasi erosi dan dilasi, lalu memanggil operasi edge detection CANNY yang umum digunakan, yaitu sebagai berikut:

```
cvErode(dst1,dst1,NULL,2);//image processing
cvDilate(dst1,dst1,NULL,2);
//deteksi tepi
cvCanny(dst1,dst1,20,20,3);
```

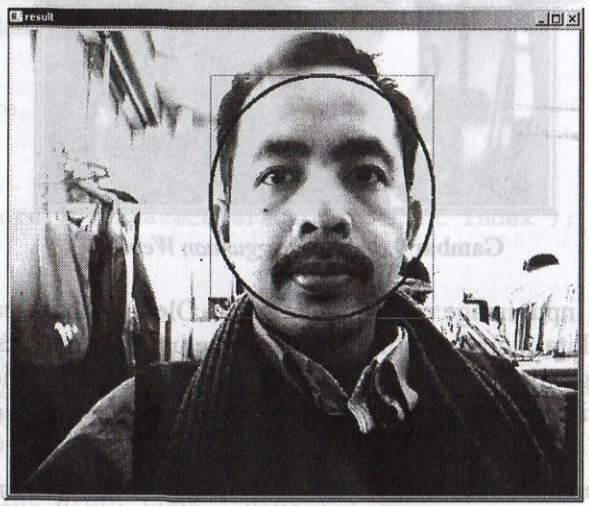
OpenCV menggunakan Haar classifier yang tersimpan dalam file XML untuk mendeteksi wajah dan bagian tubuh manusia, misalnya tersimpan di drive d:\ sebagai berikut:

```
const char* cascade_name =
"d:/haarcascade_frontalface_alt.xml";
const char* nested_cascade_name =
    "d:/haarcascades/haarcascade_eye.xml";
```

Menampilkan hasil wajah yang terdeteksi pada kamera webcam dengan garis circle dan kotak sering dibutuhkan, karena dapat digunakan untuk mengukur jarak antara kamera dan obyek sebagai pengganti sensor jarak ultrasonik. Kode berikut dapat digunakan:

```
cvCircle( img, center, radius, color, 3, 8, 0 );
cvRectangle( img,cvPoint( r->x, r->y ),cvPoint( r->x + r-
>width, r->y + r->height ),CV_RGB( 0, 255, 0 ), 1, 8, 0 );
```

Berikut ini contoh hasil dan program dari deteksi wajah menggunakan webcam Logitech:



Gambar 9.7 *Wajah yang terdeteksi*

Deteksi.cpp:

```
//Program deteksi dan tracking wajah
CvCapture* capture = 0;
static CvMemStorage* storage = 0;
static CvHaarClassifierCascade* cascade = 0;
int use_nested_cascade = 0;
void detect_and_draw( IplImage* image );
IplImage* img;

const char* cascade_name =
"d:/haarcascade_frontalface_alt.xml";
int main( int argc, char** argv )
{
    IplImage *frame, *frame_copy = 0;
    IplImage *image = 0;
    const char* scale_opt = "--scale=";
    int scale_opt_len = (int)strlen(scale_opt);
```



```
    for( i = 1; i < argc; i++ )
    {

    cascade = (CvHaarClassifierCascade*)cvLoad(
    cascade_name, 0, 0, 0 );

    if( !cascade )
    {
        fprintf( stderr, "ERROR: Could not load classifier
cascade\n" );
        fprintf( stderr,
        "Usage: facedetect [--cascade=\"<cascade_path>\"]\n"
        "   [--nested-cascade[=\"nested_cascade_path\"]]\n"
        "   [--scale[=<image scale>\n"
        "   [filename|camera_index]\n" );
        return -1;
    }
  storage = cvCreateMemStorage(0);
  if( !input_name || (isdigit(input_name[0]) &&
input_name[1] == '\0') ){
capture = cvCreateCameraCapture(1);//konfigurasi resolusi
 cvSetCaptureProperty(capture,
CV_CAP_PROP_FRAME_WIDTH,(double)640);
cvSetCaptureProperty(capture,
CV_CAP_PROP_FRAME_HEIGHT,(double)480);
      }
    cvNamedWindow( "result", 1 );
    if( capture )
    {

        for(;;)
        {
            if( !cvGrabFrame( capture ))
                break;
            frame = cvRetrieveFrame( capture );
            if( !frame )
                break;
            if( !frame_copy )
frame_copy = cvCreateImage( cvSize(frame->width,frame-
>height),IPL_DEPTH_8U, frame->nChannels );
            if( frame->origin == IPL_ORIGIN_TL )
                cvCopy( frame, frame_copy, 0 );
            else
                cvFlip( frame, frame_copy, 0 );

            detect_and_draw( frame_copy );

            if( cvWaitKey( 10 ) >= 0 )
```

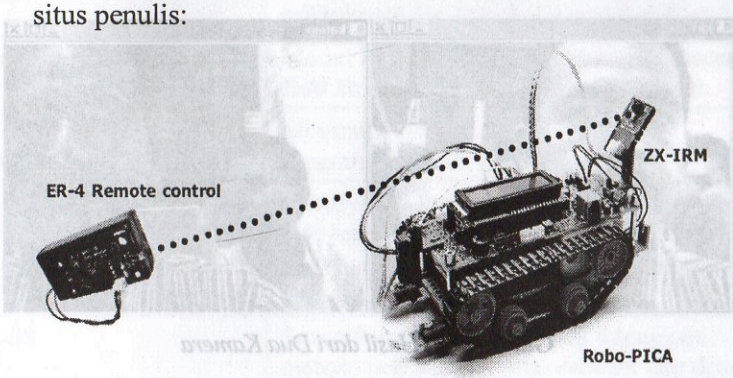
```
                goto _cleanup_;
        }
        cvWaitKey(0);
        _cleanup_:
        cvReleaseImage( &frame_copy );
        cvReleaseCapture( &capture );
    }
    cvDestroyWindow("result");

    return 0;
}

void detect_and_draw( IplImage* img )
{
    static CvScalar colors[] =
    {
        {{0,0,255}},
        {{0,128,255}},
        {{0,255,255}},
        {{0,255,0}},
        {{255,128,0}},
        {{255,255,0}},
        {{255,0,0}},
        {{255,0,255}}
    };

    IplImage *gray, *small_img;
    int i, j;

gray = cvCreateImage( cvSize(img->width,img->height),8, 1 );
small_img = cvCreateImage( cvSize( cvRound (img-
>width/scale),
    cvRound (img->height/scale)), 8, 1 );
    cvCvtColor( img, gray, CV_BGR2GRAY );
    cvResize( gray, small_img, CV_INTER_LINEAR );
    cvEqualizeHist( small_img, small_img );
    cvClearMemStorage( storage );

if( cascade )
{
double t = (double)cvGetTickCount();
CvSeq* faces = cvHaarDetectObjects( small_img, cascade,
storage, 1.1, 2, 0
|CV_HAAR_FIND_BIGGEST_OBJECT,
        cvSize(30, 30) );
        t = (double)cvGetTickCount() - t;
```



```
        printf( "detection time = %gms\n",
t/((double)cvGetTickFrequency()*1000.) );
        for( i = 0; i < (faces ? faces->total : 0); i++ )
        {
            CvRect* r = (CvRect*)cvGetSeqElem( faces, i );
            CvMat small_img_roi;
            CvSeq* nested_objects;
CvPoint center;
CvScalar color = colors[i%8];
int radius;
center.x = cvRound((r->x + r->width*0.5)*scale);
center.y = cvRound((r->y + r->height*0.5)*scale);
radius = cvRound((r->width + r->height)*0.25*scale);
cvCircle( img, center, radius, color, 3, 8, 0 );
cvRectangle( img,cvPoint( r->x, r->y ),cvPoint( r->x + r-
>width, r->y + r->height ),CV_RGB( 0, 255, 0 ), 1, 8, 0 );
cvShowImage( "result", img );
cvReleaseImage( &gray );
cvReleaseImage( &small_img );
}
```

Latihan:

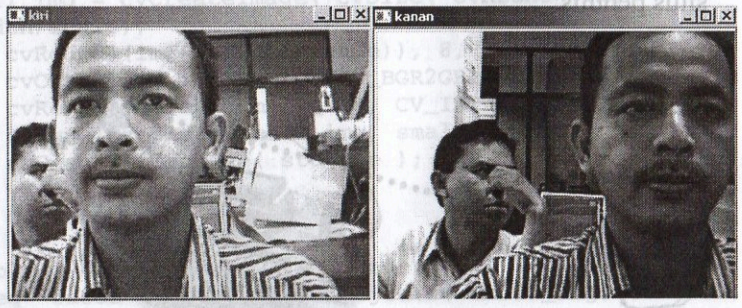
1. Jelaskan cara kerja face recognition serta beberapa teknik yang umum digunakan.
2. Kembangkan robot tank yang dikendalikan remote dan dapat memberikan informasi daerah yang dijelajahi dengan menggunakan vision seperti contoh. Kode program dapat diunduh di situs penulis:



Gambar 9.8 *Robot Tank RoboPICA2.0*

3. Kembangkanlah demo kode menggunakan stereo vision berbasis webcam untuk navigasi robot. Stereo vision mampu mendeteksi tingkat kedalaman dari suatu image. Berikut cuplikan contoh kode untuk untuk menampilkan stereo vision pada webcam:

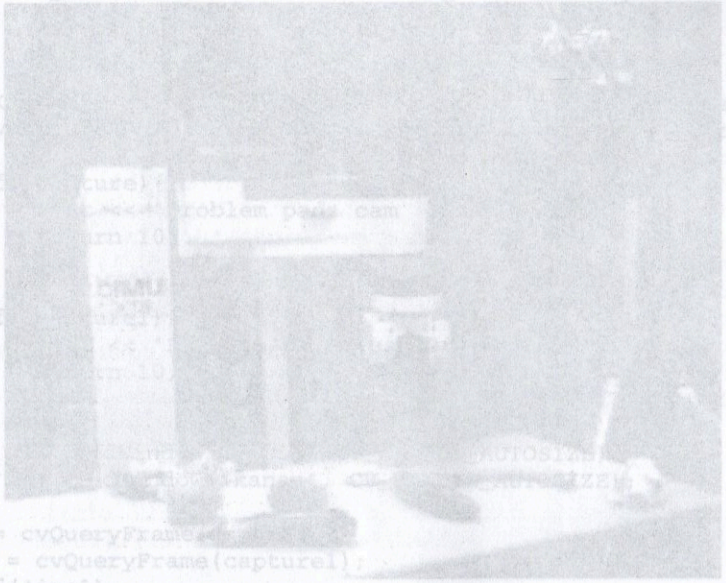
```
CvCapture* capture = cvCreateCameraCapture(1);//kanan
 CvCapture* capture1 = cvCreateCameraCapture(0);//kiri

 if(!capture){
     cout << "Problem pada cam  1." << endl;
     return 10;
 }
 if(!capture1){
     cout << "Problem pada  cam 2." << endl;
     return 10;
 }

     cvNamedWindow("kiri", CV_WINDOW_AUTOSIZE);
     cvNamedWindow("kanan", CV_WINDOW_AUTOSIZE);

img = cvQueryFrame(capture);
img1 = cvQueryFrame(capture1);
   if(!img1)
    return 20;

while(key!='q')
   {
cvShowImage("kiri",img1);
cvShowImage("kanan",img);
```



Gambar 9.9 *Hasil dari Dua Kamera*

4. Kembangkan stereo vision untuk mengenal obyek pada robot manipulator seperti contoh berikut:

Gambar 9.10 *Manipulator Diperbantukan Stereo Vision*



Gambar 9.9 *Hasil dari Dua Kamera*

LAMPIRAN

Kit Penunjang Praktikum Robotika

Kit	Penjelasan
SmartAVR ver 4.0	Kit mikro AVR Atmega8535 dan berbagai fitur menarik untuk belajar
SmartAVR Robotics Ver 4.0	Kit mikro AVR Atmega16 dan berbagai fitur menarik untuk belajar robotika dasar
D-Sonar	Ultrasonic Interface, kompatibel SRF-04 + UART Protocol
D-Voice 04	Voice Interface dengan durasi 8 / 16 menit
Delta IR Line Sensing	Penjejak garis berupa sungut yang dapat diatur tebal tipis maupun responsnya
Xbee Pro Wireless	Modul wireless lengkap dengan antena dan daya 60mW
D-Joy Controller	Modul antarmuka 4 Joystick Playstation dengan keluaran UART
D-Joy Controller (Wireless Version)	Modul antarmuka 4 Joystick Playstation dengan keluaran RF ke DX-24
DU-ISP V2.0	Downloader ISP untuk berbagai macam mikrokontroler Atmel (Kompatibel Vista)
Delta DC Driver	Modul Pengendali 2 buah Motor DC secara Half Bridge + pelindung kondisi inhibit
ST-8535 + Delta USB ISP Programmer	Minimum system AVR 8535 dengan programmer USB ISP
SST-06	Sub-System Stepper Interface yang dapat mengendalikan motor stepper bipolar maupun unipolar secara manual atau berdasar perintah dari UART
DSR-08	Smart Servo Controller yang dilengkapi metode perhitungan nilai encoder dan deteksi kondisi kegagalan mekanik

Spider Leg Mechanic	Kaki robot laba-laba dengan ruas-ruasnya
DU-232	Antarmuka USB to RS232
KIT ROBOT PELENGKAP UTAMA	
Robot Tank RoboPICA2.0	
Robot Avoider AVR 8535	
SmartArm Robot 4DOF	
Robot Line Follower AVR 8535	
Robot Pemadam Api KRCI	
ROBOT BRAT	
SG6 Arm Robot 5 axis	
Lego Mindstrom NXT	
Robot People Follower berbasis Vision dan lainnya	
Silahkan order di www.toko-elektronika.com	

robotika
teori + implementasi

- Pengantar robotika
- Mekanika robotika
- Pemrograman robotika
- Robotika vision

Buku *Robotika, Teori dan Implementasi* ini sangat tepat dibaca oleh pelajar, mahasiswa dan peneliti di bidang robotika. Buku yang memberikan gambaran lengkap konsep robot dan implementasinya ini layak dimiliki karena selain menggunakan teknologi robot terkini, dibahas pula program *computer vision* untuk pengolahan citra dan pengenalan wajah pada robot menggunakan OpenCV. Buku ini merupakan satu-satunya buku yang membahas konsep dan implementasi robotika berbasis vision yang paling lengkap di Indonesia.

Widodo Budiharto ialah praktisi dan akademisi di bidang TI pada BINUS University, konsultan senior untuk robotika dan *embedded system* di Indonesia, serta memiliki situs e-commerce www.toko-elektronika.com dan robotika-vision.com. Berbagai pengalaman skala internasional dalam riset dan studi robotika telah dilaluinya, antara lain di institusi riset Prancis, Mexico, dan Jepang. Beliau akan terus mendedikasikan diri membangun bangsa melalui buku-bukunya, dapat dihubungi di widodo@widodo.com.

HARDWARE
ISBN: 978-979-29-1440-5

Penerbit ANDI
Jl. Beo 38-40 Yogyakarta
Telp. (0274) 561881 Fax. (0274) 588282
e-mail: penerbit@andipublisher.com
website: www.andipublisher.com

Dapatkan Info Buku Baru, Kirim E-mail: info@andipublisher.com